
UNIT 1 INTRODUCTION TO PYTHON

Structure

- 1.0 Introduction
- 1.1 Objectives
- 1.2 History of Python
- 1.3 Need for Python
- 1.4 Packages for cross-platform application of Python
- 1.5 Getting Started with Python
- 1.6 Program Structure in Python
- 1.7 Running the First Program
- 1.8 Summary
- 1.9 Answer to Check your Progress
- 1.10 References/Further Readings

1.0 INTRODUCTION

Python is an easy-to-learn programming language. It is a powerful programming language and is widely used in Artificial Intelligence, Machine Learning, Neural Networks and many other advanced fields of Computer Science. Ideally, it is designed for rapid prototyping of complex applications. Python interfaces with various operating system calls and libraries and can be extended with C, C++, or Java. Many large companies like NASA, Google, Meta, Netflix, YouTube, BitTorrent, ISRO, etc. use Python to run their projects.

To build a career in IT and software development, programming is a basic requirement, but before you start learning, you should be clear about which programming language to choose. Before learning any programming language, you should figure out which language suits you. To date, if you compare C, C++, Java, and Python programming languages, it may help you understand that Python is relatively more popular and a popular choice in the IT Industry. In this course, you will learn Python programming in detail. In this unit, you will learn how to write basic Python programs and run them. A basic comparison of C and Python programming languages is given for your understanding.

Metrics	Python	C
Introduction	It is a high-level, general-purpose & interpreted programming language.	C is a general-purpose procedural programming language.
Speed	As an interpreted programming language, it runs slower than a compiled programming language (i.e., C).	As a compiled programming language, it runs faster than an interpreted programming language (i.e., Python).
Usage	The number of lines of code written in Python is quite less in comparison to C	C syntax is more complicated than Python.

Declaration of variables	Declaration of Variable type is not required; they are untyped, and a given variable can store different types of values at different instances during the execution of any Python program.	In C, declaration of variable type is a must, and it is done at the time of its creation, and only values of that declared type must be assigned to the variables.
Error Debugging	Error debugging is simple because it executes only one instruction at a time. Compilation and execution are performed simultaneously. Errors are instantly shown, and execution stops at that instruction only.	Because C is a compiler-dependent language, error debugging is difficult; it takes the entire source code, compiles it, and then shows all errors.
Function renaming mechanism	The same function can be used by two different names i.e. it supports the mechanism of function renaming	C does not support function renaming; the same function cannot be used under two different names.
Complexity	The syntax of Python programs is quite simple, easy to read, write and learn	The syntax of a C program is comparatively harder than the syntax of Python.
Memory-management	It supports automatic garbage collection for memory management.	In C, the Programmer has to manage memory explicitly.
Applications	Python is a General-Purpose programming language that can also be used for microcontrollers.	C is generally used for hardware-related applications.
Built-in functions	The library of built-in functions is quite large in Python.	The library of built-in functions is quite limited in C
Implementing Data Structures	Gives ease of implementing data structures with built-in insert and append functions.	Explicit implementation of functions is required for the implementation of data structures
Pointers	Python does not support pointers.	Pointers are frequently used in C.

Python is a high-level, general-purpose, interpreted programming language. It is dynamically typed and garbage-collected and supports multiple programming paradigms, including structured (particularly procedural), object-oriented, and functional programming; because of its rich set of standard libraries, Python is often described as a "batteries included" language.

1.1 OBJECTIVES

After going through this unit, you will be able to:

- Describe the history of Python,
 - Explain the need for Python as a programming language,
 - Describe cross-platform applications,
 - Explain the structure of a Python program, and
 - Write and execute Python code.
-

1.2 HISTORY OF PYTHON

Python was conceived in the late 1980s as a successor to the ABC language. It was created by Guido van Rossum in 1989, and its first release was in 1991. Later, Python 2.0, released in 2000, added features like list comprehensions and a garbage collection system that could collect reference cycles. Python 2 implemented the technical details of the Python Enhancement Proposal (PEP), simplifying the code development complexities of earlier versions. But because of stability issues with Python 2.x, Python 2.7 (the last version in 2.x, released in 2010) was discontinued in 2020. Developers resolved issues with Python 2 in parallel, and in 2008, Python 3.0 was released. It was a major revision of the language; Python 3 was mainly released to fix problems in Python 2. As Python 3 was not completely backwards-compatible, much of the Python 2 code does not run unmodified on Python 3. To make migration to Python 3 easier, some Python 3 features were backported to Python 2.x.

Thus, migrating a project from Python 3 to Python 2.x requires many changes, not only to the project and its applications but also to the libraries that make up the Python environment.

As of June 2026 , Python 3.14.6 was released. You can freely download the latest version of Python from <https://www.python.org/downloads/> .

A global community of programmers develops and maintains C-Python, which is an open-source reference implementation. A non-profit organisation, the Python Software Foundation, manages and directs resources for Python and C-Python development. So, what is C-python? It is the default or reference implementation of the Python programming language, and as the name suggests, C-Python is written in the C language. Some of the implementations which are based on the C-Python runtime core but with extended behaviour or features in some aspects are Stackless Python and MicroPython; Stackless Python relates to C-Python with an emphasis on concurrency using tasklets and channels (used by ds-python for the Nintendo DS), and MicroPython relates to working with microcontrollers.

C-Python compiles the Python source code into intermediate bytecode, which is executed by the C-Python virtual machine. It is to be noted that Python is a dynamic programming language. Still, it is said to be slow, because the default C-Python implementation compiles the Python source code in bytecode, which is slow as compared to machine code(native code).C-Python is distributed with a large standard library written in a mixture of C and Python, C-Python provides the highest level of compatibility with Python packages and C extension modules.

Further, Jython, IronPython, and PyPy are also implementations of Python; they also provide a good level of compatibility with Python packages. These implementations are based on Java, .NET and Python, respectively. We will briefly discuss these implementations, starting with Jython.

Jython is actually the Java platform-based implementation of the Python programming language, so that you can run Python programs on the Java platform. Programs written in Jython use Java classes, not Python modules. The Jython compiler compiles the Jython programs into Java bytecode, which can then be run by the Java virtual machine. Jython lets Python programs use Java class library functions. Compared with CPython, Jython is slower and less compatible with CPython libraries. However, the compatibility in IronPython and PyPy is quite good.

IronPython is the implementation of Python written in C# (Microsoft's .NET framework). As Jython used Java Virtual Machine (JVM), IronPython uses .Net Virtual Machine i.e., the Common Language Runtime. The .NET Framework and Python libraries are used efficiently by the IronPython. Further, because of its Just-in-Time (JIT) compiler and lack of a Global Interpreter Lock, IronPython performs better for Python programs that require threads or multiple cores.

Guido van Rossum (creator of Python) said “If you want your code to run faster, you should probably just use PyPy.” —PyPy is an implementation of the Python programming language written in Python itself. The PyPy Interpreter is written in RPython, which is a subset of Python. The PyPy interpreter uses a JIT compiler to compile source code into native machine code, making it very fast and efficient. PyPy also supports stackless mode by default, providing micro-threads for massive concurrency.

1.3 NEED FOR PYTHON

Python is a high-level, interpreted, and dynamically typed programming language. Python requires fewer lines of code compared to languages like Java or C++. This encourages the rapid development of application prototypes, makes debugging of errors easy and identifies itself as the language to code with. As Python is an interpreted language, Python code is executed line by line, allowing you to test changes instantly without waiting for compilation.

Now, you might be in a position to appreciate the simplicity and capability of Python as a programming language. It is a general-purpose language and applies to almost every domain of software development, whether web development, scientific applications, business software, or anything else you can think of. It has wider coverage of applications, and is widely used as it complies with the current needs of the software industry. Thus, it offers a promising future for learners.

It is easy to understand the need of Python if you understand the application areas of Python. You might not know that various applications like YouTube, BitTorrent, DropBox etc. uses Python to achieve their functionality. The reason behind the choice of python, for these applications is the ability of Python to be compatible with cross-platform operating systems (a cross-platform application may run on any operating system, be it Microsoft Windows or Linux, or macOS or any other), which simplifies the development of applications.

Now let us discuss the various types of applications that can be developed using Python; many applications can be built, and a few are given below.

- **Artificial Intelligence(AI) and Machine Learning(ML):** These are the need of the times, as they yield the most promising careers for the learners. AI and ML involve incorporating intelligence into machines through self-learning from data stored in the system; various algorithms support this. But to bring AI and ML into action, Python is the first choice. Why? Because various well-equipped libraries like Pandas, Scikit-Learn, NumPy, etc. are available to facilitate engineers and scientists. You just learn the algorithms, use the library, and you have your solution to the problem. It is that simple.
- **Data Science and Data Visualization:** Data Science is just another promising career; if you know how to extract relevant information from the data source, then the world is yours. But it's not easy; you need various algorithms and techniques to solve the problem. To simplify your work, Python again emerges to help you study your data, perform operations, and extract the required information. Libraries such as Panda and NumPy help you extract information. Further, libraries such as Matplotlib and Seaborn are used for data visualization. These libraries are quite helpful in plotting and visualization of data. This is how Python helps you to become a Data Scientist.
- **Mobile Applications:** In general, people think that Android and iOS are for mobile development, but what about using Python for mobile app development? Historically, Python did not have a strong track record for mobile GUI development, which was questionable since Python is positioned as a solution for a variety of applications. But over time, Python has evolved, and now it is one of the options for Mobile App development. Two well-known frameworks, i.e. Kivy and BeeWare, are options for mobile application development with Python.
- **Embedded Applications and IoT:** We learned from earlier sections of this unit that Python is based on C. Thus, it can be used to create software for embedded applications. MicroPython is a software implementation of Python 3, written in C. In fact, MicroPython is a Python compiler that runs on microcontroller hardware; it includes modules to provide access to low-level hardware. One renowned embedded application is the Raspberry Pi, which uses Python for computing. It can serve as a computer or a simple embedded board for high-level computations; you can use it for IoT or Mobile applications to build smart gadgets. Nowadays, Python is widely used in embedded and IoT application development across microcontrollers, Single-Board Computers (SBCs), and testing environments because of its rapid prototyping capabilities and massive ecosystem.
- **Web Development:** In recent years, Python has dominated web development due to its rich ecosystem of full-stack and micro-frameworks like Django, Flask, and Pyramid, which eliminate boilerplate code by providing built-in backend logic for routing, database management, and robust security middleware. These frameworks seamlessly handle low-level internet protocols—such as managing secure HTTPS and SSL encryption layers or automating file exchanges via FTP—while leveraging Python's native libraries to dynamically parse and serialize core web data formats like JSON, Email and XML etc.

- **Game Development:** Python is rapidly expanding its footprint in the booming gaming industry as a powerful language for building interactive 2D and 3D games through specialized libraries like Pygame and Pyglet for 2D logic, and PySoy for 3D engine layouts. While high-performance commercial engines rely heavily on C++ for core rendering, Python serves as the foundational scripting engine for AI, game loops, and modding tools in major industry titles, including Civilization IV, Disney's Toontown Online, and Vega Strike etc.
- **Enterprise & Business Applications:** Python is a preferred language for building custom internal enterprise software, such as Enterprise Resource Planning (ERP) tools, Customer Relationship Management (CRM) databases, and billing systems, due to its readability and security features.
- **Python for Network Security:** Python is widely used by both offensive penetration testers and defensive security analysts to automate threat mitigation and audit network infrastructure. Using Python and advanced packet-manipulation libraries like Scapy, programmers and developers can craft, sniff, and dissect traffic down to individual protocol bits to identify open ports, forge network packets, and probe vulnerabilities.

So, you can now judge that Python has a variety of applications. No matter what field you take up, Python is rewarding. The above discussion might have helped you understand some application areas of the Python programming language.

Check Your Progress -1

1. Compare Python and C programming languages

.....

.....

.....

.....

2. Discuss different variants of Python viz. C-python, Jython, IronPython, PyPy

.....

.....

.....

.....

3. List some areas of application of the Python programming language.

.....

.....

.....

.....

1.4 PACKAGES FOR CROSS PLATFORM APPLICATION OF PYTHON

Currently, Python is one of the most popular programming languages, and it has marked its presence by contributing actively to every emerging field in computer science. It has a vast set of libraries for almost every field, such as Machine

Learning (Numpy, Pandas, Matplotlib), Artificial Intelligence (PyTorch, TensorFlow), and Game development (Pygame, Pyglet), and many more.

After going through section 1.2 of this unit, you understood the meaning of Cross-platform applications, and you are now aware of the potential of Python as a cross-platform programming language. You also came to know that the development of different types of applications requires different types of packages, libraries, modules, frameworks etc. You might be wondering: what is the difference between these terms? Are they the same or different? Let us clear up this confusion first, and then we will briefly discuss the functionality of different packages used to develop various Python applications.

Python: Framework Vs Library Vs Package Vs Module

A framework is a collection of libraries. It defines the program's architecture.

A library is a collection of packages. (*We may understand that a Python library or framework is a pre-written program that is ready to use on common coding tasks.*)

A package is a collection of modules. It must contain an `__init__.py` file as a flag so that the python interpreter processes it as such. The `__init__.py` could be an empty file without causing issues. A package, in essence, is a directory that holds subpackages and modules. While we can create our own packages, we can also use one from the Python Package Index (PyPI) to use for our projects. To import a package, we **type `import Game.Sound.load`** Or we can also import it, giving it an alias **:`import Game.Sound.load as load game`**.

A module is a file which contains various Python functions and global variables. It is simply a **.py file** that contains executable Python code. We put similar code together in one module. This helps us modularise our code and makes it much easier to manage. A module also provides reusability. With a module, we do not need to rewrite the same code for a new project.

Now let us see how a module and a package differ:

1. A module is a file containing Python code. A package, however, is like a directory that holds sub-packages and modules.
2. A package must hold the file `__init__.py`. This does not apply to modules.
3. To import everything from a module, we use the wildcard `*`. But this does not work with packages.

We will learn more about them, as we proceed in this course. To start, we will discuss various Frameworks and Libraries first, and then you will learn how to use them, including methods and packages. Python has many popular libraries and frameworks. Some of these libraries/frameworks, such as NumPy, Pandas, Kivy, and Matplotlib, you will learn in detail in this course.

Some Popular Python Frameworks/Libraries:

NumPy: A library for scientific computing, also known as Numerical Python. Almost all Python machine-learning tools like Matplotlib, SciPy, Scikit-learn, etc. rely on this library to a reasonable extent. It comes with functions for dealing with complex mathematical operations like linear algebra, Fourier transformation, random numbers, along with features that work with matrices and n-arrays in Python. It is widely used in handling sound waves, images, and other binary functions.

Pandas: It is an open-source Python library that offers a wide range of tools for data analysis and manipulation. With Pandas, data from a broad range of sources like CSV, SQL databases, JSON files, and Excel can be read. Data analysis and manipulation is a prerequisite for most of the machine learning projects, where a significant amount of time is spent analysing the trends and patterns hidden in the datasets. Using Pandas, one can manage complex data operations with just one or two commands; it comes with several inbuilt methods for data handling, and it also serves as the starting point to create more focused and powerful data tools.

Kivy: It is an open-source Python Framework used for cross-platform Python GUI mobile app development. It lets you write pure-Python graphical applications that run on desktop platforms (Windows, Linux, macOS, and Raspberry Pi) and on iOS & Android.

Django: A web framework built in Python. It encourages rapid web application development. Developers can focus on writing the app only. Its primary goal is to ease the creation of complex, database-driven websites. Django emphasises code reusability and pluggability of components; thus, it supports less code, low coupling, and helps in rapid website development .

Matplotlib : This library is specifically meant for Data Science; it helps to generate data visualization like 2D diagrams and graphs like histograms, scatter plots, and even graphs for non-cartesian coordinates. It provides an object-oriented API that lets you embed plots directly into applications. This library helps Python compete with scientific tools like MATLAB and Mathematica. It is quite compatible with the popular plotting libraries, but developers are required to write more code than usual, while using this library for generating advanced visualizations.

TensorFlow: TensorFlow is an end-to-end Python machine learning library for performing high-end numerical computations. TensorFlow can handle Deep Neural Networks for image recognition, handwritten digit classification, recurrent neural networks, and NLP (Natural Language Processing).

Keras: A leading open-source Python library for developing neural networks and Machine Learning projects. It simplifies designing and developing Neural Networks for beginners in Machine Learning. Keras also supports Convolutional Neural Networks (CNNs) and Recurrent Neural Networks (RNNs), which are essential in deep learning. It includes algorithms for normalization, optimization, and activation layers. Rather than being an end-to-end Python machine learning library, Keras is a user-friendly, extensible interface that enhances modularity and expressiveness.

Theano : It is a library for scientific computation; it allows you to define, optimize as well as evaluate complex mathematical expressions, which deal with multidimensional arrays. Repeated computation of complex mathematical expressions underpins many AI and ML applications. Theano performs data-intensive calculations quickly; it runs almost 100 times faster than on a CPU alone. It aims to reduce the development and execution time of ML apps, particularly in deep learning algorithms.

Scikit-learn: A prominent open-source machine learning library in Python. It includes a wide range of algorithms like DBSCAN, gradient boosting, random forests, support vector machines, and k-means, which are generally used to implement various machine learning concepts, i.e., Classification, Clustering, Regression, etc. It can interoperate with numeric and scientific libraries of

Python like NumPy and SciPy. Scikit-learn supports both supervised and unsupervised ML.

SciPy: It is a library for all types of scientific programming projects. Its main functionality is built upon NumPy, and it includes modules for linear algebra, Integration, optimization, and statistics too. SciPy is supported with extensive documentation, which makes its working really easy.

Seaborn : It is an unparalleled visualization library, based on Matplotlib's foundations. Seaborn offers a high-level dataset-based interface to make amazing statistical graphics. With Seaborn, it is simple to create certain types of plots like time series, heat maps, and violin plots. The functionalities of Seaborn go beyond Pandas and matplotlib, with features to perform statistical estimation when combining data across observations, and to plot and visualise the suitability of statistical models to strengthen dataset patterns.

Scrapy : It is the most widely used Python library for data science, specifically data mining. It is used to build crawling programs i.e spider bots, which are used to retrieve structured data (example URLs or Contact Information) from the web. Developers use it for gathering data from APIs.

Pygame : It is a python programming language library for making multimedia applications like video games, animations etc. It includes libraries for computer graphics and sound, designed to be used with Python. Pygame is suitable to create client-side applications that can be potentially wrapped in a standalone executable.

MicroPython : It is a software implementation of Python 3, written in C. In fact, MicroPython is a Python compiler that runs on the hardware of micro-controller, it includes modules to provide access to low-level hardware. One renowned embedded application is the Raspberry Pi, which uses Python for computing. It can be used as a computer or as a simple embedded board to perform high-level computations; you can use it for IoT or Mobile applications, thus producing your smart gadgets.

BeeWare: A Python GUI and mobile development framework for building native Python Mobile Apps. The "BeeWare" project provides a set of tools and an abstraction layer, which can be used to write native-looking mobile and desktop applications using Python.

1.5 GETTING STARTED WITH PYTHON

Now, we have some clarity about Python as a programming language. From the previous sections, we know about Python's various libraries and frameworks. Now, we need to work on the IDEs (Integrated Development Environments) of Python; there are many IDEs like Jupyter Notebook, Spyder, VS Code, R Programming, etc. All these are collectively available in Anaconda (Anaconda is a free and open-source distribution of the Python and R-Programming languages for scientific computing (data science, machine learning applications, large-scale data processing, predictive analytics, etc.)), or you may also go for the cloud versions like Google Colab Notebook, where you do not need high-configuration hardware; only internet is required, and through your Gmail account you can work on Python using Jupyter Notebook.

Here we will briefly discuss some ways to work with Python; you may choose any or try all, and also explore other options.

1. Just browse for <https://www.python.org> and perform the following steps:

- Download the latest version of Python for the operating system installed on your computer; in this case, it's Windows 64-bit, so I have downloaded python-3.7.7 (python-3.7.7-amd64.exe) from <https://www.python.org/downloads/release/python-377/>
- Now Run this. exe file and install Python; just click Next and go ahead till the setup installation is finished
- Finally, you will have an interface for Python programming

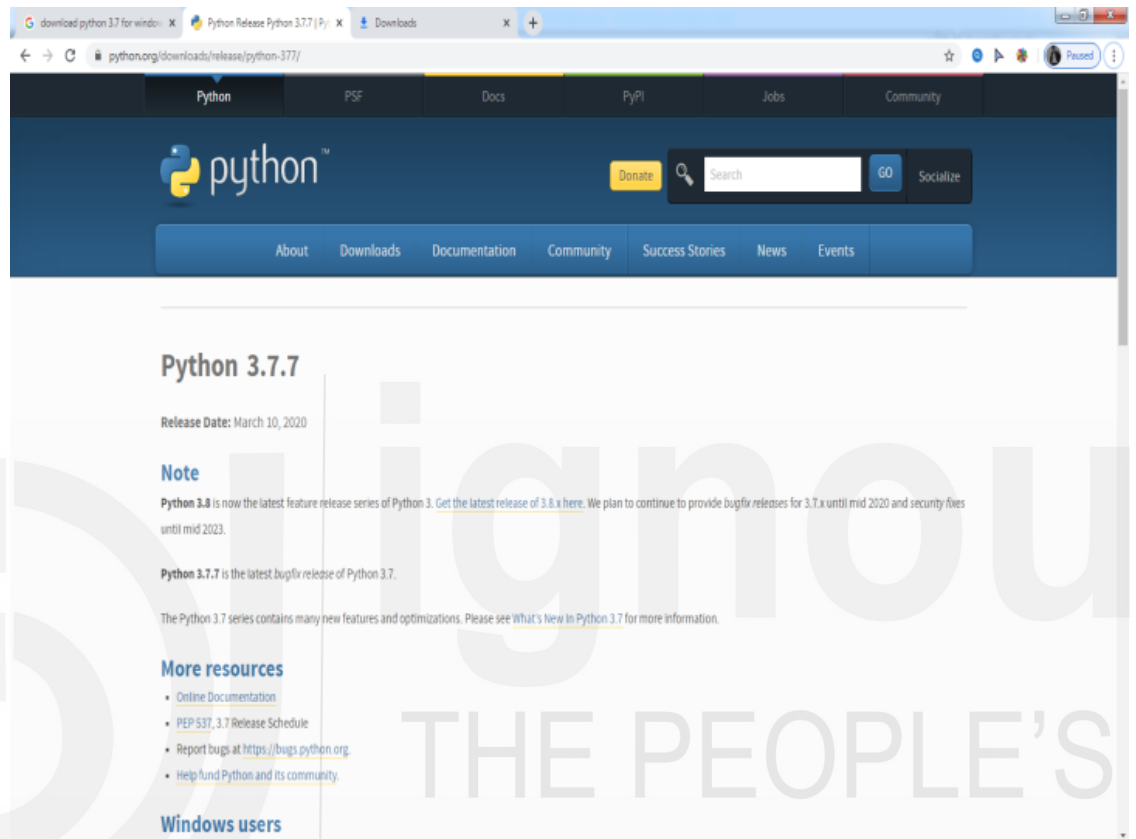


Figure 1: Downloading Python

There is a variety of IDEs for Python, like Jupyter Notebook, Spyder, VS Code, etc, and all are available at Anaconda (a free and open-source distribution of the Python and R programming languages for scientific computing (data science, machine learning applications, large-scale data processing, predictive analytics, etc.)). To start with Anaconda, just perform the following steps.

1. Just browse <https://www.anaconda.com/> and perform the following steps:
 - a) Click the Download button on the webpage of <https://www.anaconda.com/>
 - b) The distribution section <https://www.anaconda.com/distribution/> will open,
 - c) Click the download option given on this page <https://www.anaconda.com/distribution/>
 - d) <https://www.anaconda.com/distribution/#download-section> will open; the 64-bit and 32-bit graphical installer options for Python 3.x (currently 3.7) and 2.x (currently 2.7) are available under Anaconda 2020.02 for Windows installer.

- e) It is recommended to download 64bit version of Python 3.x (currently 3.7),
- f) Anaconda3-2020.02-Windows-x86_64.exe will be downloaded.
- g) Now, just run the setup of this Anaconda3-2020.02-Windows-x86_64.exe , and click next-next, till the installation is completed.
- h) Finally, you will find Anaconda Navigators shortcut on your desktop, click on it and you can start working with any IDE be it Jupyter Notebook, Spyder, VS Code etc., even you can work with R programming.

Important: Before working with IDEs, you need to understand how to work with them and which one is suitable. The following are observations, currently:

1. When you start Jupyter Notebook on Windows (by clicking on the Jupyter section given in Anaconda Navigator), a browser will open in Internet Explorer; many times, Jupyter Notebook won't work here. Just copy the URL from Internet Explorer and paste it into Google Chrome; you will find that Jupyter Notebook starts working. Other details regarding writing, saving, and execution of the program will be discussed later.
2. Program execution in Jupyter is line by line, and in VS Code and Spyder it goes sideways; even errors can also be seen sideways. Anyway all are good to work with Python.

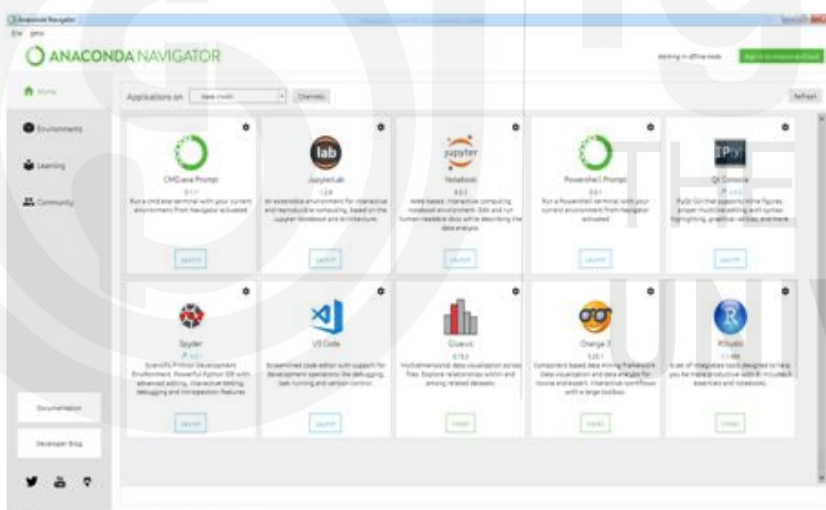


Figure 2: ANACONDA NAVIGATOR

1. Many a time, the learners may not be equipped with systems having the latest hardware configurations, as desired for the installation of Python, or there might be compatibility issues with the operating system, or maybe due to any reason you are not able to install and start your work with Python. Under such circumstances, the solution is Google Colab Notebook (<https://colab.research.google.com/notebooks/welcome.ipynb>). Use this and just log in with your Gmail account and start your work on Jupyter Notebook, as simple as that.

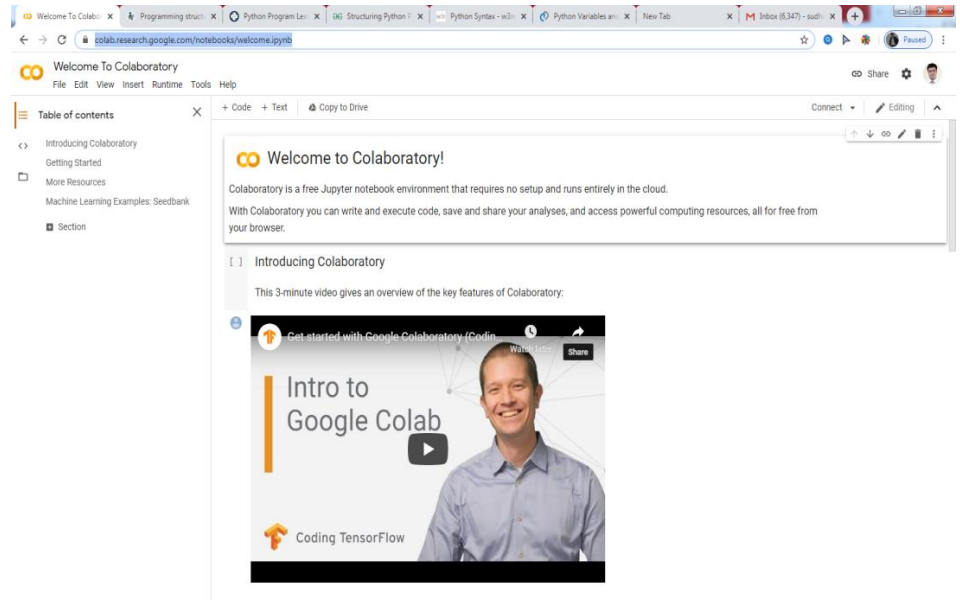


Figure 3: Google Colab

Also, you can use PyCharm, a dedicated Integrated Development Environment (IDE) for Python development built by JetBrains. It is widely considered one of the most powerful tools for Python developers. You can download it from <https://www.jetbrains.com/pycharm/download/?section=windows>.

You will see that many examples in this course material are written using PyCharm.



Figure 4: PyCharm IDE

1.6 PROGRAM STRUCTURE IN PYTHON

This section relates to the discussion over the programming structure of Python programming, i.e. the way you divide a program into your source files and mix parts from various libraries or modules or packages into a single program, i.e. directly or indirectly, a single program includes multiple files from different modules. Modules are, in fact, library tools, which are implemented to make a collection of top-level files; i.e., the high-level files may use tools defined in modules, which may use files defined in other modules. Coming to our point, in python a file takes a module to get access to the tools it defines, and also to the tools defined in other modules included in the program. In Python, a high-level file has important control of your program, where from you can start your application. Just refer to Figure 5 given below.

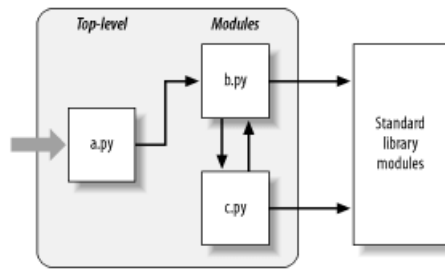


Figure 5 : Structure Python Program

To understand the structure of a Python program, you have to note that there exist three files: **a.py**, **b.py** and **c.py**. The file model **a.py** is chosen for a high-level file; it is known as a simple text file of statements. Files **b.py** and **c.py** are modules. They are also text files of statements, but they are generally not started directly; they are invoked by **a.py**, i.e., the high-level file.

This is just a general discussion over the structure of a Python program, but a Python program also includes all the components as they are in any other object-oriented programming language, such as **data types**, **conditional statements**, **looping constructs**, functions, file handling, Classes, Exception handling, Libraries, Modules, packages etc. We will discuss a few here, and cover the rest in the subsequent units of this course.

1.7 RUNNING THE FIRST PROGRAM

Click File, select New Workbook, and a new Jupyter notebook will open in Google Colab. Now you may start your work

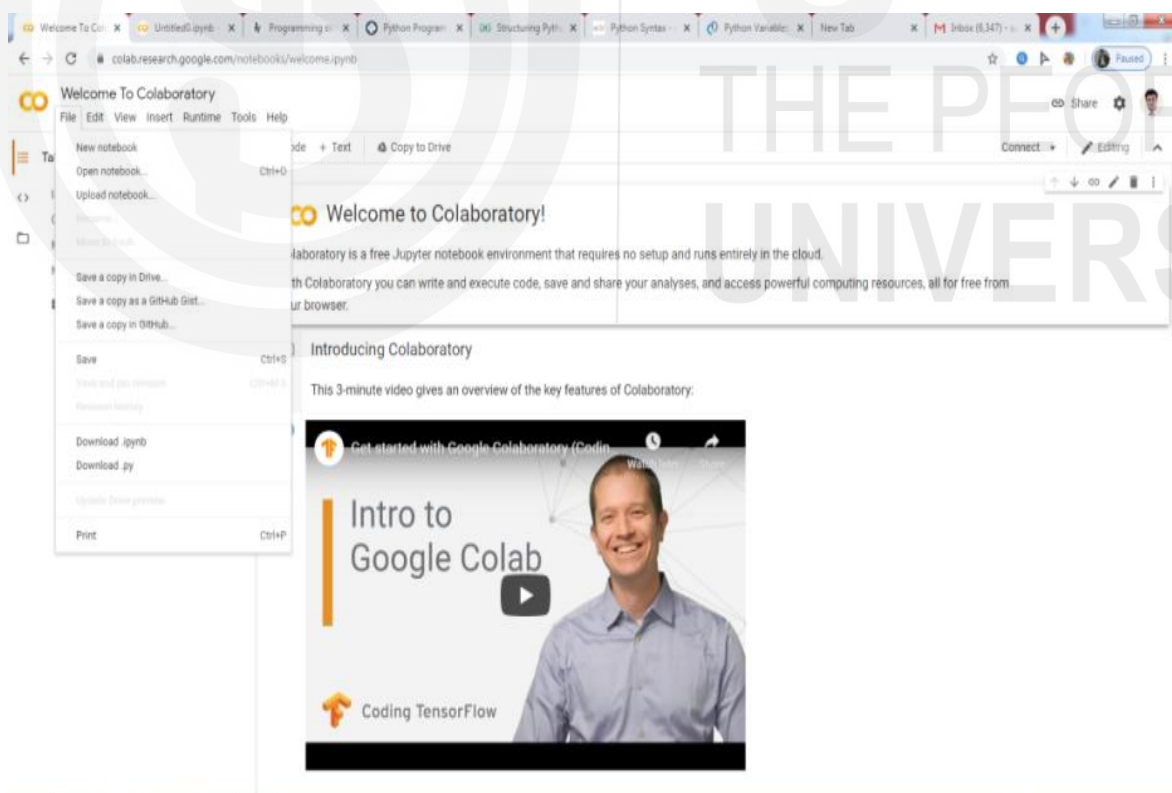
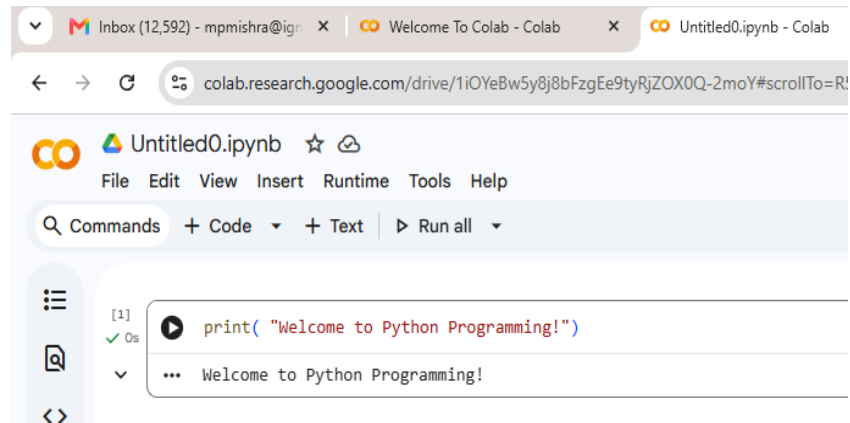


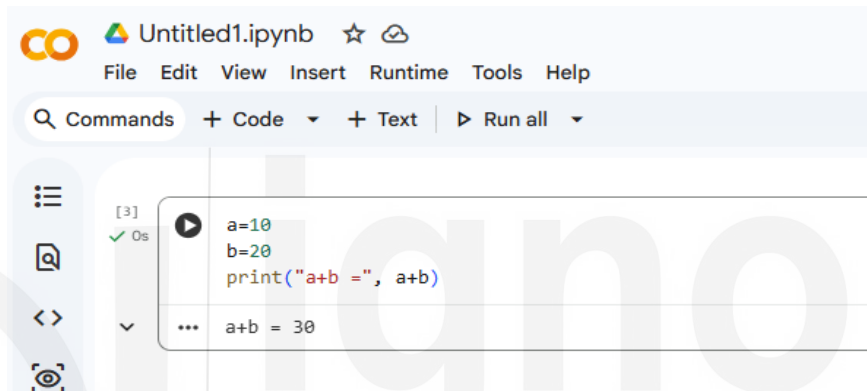
Figure 6: Google Colab

As here, I wrote my first program (`print("Welcome to Python Programming!")`), and executed it by simply pressing the **play button**. You may see, just before the print command I wrote, the output appear beneath the statement `print("Welcome to Python Programming !")`.

First Python program:



Another Python program (to add two numbers):



Check Your Progress -2

1. Compare Framework, Library, Package and Modules in Python.

.....

.....

.....

.....

2. Discuss the utility of the following in Python programming:

- NumPy
- Pandas
- Matplotlib
- Kivy

.....

.....

.....

.....

3. Write a Python program in Google Colab which takes two numbers as input and displays their sum.

.....

.....

.....

.....

1.8 SUMMARY

Guido van Rossum created Python in 1991 to prioritise clean code readability and rapid developer productivity. This unit discussed the history, need for Python and its area of applications. This unit covers not only Python's history, but also its application areas and the tools relevant to exploring them. As described in the unit, Python has an easy-to-learn syntax, dynamic typing, and a vast standard library that powers fields like AI, data science, and web development. For building and distributing cross-platform applications, Python relies on robust packages like Tkinter, PyQt, and Kivy . To start Python programming, you need to install the interpreter from python.org and choose between real-time testing in the interactive shell or writing standalone .py script files. Many platforms/IDEs, like Google Colab and PyCharm, are available for Python programming.

1.9 ANSWER TO CHECK YOUR PROGRESS

Answer to Check Your Progress -1

1.
 - The following are some key comparisons of Python and C:
 - i. Python is a high-level, interpreted language focused on developer productivity and readable syntax, whereas C is a low-level, compiled language built for high execution speed and direct hardware control.
 - ii. While Python features dynamic typing, automatic garbage collection, and a massive standard library, C requires static variable declarations and manual memory allocation
 - iii. Being interpreter-based, debugging in Python occurs line-by-line during runtime, while C compiles the entire source code at once to catch errors before execution.
 - iv. Python is best suited for AI, data science, and rapid prototyping, while C remains the basis for operating systems, embedded hardware, and performance-critical software.

A developer should choose Python when developer speed, ease of readability, rapid prototyping, and vast library support (like AI or web frameworks) are the priority. On the other hand, developers should choose C when execution speed, tight hardware control, low memory overhead, and direct system access (like kernels or firmware) are the key considerations.

2.
 - CPython: The standard, C-based reference implementation that compiles Python code into interpreted bytecode. It is best suited for General application development, data science, ML/AI, and script writing.
 - Jython: Implemented in Java to compile Python code into Java bytecode for seamless execution on the JVM. Suitable for Legacy enterprise Java systems, JVM scripting, and cross-platform Java app integrations.
 - IronPython: Built for Microsoft's .NET Common Language Runtime (CLR), providing native interop with C# and .NET. It can be used for Windows desktop apps, embedded scripting in .NET software, and C# ecosystem tools.

- PyPy: A high-performance variant using a Just-In-Time (JIT) compiler to execute Python significantly faster. It is better in cases of CPU-bound tasks, developing web servers, and algorithm execution.

3.

Some areas of application of Python programming and Python Library/Frameworks used in their development:

- Artificial Intelligence & Machine Learning: Building neural networks, predictive models, and data analytics pipelines (uses Pandas library, and PyTorch and TensorFlow are frameworks).
- Web & API Development: Designing/developing scalable backend applications and web services using frameworks like Django, Flask, and FastAPI.
- Automation & Web Scraping: Automating tedious system tasks, routine file management, and web data extraction (using Selenium, an automation framework/tool suite, and the BeautifulSoup library).
- IoT & Embedded Systems: Scripting for microcontrollers and single-board hardware like Raspberry Pi using MicroPython.

Answer to Check Your Progress -2

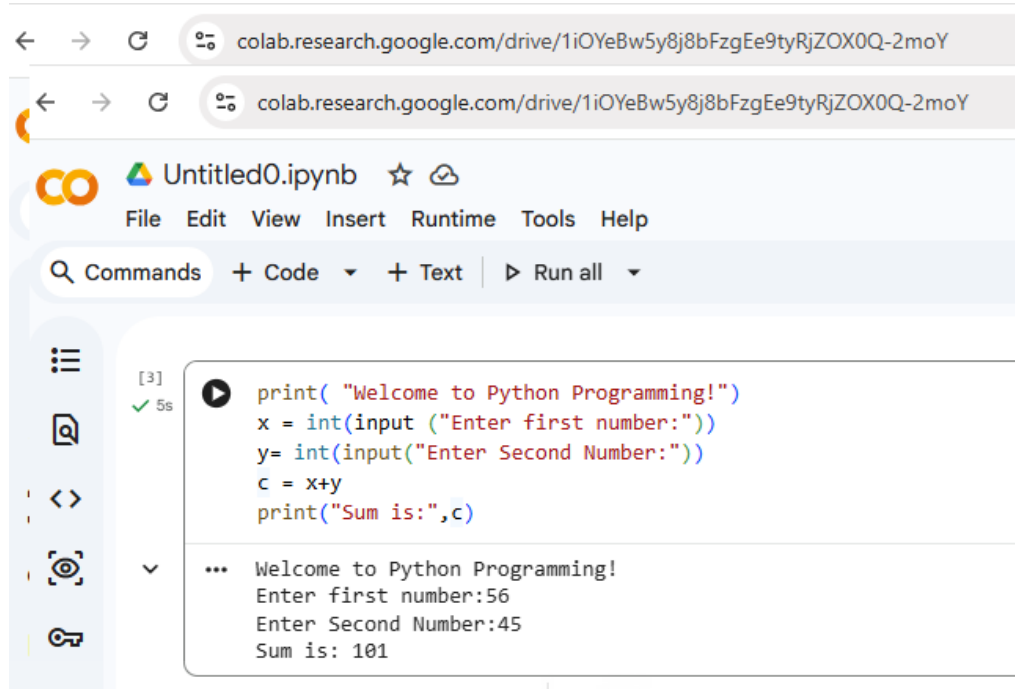
1.

- Module: A single .py file containing reusable code, functions, classes, or variables (math.py, datetime, and os etc.).
- Package: A folder/directory containing multiple related modules along with an __init__.py file (urllib,).
- Library: A broad collection of packages and modules offering specific utilities where your code calls the tools (Pandas, NumPy etc).
- Framework: A complete application skeleton that dictates structure and calls your code via Inversion of Control (Django, Flask etc.).

2.

- **NumPy:** Handles fast math calculations and large grids of numbers. It acts as the main math engine behind almost all Python data science tools.
- **Pandas:** Works with data in tables (just like Excel spreadsheets). It makes it super easy to open, filter, clean, and organise data from files like CSVs
- **Matplotlib:** Turns data into visual charts and graphs (like bar graphs and line plots). It gives you complete control over every detail of how your graph looks.
- **Kivy:** Used to build touch-friendly app screens (GUI) that can run on both computers and mobile phones (Android and iOS).

3.



The screenshot shows a Google Colab notebook interface. The browser address bar displays the URL: `colab.research.google.com/drive/1iOYeBw5y8j8bFzgEe9tyRjZOX0Q-2moY`. The notebook is titled "Untitled0.ipynb". The menu bar includes "File", "Edit", "View", "Insert", "Runtime", "Tools", and "Help". Below the menu bar is a toolbar with "Commands", "+ Code", "+ Text", and "Run all". On the left sidebar, there are icons for file explorer, search, code editor, and output. The main area shows a code cell with the following Python code:

```
[3]
✓ 5s
print( "Welcome to Python Programming!")
x = int(input ("Enter first number:"))
y= int(input("Enter Second Number:"))
c = x+y
print("Sum is:",c)
```

The output of the code cell is displayed below the code:

```
... Welcome to Python Programming!
Enter first number:56
Enter Second Number:45
Sum is: 101
```

1.10 REFERENCES/FURTHER READINGS

1. Yashavant Kanetkar and Aditya Kanetkar, "Let Us Python", 4th Edition, BPB Publications, New Delhi , 2022
2. Paul Deitel and Harvey Deitel , "Python For Programmers" , 1st Edition, Pearson India Education , Chennai, 2023
3. Sheetal Taneja and Naveen Kumar, "Python Programming a Modular Approach" , 1st Edition, Pearson India Education , Chennai , 2022
4. S. Shridhar, J. Indumathi and V.M. Hariharan , "Python Programming" , 1st Edition, Pearson India Education , Chennai, 2024
5. S. Suma Christal Mary , R. Josphine Leela, N. Senthamilarasi and Y. Chrisbin Jeeva , "Problem Solving and Python Programming", 1st Edition, Ane Books , New Delhi, 2018

UNIT 2 DATA STRUCTURES IN PYTHON

Structure

- 2.0 Introduction
- 2.1 Objectives
- 2.2 Identifiers and Keywords
- 2.3 Statements and Expressions
- 2.4 Variables
- 2.5 Operators
- 2.6 Data Types
- 2.7 Data Structures
 - 2.7.1 List
 - 2.7.2 Tuple
 - 2.7.3 Dictionaries
 - 2.7.4 Set
- 2.8 Summary
- 2.9 Answer to Check your Progress
- 2.10 References/Further Readings

2.0 INTRODUCTION

Python is a general-purpose, high level programming language which very popular and being widely used by programmers. It was created by Dutch programmer Guido Van Rossum in 1991 and further developed by the Python Software Foundation. It was designed with an emphasis on code readability, and its syntax allows programmers to express their concepts in fewer lines of code. To learn to code in any language, it helps to understand its basic components. In this unit, we will learn the basic constituents of Python programming, starting with identifiers, variables, and operators. We will see how to combine them to form expressions and statements. In this unit, you will also study the data types and data structures in Python.

2.1 OBJECTIVES

After going through this unit, you will be able to:

- Use identifiers and keywords,
- Use statements and expressions,
- Explain the basic building blocks of the Python programming language,
- Use operators in Python, and
- Use various data structures available in Python.

2.2 IDENTIFIERS AND KEYWORDS

An **identifier** is a name given to a variable, function, class or module. Identifiers may be of one or more characters in the following format:

- Identifiers can be a combination of letters in lowercase (a to z) or uppercase (A to Z) or digits (0 to 9) or an underscore (_). Names like myIndia, Marks_1 and mca_course are all valid examples. A Python

identifier may begin with an uppercase or a lowercase letter or an underscore (_).

- An identifier cannot start with a digit but is allowed everywhere else. For example, **1plus** is invalid, but **plus1** is perfectly fine.
- Keywords (listed in Table-1) cannot be used as identifiers.
- One cannot use spaces and special symbols like !, @, #, \$, % etc. as identifiers.
- An identifier can be of any length. (However, it is preferred to keep it short and meaningful).

Some examples of valid identifiers are: Subject1_Marks, Course, Age, Sex, etc.

Keywords of Python are a list of reserved words that have predefined meaning to the Python interpreter. These are special words and cannot be used by programmers as identifiers (variables), functions, constants, or any identifier name. Using a keyword as an identifier name causes an error. Because Python is case-sensitive, keywords must be written exactly as shown in Table 1.

Table -1: List of Keywords in Python Programming Language

False	class	finally	is	return
None	continue	for	lambda	try
True	def	from	nonlocal	while
and	del	global	not	with
as	elif	if	or	yield
assert	else	import	pass	
break	except	in	raise	

Literals are the raw data values or fixed data that are used to define constants. In other words, literals represent fixed values directly in your code without requiring computation. Literals represent a specific value in a program.

Python supports several types of literals:

1. String Literals: A string literal is a sequence of characters enclosed in quotation marks. Python allows three types of quotation marks for strings: single quotes, double quotes, and triple quotes.

- **Single Quotes:** 'Hello'
- **Double Quotes:** "Python"
- **Triple Quotes (for multi-line strings or docstrings):** In Python, triple quotes (""" or ''') are a unique syntax used to define both multi-line strings and docstrings (documentation strings). It preserves

formatting, indentation, and newlines exactly as typed without requiring explicit escape characters like `\n`.

Docstrings: A docstring is a specific type of string literal used to document your code. To trigger docstring behaviour, the triple-quoted string must be the very first statement inside a function, class, or module. Python automatically binds this text to the object's `__doc__` attribute.

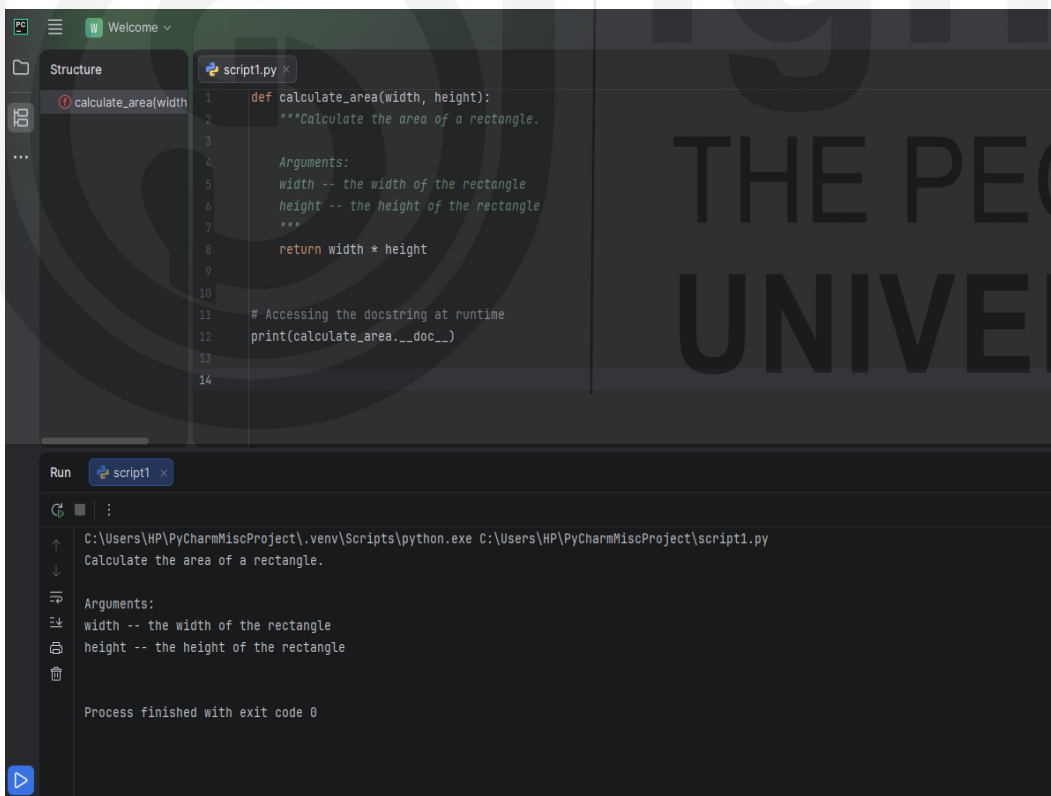
For example, if the following Python code:

```
def calculate_area(width, height):
    """ Calculate the area of a rectangle.
    Arguments:
    width -- the width of the rectangle
    height -- the height of the rectangle
    """

    return width * height
# Accessing the docstring at runtime
print(calculate_area.__doc__)
```

is executed, the output will be:

```
Arguments:
width -- the width of the rectangle
height -- the height of the rectangle
```



The screenshot shows a Python IDE with a dark theme. The editor window displays the following code:

```
1 def calculate_area(width, height):
2     """ Calculate the area of a rectangle.
3
4     Arguments:
5     width -- the width of the rectangle
6     height -- the height of the rectangle
7     """
8     return width * height
9
10
11 # Accessing the docstring at runtime
12 print(calculate_area.__doc__)
13
14
```

The Run console at the bottom shows the output of the script:

```
C:\Users\HP\PyCharmMiscProject\.venv\Scripts\python.exe C:\Users\HP\PyCharmMiscProject\script1.py
Calculate the area of a rectangle.

Arguments:
width -- the width of the rectangle
height -- the height of the rectangle

Process finished with exit code 0
```

2. Numeric Literals: In Python, numeric literals are immutable and are used to represent numbers. Python supports three distinct numeric types:

Table - 2: Numeric Literals in Python

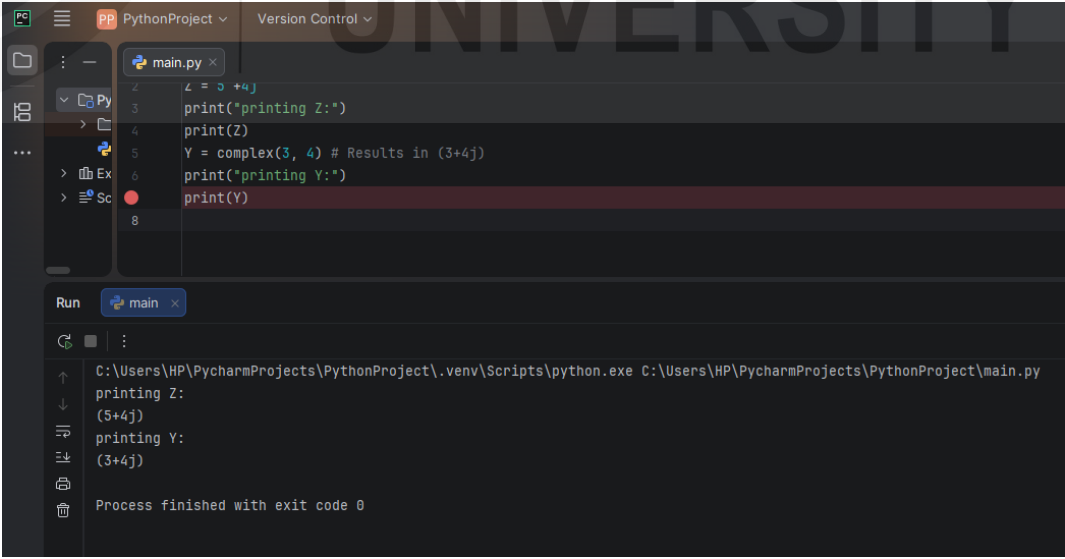
Type	Description	Examples
Integer (int)	Whole numbers, positive or negative, without a decimal point. Includes decimal, octal, hexadecimal, and binary forms.	10, -5, 0xAF (hex), 0b10101 (binary)
Float (float)	Real numbers, positive or negative, containing a decimal point or expressed in scientific notation.	10.5, -3.14, 1.2e5 (1.2 times 10 to the power of 5)
Complex (complex)	Numbers in the form a + bj, where a is the real part and b is the imaginary part.	3 + 4j

Example: Printing complex numbers

If you run the following code:

```
Z = 5 + 4j
print("printing Z:")
print(Z)
Y = complex(3, 4) # Results in (3+4j)
print("printing Y:")
print(Y)
```

As shown in the following screenshot, the complex() function also returns a complex number.

A screenshot of a Python IDE (PyCharm) showing a code editor and a run console. The code editor displays the following Python code:

```
1 Z = 5 + 4j
2 print("printing Z:")
3 print(Z)
4
5 Y = complex(3, 4) # Results in (3+4j)
6 print("printing Y:")
7 print(Y)
8
```

The run console at the bottom shows the output of the code:

```
C:\Users\HP\PycharmProjects\PythonProject\.venv\Scripts\python.exe C:\Users\HP\PycharmProjects\PythonProject\main.py
printing Z:
(5+4j)
printing Y:
(3+4j)
Process finished with exit code 0
```

3. Boolean Literals: A Boolean literal can have one of two values:

- True: Represents the logical truth.
- False: Represents the logical falsity.

Python has one special literal: **None**. It indicates that a variable has no value. Notably, it is not the same as 0 or an empty string. It is often used as a placeholder for a value that is yet to be provided. The following points should be noted about None:

- **It is a Singleton:** Python creates exactly one **None object** in memory during execution. All variables assigned **None point** to this identical memory address.
- **Falsy but Distinct:** In conditional statements, **None** evaluates to False. However, it does not equal 0, False, or an empty string "".
- **Implicit Return Value:** If a function does not contain an explicit return statement, Python automatically returns None.

2.3 STATEMENTS AND EXPRESSIONS

A statement is a unit of code that the Python interpreter can execute. Generally, statements fall into two kinds: expression statements and assignment statements. Note that the assignment statement produces no output.

When you type a statement in interactive mode, the interpreter executes it and displays the result, if there is one.

A script usually contains a sequence of statements. If there is more than one statement, the results appear one at a time as the statements execute.

For example, the script:

```
print(1)
x = 2
print(x)
```

produces the output:

```
1
2
```

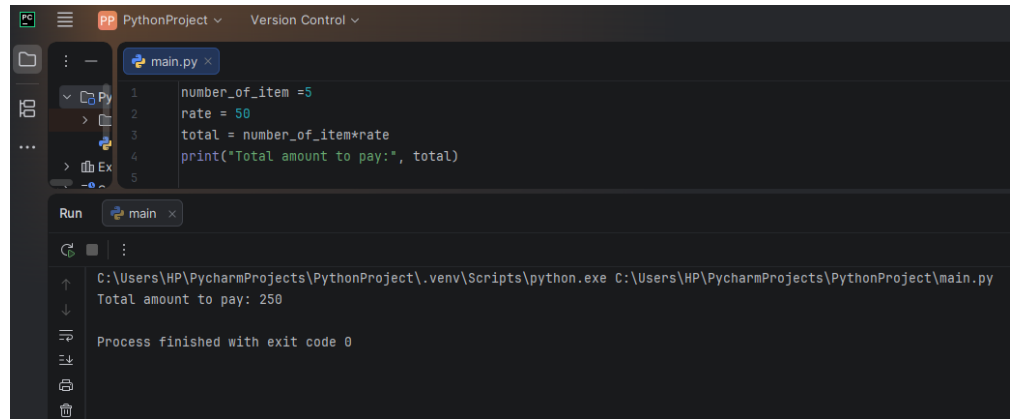
An **expression** is an arrangement of values and operators that evaluates to a new value. Expressions are statements as well. A value is the representation of some entity like a letter or a number that a program can manipulate. A single value **25** or a single variable **x** or a combination of a variable, operator and value **x + 25** are examples of expressions. In interactive mode, the interpreter evaluates an expression and displays the result instantly. For example,

```
X = 7 + 3
```

Output: 10

Another example of preparing an expression is calculating the total amount you need to pay for items you buy at a shop.

```
total = number_of_item*rate
```

A screenshot of a Python IDE interface. The top pane shows a file named 'main.py' with the following code:

```
1 number_of_item = 5
2 rate = 50
3 total = number_of_item * rate
4 print("Total amount to pay:", total)
5
```

The bottom pane shows the execution output:

```
C:\Users\HP\PycharmProjects\PythonProject\.venv\Scripts\python.exe C:\Users\HP\PycharmProjects\PythonProject\main.py
Total amount to pay: 250
Process finished with exit code 0
```

Note that in a script, an expression by itself does not show any output until you explicitly print the result.

2.4 VARIABLES

A variable is a named placeholder that holds data the program can use, assign, and modify during execution. As Python is a Dynamically Typed Language, there is no need to declare a variable explicitly by specifying whether the variable is an integer or a float or any other type. To define a new variable in Python, we simply assign a value to a name.

Variable names can be arbitrarily long. They can contain both letters and numbers, but you should follow the normal naming convention for variables. It is legal to use uppercase letters, but it may be a good idea to begin variable names with a lowercase letter. Note: variable names are case-sensitive. e.g., IGNOU and Ignou are two different variables.

The underscore character can appear in a name. It is often used in multi-word names, such as `my_name` or `marks_in_maths`. Variable names can start with an underscore character, but we generally avoid doing this unless we are writing library code for others to use.

The general format for assigning values to variables is as follows:

variable_name = expression

The equal sign (`=`), also known as the simple assignment operator, assigns values to variables. In the general format, the operand to the left of the `=` operator is the name of the variable, and the operand to the right of the `=` operator is the expression, which can be a value or any code snippet that results in a value. That value is stored in the variable on the execution of the assignment statement. The assignment operator should not be confused with the `=` used in algebra to denote equality. e.g.,

number = 100 *#integer type value is assigned to a variable number*
name = "Python" *#string type value is assigned to variable name*

In the above two statements

#integer type value is assigned to a variable number

and

#string type value is assigned to variable name

are comments. Note that comments in Python are created using the **hash character (#)**, which tells the interpreter to ignore all text following it on that line.

Python does not have multiline comments, but you can place a # at the beginning of every individual line to create multiline comments. This is a PEP 8-compliant approach to multiline comments in Python. Another way of multiline comment is to use unassigned string literals; in this approach, **you need to enclose** your comment block within triple quotes (""" or "). If a string literal is not assigned to a variable, Python allocates memory but ignores the text at runtime (effectively treating it as a comment block).

Example of multiline comment:

```
""" Try this, it is a multiline comment. It works fine as Python ignores this
block during execution. """
```

Python is dynamically typed. In Python, variables are not fixed "boxes" locked into a specific data type. Instead, variables are labels or references that point to objects in memory.

It means when we create a variable in Python:

1. It creates an object in memory that contains both the value and its data type.
2. The variable name acts as a tag attached to that object.

Here, the data type belongs to the object in memory, not the variable name. Hence, you can freely point the same variable name to an entirely different type of object whenever you like.

For example, you can assign an integer value to a variable, use it as an integer for a while and then assign a string to the variable. A new assignment overrides any previous assignments.

```
year = 2026          #an integer value is assigned to year variable
year = "twenty twenty six" # a string value is assigned to year variable
```

Python allows you to assign a single value to several variables simultaneously.

For example:

```
x = y = z = 5 #an integer value is assigned to variables x, y, z
simultaneously.
```

2.5 OPERATORS

Operators are special symbols that represent computations like addition and multiplication. The values the operator is applied to are called operands.

Python language supports a wide range of operators:

1. Arithmetic Operators
2. Assignment Operators
3. Relational Operators
4. Logical Operators
5. Bitwise Operators

Table - 3: List of Arithmetic Operators

Operator	Operator Name	Description	Example (try in lab) n1 = 5, n2 = 6
+	Addition operator	Adds two operands, producing their sum	R = n1 + n2 Value of R is 11
-	Subtraction operator	Subtracts the two operands, producing their difference	R = n1 - n2 Value of R is -1
*	Multiplication operator	Produces the product of the operands	R = n1 * n2 Value of R is 30
/	Division operator	Produces the quotient of its operands where the left operand is the dividend and the right operand is the divisor	R = n1 / n2 Value of R is 0.833333
%	Modulus operator	Divides the left-hand operand by the right-hand operand and returns the remainder	R = n1 % n2 Value of R is 5
**	Exponent operator	Performs exponential (power) calculation on operators	R = n1 ** n2 Value of R is 15625
//	Floor division operator	Returns the integral part of the quotient	R = n1 // n2 Value of R is 0

Table - 4: List of Assignment Operators

Operator	Operator Name	Description	Example (try in lab) (‘~’ stands for is equivalent to) n1 = 5, n2 = 6
=	Assignment	Assigns values from right-side operands to the left-side operand	m = n1 + n2 Value of m is 11
+=	Addition Assignment	Adds the value of right operand to the left operand and assigns the result to left operand	m += n1 ~ m = m + n1 when m += n1 Value of m is 16
-=	Subtraction Assignment	Subtracts the value of right operand from the left operand and assigns the result to left operand	m -= n1 ~ m = m - n1 when m -= n1 Value of m is 11
*=	Multiplication Assignment	Multiplies the value of right operand with the left operand and assigns the result to left operand	m *= n1 ~ m = m * n1 when m *= n1 Value of m is 55
/=	Division Assignment	Divides the value of right operand with the left operand and assigns the result to left operand	m /= n1 ~ m = m / n1 when m /= n1 Value of m is 11.0
**=	Exponentiation Assignment	Evaluates to the result of	m ** n1 ~ m = m **

	ion Assignment	raising the first operand to the power of the second operand	n1 when m **= n1 Value of m is 161051.0
//=	Floor Division Assignment	Produces the integral part of the quotient of its operands where the left operand is the dividend and the right operand is the divisor	m //= n1 ~ m = m // n1 when m //= n1 Value of m is 32210.0
%=	Remainder Assignment	Computes the remainder after division and assigns the value to the left operand.	m %= n1 ~ m = m % n1 when m %= n1 Value of m is 0.0

Table - 5: List of Relational Operators

Operator	Operation	Description	Example(Try in Lab) n1 = 10, n2 = 0, s1 = "Hello", s2="World"
==	Equals to	If the values of two operands are equal, then the condition is True; otherwise, it is False.	n1 == n2 results in: False s1 == s2 results to: False
!=	Not equal to	If the values of two operands are not equal, then the condition is True; otherwise, it is False.	n1 != n2 results in: True s1 != s2 results in: False
>	Greater than	If the value of the left operand is greater than the value of the right operand, then the condition is True; otherwise, it is False.	n1 > n2 results in: True s1 > s2 results in: True
<	Less than	If the value of the left operand is less than the value of the right operand, the condition is True; otherwise, it is False.	n1 < n2 results in: False
>=	Greater than or equal to	If the value of the left operand is greater than or equal to the right operand, the condition is True; otherwise, it is False.	n1 >= n2 results in: True
<=	Less than or equal to	If the value of the left operand is less than or equal to the right operand, the condition is True; otherwise, it is False.	n1 <= n2 results in: False

Note: For strings, the characters in both strings are compared one by one based on their Unicode value. The character with the lower Unicode value is considered to be smaller.

Table - 6: List of Logical Operators

Operator	Operation	Description	Example (Try in Lab) n1 = 10, n2 = -20
And	Logical AND	If both operands are True, then the condition becomes True.	n1 == 10 and n2 == -20 results in: True
Or	Logical OR	If any of the two operands is True, then the condition becomes True.	n1 >= 10 or n2 < -20 results in: True
Not	Logical NOT	Used to reverse the logical state of its operand.	not(n1 == 20) results in: True

Table -7: Boolean Logic Truth Table

n1	n2	n1 and n2	n1 or n2	not n1
True	True	True	True	False
True	False	False	True	
False	True	False	True	True
False	False	False	False	

Table -8: List of Bitwise Operators

Operator	Operator Name	Description	Example (try in Lab) n1 = 60, n2 = 13
&	Binary AND	Result is one in each bit position for which the corresponding bits of both operands are 1s.	n1 & n2 results in: 12 (means 0000 1100)
	Binary OR	Result is one in each bit position for which the corresponding bits of either or both operands are 1s.	n1 n2 results in: 61 (means 0011 1101)
^	Binary XOR	Result is one in each bit position for which the corresponding bits of either but not both operands are 1s.	n1 ^ n2 results in: 49 (means 0011 0001)
~	Binary Ones Complement	Inverts the bits of its operand.	~n1 results in: -61 (means 1100 0011 in 2's complement form due to a signed binary number)
<<	Binary Left Shift	The left operand's value is moved left by the number of bits specified by the right operand.	n1 << 2 results in: 240 (means 1111 0000)
>>	Binary Right Shift	The left operand value is moved right by the number of bits specified by the right operand.	n1 >> 2 results in: 15 (means 0000 1111)

Table -9: Bitwise Truth Table

n1	n2	n1 & n2	n1 n2	n1 ^ n2	~n1
0	0	0	0	0	1
0	1	0	1	1	
1	0	0	1	1	0
1	1	1	1	0	

Table -10: Operator Precedence in Python

Operators	Meaning
()	Parentheses
**	Exponent
+x, -x, ~x	Unary plus, Unary minus, Bitwise NOT
*, /, //, %	Multiplication, Division, Floor Division, Modulus
+, -	Addition, Subtraction
<<, >>	Bitwise shift operators
&	Bitwise AND
^	Bitwise XOR
	Bitwise OR
==, !=, >, >=, <, <=	Comparisons
Not	Logical NOT
And	Logical AND
Or	Logical OR

2.6 DATA TYPES

Data types specify the type of data, such as numbers and characters, that a program stores and manipulates. An important point to note is that in Python, every data type is an object, and there are no "primitive" data types as you find in languages such as C, C++ or Java. Every time we create a variable or a value, Python wraps it inside a robust container called an object. Each object in Python always contains three things:

1. **Identity:** It is a unique identifier (essentially it is a memory address, which you can check using the function `id()`).
2. **Type:** The class that defines what the object can do (which you can check using the function `type()`).
3. **Value:** The actual data stored inside the object.

Python supports the following five main built-in object types:

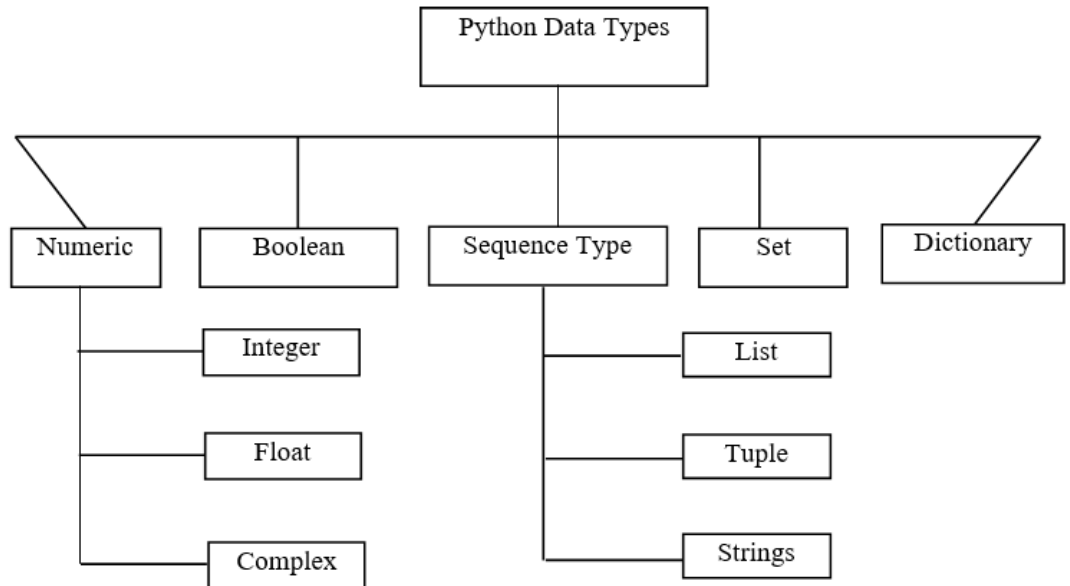


Figure 1: Python Built in Object Types

2.6.1 Numeric

Numeric data types mainly include numbers. These numbers also come in four types: Integer, Float, and Complex.

Table -11: Data Types in Python

Type	Built-in Class	Description	Examples
Integer	int	Integer numbers	-12, -3, 0, 123, 2
Float	float	Floating point numbers	-2.04, 4.0, 14.23
Complex	complex	Complex numbers	3+4j, 2-2j

2.6.2 Boolean

Boolean: In Python, Boolean contains only two values: True and False. The bool class is a direct subclass of the **int** (integer) class, and there are only two instances of this class: True (which evaluates numerically to 1) and False (which evaluates to 0).

2.6.3 String

A string is an ordered sequence of characters. It belongs to the built-in **str** class. These characters may be alphabets, digits or special characters including spaces. String values are enclosed in single quotation marks (e.g. 'hello') or in double quotation marks (e.g., "python course"). The quotes are not part of the string; they mark the string's beginning and end for the interpreter. In Python, there is no character data type; a character is a string of length one. Some of the characteristics of strings are:

- Numerical operations can not be performed on strings, even when the string contains a numeric value like str2 defined below.
- A string has a length. Use the len() built-in function to get the length.

- A string is indexable. Get a single character at a position in a string with the square bracket operator, for example `str1[4]`. Indexing always starts with 0.
- One can retrieve a slice (sub-string) of a string with a slice operation, for example `str1[4:8]`.
- Strings are immutable, which means you can not change an existing string.

Examples:

`str1 = 'Hello Friend'` #variable str1 of string type is declared

`str2 = "452"` #variable str2 of string type is declared

`len(str1)` #length of string str1

will produce: 12

`str1[4]` #to access 5th character in the string

will produce: 'o'

`str1[-1]` #negative indices can be used, which count backwards from the end of the string. [-1] yields the last letter.

will produce: 'd'

`str1[4:8]` # Operator returns the part of the string from the first index to the second index, including the first but excluding the last. So, the substring starting from the 5th character to the 8th character is extracted.

will produce: 'o Fr'

If you omit the first index (before the colon), the slice starts at the beginning of the string; if you omit the second index, the slice goes to the end of the string.

`str1[:3]`

will produce: 'hel'

`str1[3:]`

will produce: 'lo Friend'

`str1[2] = 'a'`

TypeError: 'str' object does not support item assignment.

`str2 = 'Narender'`

`str1 + str2` #for concatenating two strings, "+" operator is used

will produce: 'hello Friend Narender'

```
str1 * 3                                #for creating multiple concatenated copies of a  
                                         string, “*” operator is used
```

will produce: ‘hello FriendhelloFriendhello Friend’

2.6.4 List

A list is a sequence of items separated by commas and enclosed in square brackets []. These are similar to arrays in other programming languages, but unlike arrays, list items can be of different data types. Lists are mutable, and hence, they can be altered even after their creation. Python lists are ordered and have a definite sequence, and indexing begins at 0 (0 being the first index). Each element has a definite place in the list, which allows duplicates, with each element having its own distinct place and credibility. When describing list elements as having “credibility”, it means to say that predictability and reliability: as every element’s place is guaranteed and distinct, hence, you can trust its location and identity completely. It means a list in Python is mutable, and you can change, add, or remove its elements after it has been created without altering its unique memory identity (**id()**). It belongs to the built-in class list.

```
list1 = [5, 3.4, 'IGNOU', 'Part 3', 45]    #to create a list  
list1                                       #to print the elements of the list list1  
will produce: [5, 3.4, 'IGNOU', 'Part 3', 45]  
  
list1[0]                                   #accessing first element of the list  
will produce: 5  
  
list1[3]                                   #accessing 4th element of the list  
will produce: Part 3  
  
list1[-1]                                  #accessing last element of the list  
will produce: 45  
  
list1[-3]                                  #accessing last 3rd element of the list  
will produce: 'IGNOU'
```

2.6.5 Tuple

A tuple is similar to the list in many ways. Like lists, tuples also contain a collection of items of different data types. It belongs to the built-in **class tuple**. The items of the tuple are separated with a comma(,) and enclosed in parentheses(()). A tuple is a **read-only data structure**, as we can not modify the size and value of the items of a tuple, and it is immutable.

```
t = ('hello','world',2)                   #creating tuple t  
t                                           # printing tuple t  
will produce: ('hello', 'world', 2)  
  
t[1]                                       #accessing specific element of the tuple say second element  
will produce: 'world'
```

2.6.6 Dictionary

A dictionary in Python holds data items in **key-value pairs of items**. It belongs to the built-in **class dict**. Items in a dictionary are separated by commas and enclosed in curly brackets {}. Dictionaries permit faster access to data. Every key is separated from its value using a colon(:) sign. Keys in a dictionary are case-sensitive. Keys can not be repeated and are immutable. You can access a dictionary's key-value pairs using the key. Keys are usually strings, and values can be any data type and can be duplicated. To access a value in a dictionary, we must specify its key in square brackets[].

#Creating dictionary dict1

```
dict1 = {'Fruit':'Apple','Climate':'Cold','Price(Kg)':120}
```

```
dict1          #printing the contents of dictionary dict1
```

will produce: {'Fruit': 'Apple', 'Climate': 'Cold', 'Price(Kg)': 120}

```
dict1['Climate'] #Getting value by specifying the key
```

will produce: Cold

```
dict1.keys()    #printing all the Keys in the dictionary
```

will produce: dict_keys(['Fruit', 'Climate', 'Price(Kg)'])

```
dict1.values()      #printing all the values in the dictionary
```

will produce: dict_values(['Apple', 'Cold', 120])

2.6.7 Set

In Python, a **set** is an unordered collection of data types that is iterable, mutable and has no duplicate elements. It belongs to the built-in set class. The order of elements in a set is undefined, though it may consist of various elements. The major advantage of using a set, as opposed to a list, is its highly optimised method for checking whether a specific element is in the set.

You can create sets using the built-in set() function with an iterable object or by placing a sequence in curly braces, separated by commas. A set contains only unique elements, but at the time of set creation, multiple duplicate values can also be passed. The order of elements in a set is undefined and is unchangeable. Elements in a set need not be the same type; you can also pass mixed data types to a set.

```
set1 = set("IGNOU MCA BATCH") #creating set set1
```

```
set1    #displaying contents of set1
```

will produce: {' ', 'A', 'B', 'C', 'G', 'H', 'I', 'M', 'N', 'O', 'T', 'U'}

```
set2 = set([2, 3, 2, 'MCA', 'IGNOU', 1, 'IGNOU']) #set with the use of mixed values
```

```
set2
```

will produce: {1, 2, 3, 'IGNOU', 'MCA'}

2.6.8 None

The **None** is another special data type in Python. A **None** is frequently used to represent the absence of a value. **None** is the "null" of Python, and **NoneType** is its type. In other words, **NoneType** is the type of the **None** object (**None** belongs to the class **NoneType**).

```
money = None           #None value is assigned to variable money
```

Interestingly, some Python literature treats List, Tuple, Set, and Dictionary as additional data types, while others treat them as built-in data structures. We will put them in the category of data structures and discuss them there, but considering them as data types is also not wrong.

Check your progress - 1

Q1. What is the maximum possible length of an identifier in Python?

- a) 32 characters
- b) 64 characters
- c) 79 characters
- d) An identifier can be of any length

Q2. Which of the following can be part of an expression in Python?

- a) Values
- b) Variables
- c) Operators
- d) All of the above

Q3. What is the value of variable **a** after executing the following code?

```
a = 25  
b = 35  
a = b
```

- a) 25
- b) 35
- c) None
- d) It will raise an error

Q4. What error occurs when you execute the following Python code snippet (assuming **mango** has not been defined)?

```
apple = mango
```

- a) `SyntaxError`
- b) `NameError`
- c) `ValueError`
- d) `TypeError`

5. What is the output of `print(2 ** 3 ** 2)`?

- a) 64
- b) 128
- c) 256
- d) 512

6. What is the output of the following code?

```
a = 100  
b = 200  
print (a and b)
```

- a) 100
- b) 200
- c) True

d) False

7. 4 is 100 in binary and 11 is 1011. What is the output of the following bitwise operators?

```
a = 4
```

```
b = 11
```

```
print (a | b)
```

```
print (a >> 2)
```

a) 15, 1

b) 14, 1

c) 15, 2

d) 14, 2

8. If we convert one data type to another, then it is called

a) Type Conversion

b) Type Casting

c) Both of the above

d) None of the above

9) State the rules for naming identifiers in Python.

10) Can an expression exist inside a statement? Explain with the help of a simple program.

11) Do we need to declare variables before using them in Python?

12) What is the difference between “/ and //” operators? Explain with an example.

2.7 DATA STRUCTURES

Data structures organise, store, and manage data for efficient access and modification. Some literature on Python categorises List, Tuple, Dictionary, and Set as data types, while others categorise them as primitive data structures. In Python, List, Tuple, Dictionary, and Set are officially built-in compound data types, but structurally, they are non-primitive data structures. As you know, lists, tuples, sets, and dictionaries have internal rules governing how data is ordered, indexed, and stored; that is why they are categorised as built-in data structures.

We have already seen these structures in the previous section of this unit. Now let us discuss them in detail here:

2.7.1 List

A **list** is a container data type that acts as a dynamic array. A list is a sequence you can index, and that can grow and shrink. A few characteristics of lists are:

- A list has a (current) length – Get the length of a list with the len() function.
- A list has an order – the items in a list are ordered, with order going from left to right.
- A list is heterogeneous –different types of objects can be inserted into the same list.
- Lists are mutable, and one can extend, delete, insert or modify items in the list.

The syntax for creating a list is:

```
list_name = [item_1, item_2, item_3, ....., item_n]
```

You can store any item in a list, such as a string, number, object, another variable, or even another list. A list can contain a mix of different item types, and these types need not be homogeneous. For example, a list can mix numbers, strings, and another list. The contents of the list variable are displayed by executing the list variable name.

Examples :

```
list_name = [ ]           #empty list without any it
```

```
#when each item in the list is a string
```

```
stringlist = ["mahesh", "ramesh", "suresh", "vishnu", "ganesh"]
```

```
#when each item in the list is a number
```

```
numlist = [4, 6, 7, 10, 15, 13]
```

```
#when list contains mixed items
```

```
mixed_list = ['cat', 87.23, 65, [9, 8, 1]]
```

2.7.1.1 Indexing and Slicing in Lists

As an ordered sequence of elements, each item in a list can be accessed individually through indexing. The expression inside the bracket is called the index. Lists use square brackets [] to access individual items, with the first item at index 0, the second item at index 1 and so on. The index provided within the square brackets indicates the value being accessed.

The syntax for accessing an item in a list is:

list_name [index]

where index should always be an integer value and indicates the item to be selected.

if

```
stringlist = ["mahesh", "ramesh", "suresh", "vishnu", "ganesh"]
```

then

```
stringlist[0]          #to access the first item in stringlist
```

will produce: 'mahesh'

and

in

```
numlist = [4, 6, 7, 10, 15, 13]
```

if you access

```
numlist[4]             #to access fifth item in the numlist
```

it will produce: 15

and

```
numlist[6]             #if index value is more than the number of items in the list, it  
will raise an error
```

If you try to access numlist[6] output would be :

Traceback (most recent call last):

File "<stdin>", line 1, in <module>

```
numlist[6]
```

IndexError : List index out of range

Here, in numlist, index numbers range from 0 to 5 as it contains 6 items.

In addition to positive index numbers, you can also access items in a list with a negative index number by counting backwards from the end of the list, starting at -1. It is useful if the list is long and we want to locate an item towards the end of the list.

For example, if you access

```
stringlist[-2]         #to access the second list item from the list
```

it will produce: 'vishnu'

The slice operator also works on lists:

```
numlist[1:3]
```

will produce: [6, 7]

```
mixed_list[:3]
```

will produce: ['cat', 87.23, 65]

mixed_list[2:]

will produce: [65, [9, 8, 1]]

numlist[:]

will produce: [4, 6, 7, 10, 15, 13]

If you omit the first index, the slice starts at the beginning; if you omit the second index, the slice goes to the end. So, if both indices are omitted, the slice copies the whole list.

2.7.1.2 List Operations

The + operator concatenates lists:

```
a = [1, 2, 3]
```

```
b = [4, 5, 6]
```

```
then a + b
```

```
will be : [1, 2, 3, 4, 5, 6]
```

Similarly, the * operator repeats a list given a number of times:

```
c = ['pass']
```

```
c * 3
```

```
will be : ['pass', 'pass', 'pass']
```

== operator is used to compare the two lists.

```
a == b
```

```
will be: False
```

in and not in membership operators are used to check for the presence of an item in the list:

2 in a will give True and 4 not in [a] will also give True

2.7.1.3 The list() function

The built-in list () function creates a list. The syntax for list() function is,

```
list([sequence])
```

where the sequence can be a string, tuple or list itself. If you don't specify the optional sequence, it creates an empty list.

```
greet = "How are you doing?"           #string declaration
```

```
greet
```

will produce: 'How are you doing?'

```
str_to_list = list(greet)              #string converted to list using list()
```

```
str_to_list
```

will produce: ['H','o','w',' ','a','r','e',' ','y','o','u',' ','d','o','i','n','g','?']

```
friend = "nidhi"                #string declaration
```

```
str_to_list + friend            #list concatenated with string
```

TypeError: can only concatenate list (not "str") to list

```
str_to_list + list(friend)      #list concatenated with list
```

will produce : ['H','o','w',' ','a','r','e',' ','y','o','u',' ','d','o','i','n','g','?','n','i','d','h','i']

2.7.1.4 Modifying items in Lists

Lists are mutable because you can modify list items after a list is created. You can also modify a list by replacing an older item with a newer one, without assigning the list to a completely new variable.

```
stringlist[1] = "umesh"
```

and

```
stringlist
```

will produce: ['mahesh', 'umesh', 'suresh', 'vishnu', 'ganesh']

When an existing list variable is assigned to a new variable, an assignment (=) on lists does not make a new copy. Instead, assignment makes both variable names point to the same list in memory. That is why any change in one list will also reflect in the other.

```
listofstrings = stringlist
```

```
listofstrings[1] = "ramesh"
```

```
listofstrings
```

will produce: ['mahesh', 'ramesh', 'suresh', 'vishnu', 'ganesh']

and

```
stringlist
```

will produce: ['mahesh', 'ramesh', 'suresh', 'vishnu', 'ganesh']

2.7.1.5 Traversing a list

The most common way to traverse the elements of a list is with a for loop.

for names in stringlist :

```
print(names)
```

will produce:

```
mahesh
```

```
ramesh
```

suresh

vishnu

ganesh

This works well if one needs to read the elements of the list. But if you want to write or update the elements, you need the indices. For this, one has to combine the range () function and len() function:

```
for i in range(len(numlist)) :
```

```
    numlist[i] = numlist[i] * 2
```

This loop traverses the list and updates each element. The len () command returns the number of elements in the list. The range () command returns a list of indices from 0 to n-1, where n is the list length. In each loop iteration, the variable i gets the index of the next element. The assignment statement in the body uses i to read the old value of the element and to assign the new value.

```
numlist
```

will produce: [8, 12, 14, 20, 30, 26]

2.7.1.6 List Methods

Python provides methods that operate on lists.

For example,

- **append** adds a new element to the end of a list:

```
numlist.append(50)
```

now,

```
numlist
```

will produce: [8, 12, 14, 20, 30, 26, 50]

- **extend** takes a list as an argument and appends all of the elements:

```
nlist = [25, 35, 45]
```

when we do the following:

```
numlist.extend(nlist)
```

```
numlist
```

will produce: [8, 12, 14, 20, 30, 26, 50, 25, 35, 45]

- **sort** arranges the elements of the list from low to high:

```
numlist.sort()
```

now,

```
numlist
```

will produce: [8, 12, 14, 20, 25, 26, 30, 35, 45, 50]

- **pop** is used to delete elements from a list if the index of the element is known:

```
numlist.pop(5)
```

will produce: 26

and now,

```
numlist
```

will produce: [8, 12, 14, 20, 25, 30, 35, 45, 50]

pop modifies the list and returns the removed element.

- If the removed element is not required, then one can use the **del** operator:

```
del numlist[0]
```

then,

```
numlist
```

will produce: [12, 14, 20, 25, 30, 35, 45, 50]

- To remove more than one element, **del** can be used with a slice index

```
del numlist[0:2]
```

now,

```
numlist
```

will produce: [20, 25, 30, 35, 45, 50]

- If an element from the list is known which needs to be removed and not the index, **remove** can be used:

```
numlist.remove(30)
```

now,

```
numlist
```

will produce: [20, 25, 35, 45, 50]

The return value for remove is None.

To get a list of all methods associated with the list, pass the list to **dir()**.

dir(list)

Table-12: Various List Methods

List Methods	Syntax	Description
append()	list.append(item)	The append() method adds a single item to the end of the list. This method does not return a new list, and it just modifies the original.
count()	list.count(item)	The count() method counts the number of times the item has occurred in the list and returns it.
insert()	list.insert(index, item)	The insert() method inserts the item at the given index, shifting items to the right.
extend()	list.extend(list2)	The extend() method adds the items in list2 to the end of the list.
index()	list.index(item)	The index() method searches for the given item from the start of the list and returns its index. If the value appears more than once, it returns the index of the first occurrence. If the item is not present in the list, then ValueError is thrown by this method.
remove()	list.remove(item)	The remove() method searches for the first instance of the given item in the list and removes it. If the item is not present in the list, this method throws a ValueError.
sort()	list.sort()	The sort() method sorts the items in the list in place. This method modifies the original list, and it does not return a new list.
reverse()	list.reverse()	The reverse() method reverses the items in place in the list. This method modifies the original list, and it does not return a new list.
pop()	list.pop([index])	The pop() method removes and returns the item at the given index. If you omit the index, it returns the rightmost item.

Note: Replace the term “list” mentioned in the syntax with your actual list name in your code.

2.7.1.7 Built-In Functions Used on Lists

Many built-in functions accept a list as an argument.

Table -13: Built-In Functions Used on Lists

Built-In Functions	Description
len()	The len() function returns the number of items in a list.
sum()	The sum() function returns the sum of numbers in the list.

any()	The any() function returns True if any of the Boolean values in the list is True.
all()	The all() function returns True if all the Boolean values in the list are True, else returns False.
sorted()	The sorted() function returns a modified copy of the list while leaving the original list untouched.

```
len(stringlist)
```

```
will produce: 5
```

```
sum(numlist)
```

```
will produce:175
```

```
max(numlist)
```

```
will produce:50
```

```
min(numlist)
```

```
will produce:20
```

```
any([1, 1, 0, 0, 1, 0])
```

```
will produce:True
```

```
all([1, 1, 1, 1])
```

```
will produce:True
```

```
sorted_stringlist = sorted(stringlist)
```

```
sorted_stringlist
```

```
will produce: ['ganesh', 'mahesh', 'ramesh', 'suresh', 'vishnu']
```

2.7.1.8 Nested List

A list inside another list is called a **nested list**, and you can get the behaviour of nested lists in Python by storing lists within the elements of another list. The syntax for nested lists is:

```
nested_list_name = [[item_1, item_2, item_3],
                    [item_4, item_5, item_6],
                    [item_7, item_8, item_9]]
```

Each inner list is separated by a comma. You can traverse the items in nested lists using a for loop.

```
asia = [["India", "Japan", "Korea"],
        ["Srilanka", "Myanmar", "Thailand"],
        ["Cambodia", "Vietnam", "Israel"]]
```

here,
asia[0]
will produce: ['India', 'Japan', 'Korea']
asia[0][1]
will produce: 'Japan'
asia[1][2]
will produce: 'Thailand'

Python Program to find the Transpose of a Matrix:

```
1. matrix = [[10, 20],  
2.           [30, 40],  
3.           [50, 60]]  
4. matrix_transpose = [[0, 0, 0],  
5.                     [0, 0, 0]]  
6. for rows in range(len(matrix)):  
7.     for columns in range(len(matrix[0])):  
8.         matrix_transpose[columns][rows] = matrix[rows][columns]  
9. print("Transposed Matrix is")  
10. for items in matrix_transpose:  
11.     print(items)
```

Screen shot of execution of above Program :

```
main.py x  
1 matrix = [[10, 20], [30, 40], [50, 60]]  
2 matrix_transpose = [[0, 0, 0], [0, 0, 0]]  
3 for rows in range(len(matrix)):  
4     for columns in range(len(matrix[0])):  
5         matrix_transpose[columns][rows] = matrix[rows][columns]  
6 print("Transposed Matrix is:")  
7 for items in matrix_transpose:  
8     print(items)  
9  
main x  
:  
C:\Users\HP\PycharmProjects\PythonProject1\.venv\Scripts\python.exe C:\Users\HP\PycharmProjects\PythonProject1\main.py  
Transposed Matrix is:  
[10, 30, 50]  
[20, 40, 60]  
Process finished with exit code 0
```

2.7.2 Tuple

In mathematics, a tuple is a finite ordered list (sequence) of elements. A **tuple** is defined as a data structure that comprises an ordered, finite sequence of immutable, heterogeneous elements that are of fixed sizes. Often, we may want to return more than one value from a function. Tuples solve this problem. They can also be used to pass multiple values through one function parameter.

2.7.2.1 Creating Tuples

A tuple is a finite ordered list of values of possibly different types which is used to bundle related values together without having to create a specific type to hold them. Tuples are immutable. Once created, you cannot change a tuple's values. A tuple is defined by putting a comma-separated list of values inside parentheses. Each value inside a tuple is called an item. The syntax for creating tuples is:

```
tuple_name = (item_1, item_2, item_3, ....., item_n)
```

Syntactically, a tuple is a comma-separated list of values:

```
t = 'a', 'b', 'c', 'd', 'e'
```

Although it is not necessary, it is common to enclose tuples in parentheses to help us quickly identify tuples when Python code is looked at:

```
t = ('a', 'b', 'c', 'd', 'e')
```

The comma forms a tuple, so the commas matter, not the parentheses.

To create a tuple with a single element, one must include the final comma:

```
t1 = ('a',)
```

here,

```
type(t1)
```

will produce: tuple

In Python, the built-in data type for a tuple type is **tuple**. It is worth noting that without the comma, Python treats ('a') as an expression with a string in parentheses that evaluates to a string:

For example,

```
t2 = ('a')
```

and the `type(t2)`

will produce: str

One can create an empty tuple without any values. For that, the syntax is:

```
tuple_name = ()
```

```
empty_tuple = ()
```

```
empty_tuple
```

will produce: ()

One can store any type of type string, number, object, another variable, and even another tuple itself. One can have a mix of different types of items in tuples, and they need not be homogeneous.

if,

```
socis = ('mca','6 sem','3-6 yrs',54000)
```

and,

```
ignou = ('soe','soh',socis,'soms')
```

then,

```
ignou
```

will have: ('soe', 'soh', ('mca', '6 sem', '3 – 6 yrs', 54000), 'soms')

2.7.2.2 Basic Tuple Operations

Most list operators also work on tuples. The bracket operator indexes an element:

When,

```
t = ('a', 'b', 'c', 'd', 'e')
```

```
t[0]
```

will produce: 'a'

And the slice operator selects a range of elements.

```
t[1:3]
```

will produce: ('b', 'c')

But if you try to modify one of the elements of the tuples, you will get an error:

```
t[0] = 'A'
```

TypeError: 'tuple' object does not support item assignment

We can not modify the elements of a tuple, but can replace one tuple with another:

```
t = ('A',) + t[1:]
```

now,

```
t
```

will have: ('A', 'b', 'c', 'd', 'e')

Here, + operator is used to concatenate two tuples together. Similarly, * operator can be used to repeat a sequence of tuple items.

```
t * 2
```

will produce: ('A', 'b', 'c', 'd', 'e', 'A', 'b', 'c', 'd', 'e')

in and **not in** membership operators are used to check for the presence of an item in a tuple.

Comparison operators like <, <=, >, >=, == and != are used to compare tuples.

Python starts by comparing the first element from each sequence. If they are equal, it goes on to the next element, and so on, until it finds elements that differ. Subsequent elements are not considered even if they are really big.

```
(0, 1, 2) < (0, 3, 4)
```

will produce: True

and

```
(0, 1, 500000) < (0, 3, 4)
```

will produce: True

2.7.2.3 The tuple() Function

The built-in tuple() function is used to create a tuple. The syntax for the tuple() function is:

```
tuple([sequence])
```

where sequence can be a number, string or tuple itself. If the optional sequence is not specified, then an empty tuple is created.

```
t3 = tuple()
```

t3

will produce: ()

If the argument is a sequence (string, list or tuple), the result of the call to tuple is a tuple with the elements of the sequence:

```
t3 = tuple('IGNOU')
```

then,

t3

will produce: ('I', 'G', 'N', 'O', 'U')

```
t4 = (1, 2, 3, 4)
```

```
nested_t = (t3, t4)
```

then,

nested_t

will produce: (('I', 'G', 'N', 'O', 'U'), (1, 2, 3, 4))

2.7.2.4 Built-In Functions Used on Tuples

There are many built-in functions, as listed in Table -14, for which a tuple can be passed as an argument.

Table -14: Built-In Functions Used on Tuples

Built-In Functions	Description
len()	The len() function returns the number of items in a tuple.
sum()	The sum() function returns the sum of numbers in the tuple.
sorted()	The sorted() function returns a sorted copy of the tuple as a list while leaving the original tuple untouched.

```
len(t3)
```

will produce: 5

```
sum(t4)
```

will produce: 10

```
t5 = sorted(t3)
```

then,

```
t5
```

will produce: ['G', 'I', 'N', 'O', 'U']

2.7.2.5 Tuple Assignment

One of the unique syntactic features of the Python language is the ability to have a tuple on the left side of an assignment statement. This lets you assign multiple variables at once when the left side is a sequence.

In this example, we have a two-element list (a sequence) and assign its first and second elements to the variables x and y in a single statement.

```
m = ['good', 'luck']
```

```
x, y = m
```

```
x
```

will produce: 'good'

```
y
```

will produce: 'luck'

Python roughly translates the tuple assignment syntax to be the following:

```
m = ['good', 'luck']
```

```
x = m[0]
```

```
y = m[1]
```

Stylistically, when we use a tuple on the left side of the assignment statement, we omit the parentheses, but the following is an equally valid syntax:

```
(x, y) = m
```

2.7.2.6 Relation between Tuples and Lists

Though tuples may seem similar to lists, they are often used in different situations and for different purposes. Tuples are immutable and usually contain a heterogeneous sequence of elements accessed via unpacking or

indexing. Lists are mutable, and you access their items via indexing. You cannot add, remove, or replace items in a tuple.

If an item within a tuple is mutable, then you can change it. Consider the presence of a list as an item in a tuple; then any changes to the list get reflected in the overall items in the tuple.

```
if
ignou = ['soe', 'soms', 'socis']
univ = ('du', 'ggsipu', 'jnu', ignou)
then,
univ
will have: ('du', 'ggsipu', 'jnu', ['soe', 'soms', 'socis'])
and after
univ[3].append('soss')
univ
will have: ('du', 'ggsipu', 'jnu', ['soe', 'soms', 'socis', 'soss'])
```

2.7.2.7 Tuple Methods

To get a list of all the methods associated with the tuple, pass the tuple to `dir()`.

Methods associated with tuples are listed in Table - 15

Table -15: Tuple Methods

Tuple Methods	Syntax	Description
<code>count()</code>	<code>tuple_name.count(item)</code>	The <code>count()</code> method counts the number of times the item has occurred in the tuple and returns it.
<code>index()</code>	<code>tuple_name.index(item)</code>	The <code>index()</code> method searches for the given item from the start of the tuple and returns its index. If the value appears more than once, it returns the index of the first occurrence. If the item is not present in the tuple, then <code>ValueError</code> is thrown by this method.

Note: You may replace the word “`tuple_name`” mentioned in the syntax with your actual tuple name in the code.

Consider followings:

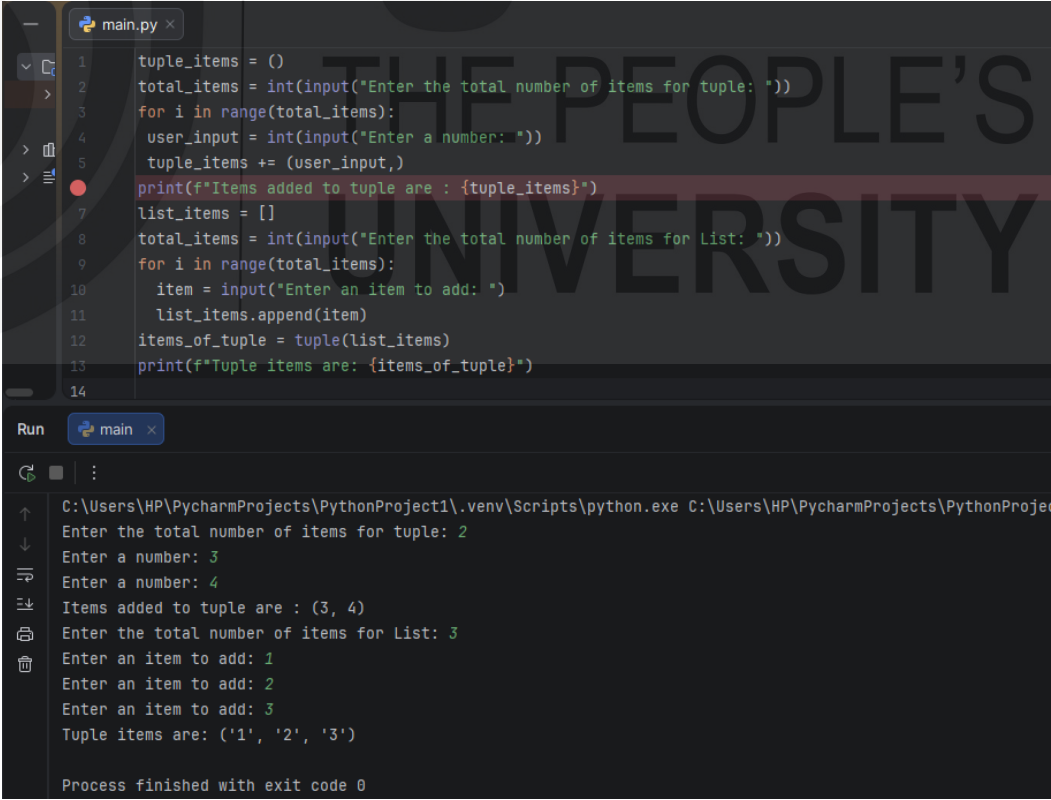
```
channels = ("dd", "star", "sony", "zee", "dd", "sab")
channels.count("dd")
```

will result: 2
channels.index("dd")
will give: 0
channels.index("zee")
will give: 3
and
channels.count("sab")
will give: 1

Program for Creating/Populating Tuple and List with User-Entered Items

```
tuple_items = ()
total_items = int(input("Enter the total number of items for tuple: "))
for i in range(total_items):
    user_input = int(input("Enter a number: "))
    tuple_items += (user_input,)
print(f"Items added to tuple are : {tuple_items}")
list_items = []
total_items = int(input("Enter the total number of items for List: "))
for i in range(total_items):
    item = input("Enter an item to add: ")
    list_items.append(item)
items_of_tuple = tuple(list_items)
print(f"Tuple items are: {items_of_tuple}")
```

Screenshot of execution of the above program:



The screenshot shows a Python IDE with a file named 'main.py' open. The code in the editor is as follows:

```
1 tuple_items = ()
2 total_items = int(input("Enter the total number of items for tuple: "))
3 for i in range(total_items):
4     user_input = int(input("Enter a number: "))
5     tuple_items += (user_input,)
6     print(f"Items added to tuple are : {tuple_items}")
7 list_items = []
8 total_items = int(input("Enter the total number of items for List: "))
9 for i in range(total_items):
10     item = input("Enter an item to add: ")
11     list_items.append(item)
12 items_of_tuple = tuple(list_items)
13 print(f"Tuple items are: {items_of_tuple}")
14
```

The output window shows the following execution:

```
Enter the total number of items for tuple: 2
Enter a number: 3
Enter a number: 4
Items added to tuple are : (3, 4)
Enter the total number of items for List: 3
Enter an item to add: 1
Enter an item to add: 2
Enter an item to add: 3
Tuple items are: ('1', '2', '3')
Process finished with exit code 0
```

Items are inserted into the tuple using two methods: using the continuous concatenation += operator and by converting list items to tuple items. In the code, tuple_items is of tuple type. In both methods, the code specifies the total number of items to insert into the tuple beforehand. Based on this number, the loop is iterated using the range() function. In the first method,

the user-entered items are continuously concatenated to the tuple using the += operator. Tuples are immutable and should not be changed. During each iteration, each original_tuple is replaced by original_tuple + (new_element), thus creating a new tuple. Notice a comma after new_element. In the second method, we create a list. For each iteration, the user-entered value is appended to the list_variable. This list is then converted

Program to Swap Two Numbers without Using Intermediate/Temporary

```
a = int(input("Enter a value for the first number:"))
b = int(input("Enter a value for the second number:"))
b,a = a,b
print("After Swapping")
print(f"Value for first number: {a}")
print(f"Value for second number: {b}")
```

Screenshot of execution of the program:

The contents of variables a and b are reversed. The tuple variables are on the left side of the assignment operator, and the tuple values are on the right side. The number of variables on the left and the number of values on the right must match. Each value is assigned to its respective variable.

Check your progress – 2

- Which of the following statements creates an empty list in Python?
 - l = []
 - l = list()
 - Both a and b
 - Neither a nor b
- What error is raised in Python when attempting to access a list element using an invalid index position?
 - NameError: list is not defined
 - IndexError: list index out of range
 - BoundsError: list index out of bounds
 - No error (returns None)
- Given tupl = (5, 2, 7, 0, 3), which operation executes without error?
 - tupl[1] = 2
 - tupl.append(2)
 - tupl1 = tupl + tupl
 - tupl.sort()

4) Why is a list said to be mutable?

.....
.....

5) Explain the use of the “in” and “not in” operators in a list?

.....
.....

6) Why can tuples contain mutable items?

.....
.....

7) Show how to initialise a list of tuples.

.....
.....

2.7.3 Dictionaries

In the real world, you have seen your contact list on your phone. It is practically impossible to memorise everyone’s mobile number. In the contact list, you store the person's name as well as their number. This lets you identify a mobile number by a person’s name. One can think of a person’s name as the key that retrieves his mobile number, which is the value associated with the key. So, a dictionary can be thought of as:

- an unordered collection of key-value pairs, with the requirement that the keys are unique within a dictionary.
- as a mapping between a set of indices (which are called keys) and a set of values. Each key maps to a value. The association of a key and a value is called a key-value pair or sometimes an item.
- has a length, specifically the number of key-value pairs.
- provides fast lookup by key.

Dictionary keys must be immutable object types and are case-sensitive. Keys can be strings or numbers. But lists can not be used as keys. A value in the dictionary can be of any data type, including string, number, list or dictionary itself.

Dictionaries are constructed using curly braces {}, wherein a list of key: value pairs gets separated by commas. There is a colon(:) separating each of these **key: value** pairs, where the words to the left of the colon operator are the keys and the words to the right of the colon operator are the values.

The syntax for creating a dictionary is :

```
dictionary_name = {key_1:value_1, key_2:value_2, key_3:value_3, ..... ,  
key_n : value_n}
```

Example:

```
eng2sp = {'one' : 'uno', 'two' : 'dos', 'three' : 'tres'}
```

```
print(eng2sp)
```

will display: {'one' : 'uno', 'two' : 'dos', 'three' : 'tres'}

With Python 3.6 onwards, dictionary output shows ordered key:value pairs. Here, ordered means “insertion ordered”, i.e, dictionaries remember the order in which the **key:value** pairs were inserted. The elements of a dictionary are never indexed with integer indices. Instead, the keys are used to look up the corresponding values:

```
print(eng2sp['two'])
```

will display: dos

The key 'two' always maps to the value 'dos' so the order of the items does not matter. If the key is not in the dictionary, one get an exception:

```
print( eng2sp['four'])
```

will display: KeyError: 'four'

Slicing is not allowed in dictionaries because they are not ordered like lists.

2.7.3.1 Built-in Functions Used on Dictionaries

There are many built-in functions for which a dictionary can be passed as an argument. The main operations on a dictionary are storing a value with some key and extracting the value for a given key.

Table – 16: Built-In Functions Used on Dictionaries

Built-in Functions	Description
len()	The len() function returns the number of items (key:value pairs) in a dictionary.
all()	The all() function returns the Boolean value True if all the keys in the dictionary are True; otherwise, it returns False.
any()	The any() function returns the Boolean value True if any of the keys in the dictionary is True; otherwise, it returns False.
sorted()	The sorted() function by default returns a list of items, which are sorted based on dictionary keys.

```
print( len(eng2sp))
```

will display: 3

len() function can be used to find the number of **key:value** pairs in the dictionary eng2sp. In Python, any non-zero integer value is True, and zero is interpreted as False. With all() function, if all the keys are Boolean True values, then the output is True else it is False.

For **any()** function, if any one of the keys is True then it results in a True Boolean value else False Boolean.

```
dict_func = {0 : True, 1 : False, 4 : True}
```

```
print(all(dict_func))  
  
will display: False  
  
and  
  
print( any(dict_func))  
  
will display: True
```

The **sorted()** function returns the sorted list of keys by default in ascending order without modifying the original key:value pairs. For list of keys sorted in descending order by passing the second argument as `reverse = True`.

```
sorted(eng2sp)  
  
['one', 'three', 'two']  
  
print(sorted(eng2sp, reverse= False))  
  
will display: ['one', , 'three', 'two']
```

The **in** operator works on dictionaries; it tells whether something appears as a key in the dictionary (appearing as a value is not good enough).

```
'one' in eng2sp  
  
will result: True  
  
and  
  
'uno' in eng2sp  
  
will result: False
```

The **in** operator uses different algorithms for lists and dictionaries. For lists, it uses a linear search algorithm. As a list gets longer, search time increases in direct proportion to the list's length. For dictionaries, Python uses an algorithm called a hash table, so the **in** operator takes about the same amount of time no matter how many items there are in a dictionary.

2.7.3.2 Dictionary Methods

Various methods associated with dictionary are given in Table -17.

Table -17: Various Dictionary Methods

Dictionary Methods	Syntax	Description
clear()	dictionary_name.clear()	The clear() method removes all the key:value pairs from the dictionary.
fromkeys()	dictionary_name. fromkeys(seq[, value])	The fromkeys() method creates a new dictionary from the given sequence of elements with a value provided by the user.
get()	dictionary_name. get(key[, default])	The get() method returns the value associated with the specified key in the dictionary. If the

		key is not present, then it returns the default value. If default is not given, it defaults to None, so that this method never raises a KeyError.
items()	dictionary_name.items()	The items() method returns a new view of the dictionary's key and value pairs as tuples.
keys()	dictionary_name.keys()	The keys() method returns a new view consisting of all the keys in the dictionary.
pop()	dictionary_name. pop(key [, default])	The pop() method removes the key from the dictionary and returns its value. If the key is not present, then it returns the default value. If the default is not given and the key is not in the dictionary, then it results in KeyError.
popitem()	dictionary_name. popitem ()	The popitem() method removes and returns an arbitrary (key, value) tuple pair from the dictionary. If the dictionary is empty, then calling popitem() results in KeyError.
setdefault()	dictionary_name. setdefault(key[, default])	The setdefault() method returns a value for the key present in the dictionary. If the key is not present, then insert the key into the dictionary with a default value and return the default value. If the key is present, the default is None, so this method never raises a KeyError.
update()	dictionary_name. update([other])	The update() method updates the dictionary with the key:value pairs from another dictionary object, and it returns None.
values()	dictionary_name.values()	The values() method returns a new view consisting of all the values in the dictionary.

Note: Replace the word “dictionary_name” mentioned in the syntax with the actual dictionary name.

Example:

```
million_dollar = {"sanju" : 2018, "tiger zindahai" : 2017, "baahubali 2" : 2017, "dangal" : 2016, "bajrangibhaijaan" : 2015}
```

```
bolwd_million_dollar = million_dollar.fromkeys(million_dollar, "1,00,00,000")
```

```
print(bolwd_million_dollar)
```

will display:

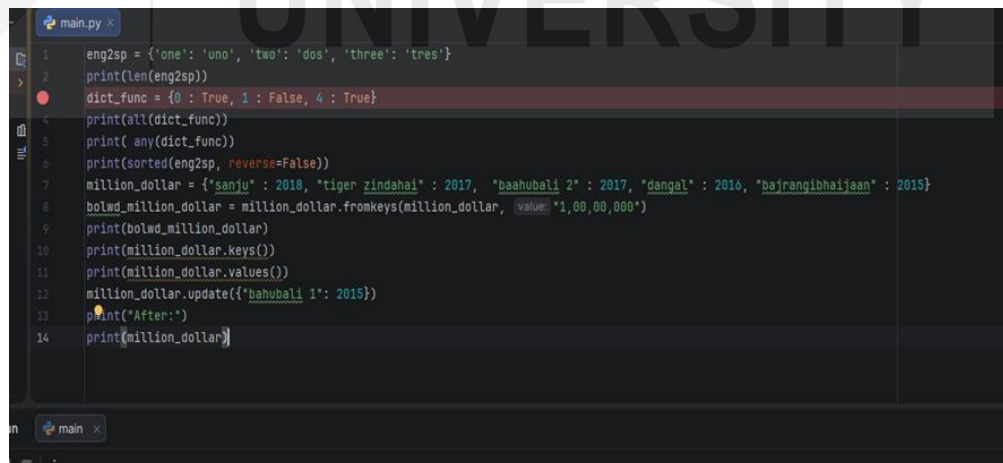
```
{'sanju': '1,00,00,000', 'tiger zindahai': '1,00,00,000',  
 'baahubali 2': '1,00,00,000', 'dangal': '1,00,00,000',
```

```
'bajrangibhaijaan': '1,00,00,000'}  
  
print( million_dollar.get("baahubali 1"))  
will display:None  
  
print(million_dollar.get("baahubali 2"))  
will display: 2017  
  
print( million_dollar.keys())  
will display: dict_keys(['sanju', 'tiger zindahai', 'baahubali 2', 'dangal',  
'bajrangibhaijaan'])  
  
print(million_dollar.values())  
will display: dict_values([2018, 2017, 2017, 2016, 2015])  
  
print(million_dollar.items())  
will display: dict_items([('sanju', 2018), ('tiger zindahai', 2017),  
( 'baahubali 2', 2017), ('dangal', 2016), ('bajrangibhaijaan', 2015)])
```

When we update :

```
million_dollar.update({"bahubali 1" : 2015})  
print(million_dollar)  
will display: {'sanju': 2018, 'tiger zindahai': 2017,  
'baahubali 2': 2017, 'dangal': 2016, 'bajrangibhaijaan': 2015,  
'bahubali 1': 2015}
```

Screenshot of execution of the above functions:



```
main.py  
1 eng2sp = {'one': 'uno', 'two': 'dos', 'three': 'tres'}  
2 print(len(eng2sp))  
3 dict_func = {0 : True, 1 : False, 4 : True}  
4 print(all(dict_func))  
5 print(any(dict_func))  
6 print(sorted(eng2sp, reverse=False))  
7 million_dollar = {'sanju' : 2018, 'tiger zindahai' : 2017, 'baahubali 2' : 2017, 'dangal' : 2016, 'bajrangibhaijaan' : 2015}  
8 bold_million_dollar = million_dollar.fromkeys(million_dollar, value="1,00,00,000")  
9 print(bold_million_dollar)  
10 print(million_dollar.keys())  
11 print(million_dollar.values())  
12 million_dollar.update({'bahubali 1': 2015})  
13 print("After:")  
14 print(million_dollar)
```

```

C:\Users\HP\PycharmProjects\PythonProject1\.venv\Scripts\python.exe C:\Users\HP\PycharmProjects\PythonProject1\main.py
3
False
True
['one', 'three', 'two']
{'sanju': '1,00,00,000', 'tiger zindahai': '1,00,00,000', 'bahubali 2': '1,00,00,000', 'dangal': '1,00,00,000', 'bajrangibhaijaan': '1,00,00,000'}
dict_keys(['sanju', 'tiger zindahai', 'bahubali 2', 'dangal', 'bajrangibhaijaan'])
dict_values(['1,00,00,000', '1,00,00,000', '1,00,00,000', '1,00,00,000', '1,00,00,000'])
After:
{'sanju': 2018, 'tiger zindahai': 2017, 'bahubali 2': 2017, 'dangal': 2016, 'bajrangibhaijaan': 2015, 'bahubali 1': 2015}
Process finished with exit code 0

```

One common way to populate dictionaries is to start with an empty dictionary {}, then use the update() method to assign a value to a key using the assignment operator. If the key does not exist, then the key:value pairs will be created automatically and added to the dictionary.

```
states = {}
```

```
states.update({"Haryana":"Chandigarh"})
```

```
states.update({"Bihar":"Patna"})
```

```
states.update({"west bengal" : "kolkata"})
```

```
print( states)
```

will display: {'Haryana': 'Chandigarh', 'Bihar': 'Patna', 'west bengal': 'kolkata'}

```

1 states = {}
2 states.update({"Haryana":"Chandigarh"})
3 states.update({"Bihar":"Patna"})
4 states.update({"west bengal" : "kolkata"})
5 print( states)
6
Run main x
C:\Users\HP\PycharmProjects\PythonProject1\.venv\Scripts\python.exe C:\Users\HP\PycharmProjects\PythonProject1\main.py
{'Haryana': 'Chandigarh', 'Bihar': 'Patna', 'west bengal': 'kolkata'}
Process finished with exit code 0

```

If a dictionary is used as the sequence in a for statement, it traverses the keys of the dictionary. This loop prints each key and the corresponding value:

```
for key in states :
```

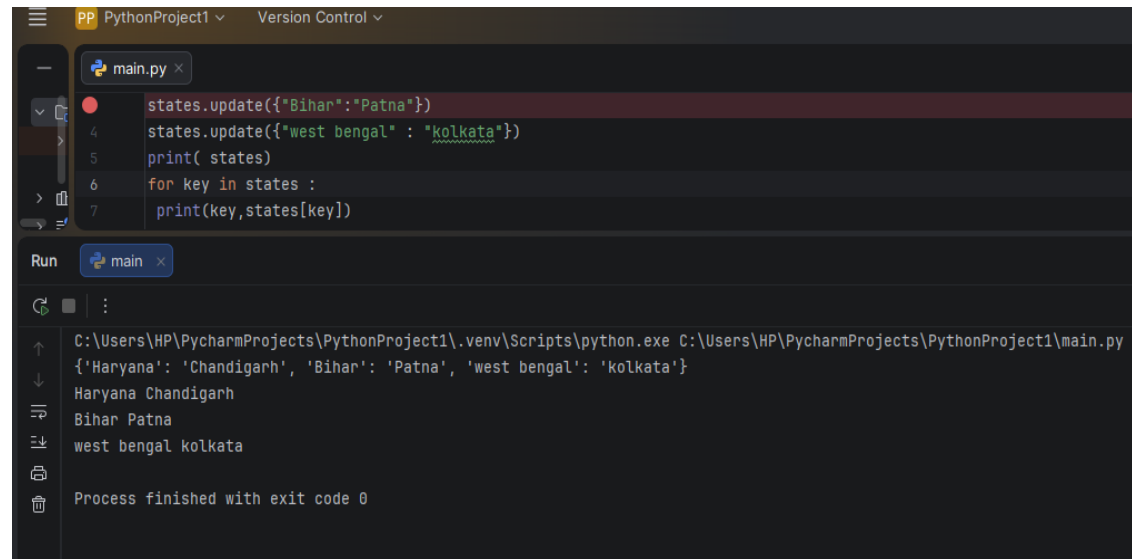
```
print(key, states[key])
```

will display:

```

Haryana Chandigarh
Bihar Patna
west bengal kolkata

```



```
main.py
1 states.update({"Bihar": "Patna"})
2 states.update({"west bengal" : "kolkata"})
3 print( states)
4
5 for key in states :
6     print(key,states[key])
```

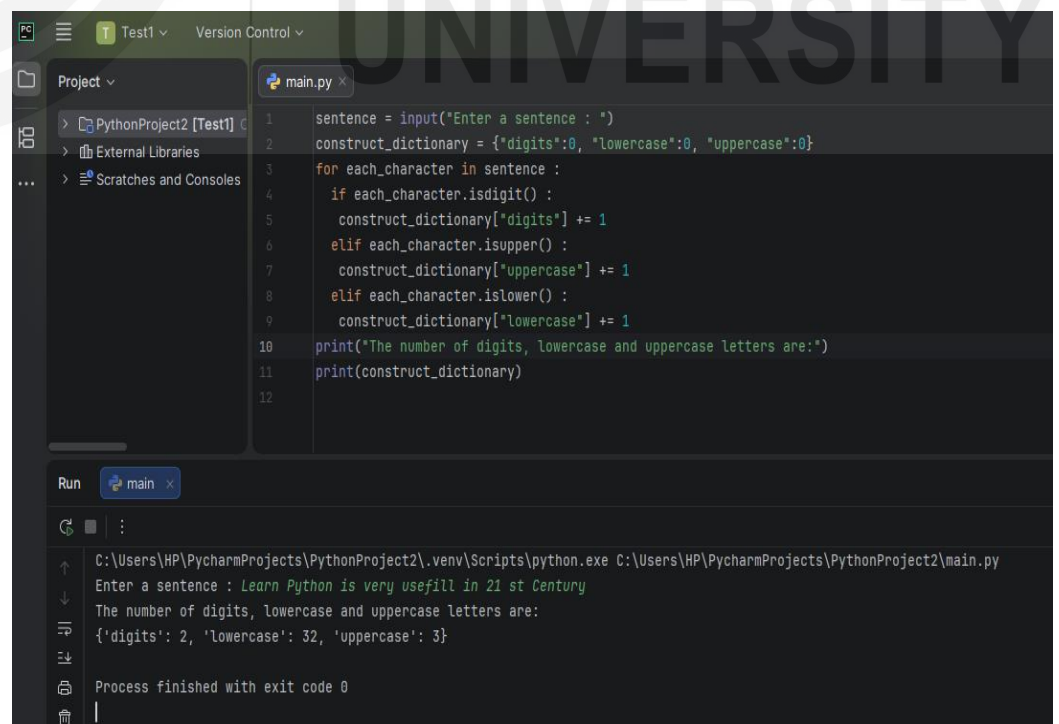
Run main

```
C:\Users\HP\PycharmProjects\PythonProject1\.venv\Scripts\python.exe C:\Users\HP\PycharmProjects\PythonProject1\main.py
{'Haryana': 'Chandigarh', 'Bihar': 'Patna', 'west bengal': 'kolkata'}
Haryana Chandigarh
Bihar Patna
west bengal kolkata
Process finished with exit code 0
```

Python Program which accepts a sentence and calculate the number of digits, uppercase laetters and owercase letters using dictionary:

```
sentence = input("Enter a sentence : ")
construct_dictionary = {"digits":0, "lowercase":0, "uppercase":0}
for each_character in sentence :
    if each_character.isdigit() :
        construct_dictionary["digits"] += 1
    elif each_character.isupper() :
        construct_dictionary["uppercase"] += 1
    elif each_character.islower() :
        construct_dictionary["lowercase"] += 1
print("The number of digits, lowercase and uppercase letters are:")
print(construct_dictionary)
```

Fig. 4: Screen Shot of execution of above Python program



```
main.py
1 sentence = input("Enter a sentence : ")
2 construct_dictionary = {"digits":0, "lowercase":0, "uppercase":0}
3 for each_character in sentence :
4     if each_character.isdigit() :
5         construct_dictionary["digits"] += 1
6     elif each_character.isupper() :
7         construct_dictionary["uppercase"] += 1
8     elif each_character.islower() :
9         construct_dictionary["lowercase"] += 1
10 print("The number of digits, lowercase and uppercase letters are:")
11 print(construct_dictionary)
```

Run main

```
C:\Users\HP\PycharmProjects\PythonProject2\.venv\Scripts\python.exe C:\Users\HP\PycharmProjects\PythonProject2\main.py
Enter a sentence : Learn Python is very usefill in 21 st Century
The number of digits, lowercase and uppercase letters are:
{'digits': 2, 'lowercase': 32, 'uppercase': 3}
Process finished with exit code 0
```

It is very important to note in the above program that proper Python code indentation is necessary. You have to remember that in Python, indentation is a strict syntactical requirement used to define the structure and scope of code blocks. Other programming languages like Java, C, or C++ use curly braces {} to group statements; Python relies entirely on whitespace at the beginning of a line to determine which commands belong together.

Dictionary **del** statement :

To delete the **key:value pair**, use the **del** statement followed by the name of the dictionary along with the key you want to delete.

if

```
states contains: {'Haryana': 'Chandigarh', 'Bihar': 'Patna', 'west bengal': 'kolkata'}
```

then after

```
del states["west bengal"]
```

```
states will contain: {'Haryana': 'Chandigarh', 'Bihar' : 'Patna'}
```

2.7.3.3 Relation between Tuples and Dictionaries

In Python, tuples and dictionaries are two closely interconnected data structures. Although they have completely different underlying architectures, they still frequently work together to convert, index, and unpack data. You can use tuples as key:value pairs to build dictionaries.

```
If tuple pm_year =  
(('jln',1947),('lbs',1964),('ig',1966),('rg',1984),('pvn',1991),('abv',1998),('nm',2014))
```

Then

```
print( pm_year ) will display:
```

```
(('jln', 1947),  
 ('lbs', 1964),  
 ('ig', 1966),  
 ('rg', 1984),  
 ('pvn', 1991),  
 ('abv', 1998),  
 ('nm', 2014))
```

If you construct dictionary using tuple pm_year:

```
pm_year_dict = dict(pm_year)
```

then

```
print( pm_year_dict ) will display:
```

```
{'jln': 1947,  
 'lbs': 1964,  
 'ig': 1966,
```

```
'rg': 1984,  
'pvn': 1991,  
'abv': 1998,  
'nm': 2014}
```

You can convert tuples to dictionaries by passing the tuple name to the **dict()** function. This is achieved by nesting tuples within tuples, where each nested tuple has two items. The first item becomes the key and the second item becomes its value when the tuple gets converted to a dictionary.



The screenshot shows a Python IDE with a project named 'PythonProject2 [Test1]'. The main.py file contains the following code:

```
12  
13 pm_year = (('jln', 1947), ('lbs', 1964), ('ig', 1966), ('rg', 1984), ('pvn', 1991), ('abv', 1998), ('nm', 2014))  
14 print("tuple is:")  
15 print( pm_year )  
16 pm_year_dict = dict(pm_year)  
17 print("dictionary is:")  
18 print( pm_year_dict )  
19
```

The Run window shows the output of the program:

```
C:\Users\HP\PycharmProjects\PythonProject2\.venv\Scripts\python.exe C:\Users\HP\PycharmProjects\PythonProject2\main.py  
tuple is:  
(('jln', 1947), ('lbs', 1964), ('ig', 1966), ('rg', 1984), ('pvn', 1991), ('abv', 1998), ('nm', 2014))  
dictionary is:  
{'jln': 1947, 'lbs': 1964, 'ig': 1966, 'rg': 1984, 'pvn': 1991, 'abv': 1998, 'nm': 2014}  
Process finished with exit code 0
```

2.7.4 Sets

A **set** is an unordered collection with no duplicate items. Primary uses of sets include membership testing and eliminating duplicate entries. Sets also support mathematical operations, such as union, intersection, difference, and symmetric difference.

Curly braces{} or the set() function can be used to create sets with a comma-separated list of items inside curly brackets{}.

Note: In Python to create an empty set you have to use set() and not{} as the latter creates an empty dictionary.

Sets are mutable and indexing is not possible in sets, since set items are unordered. You cannot access or change a set item using indexing or slicing.

Example:

```
basket = {'apple', 'orange', 'apple', 'pear', 'orange', 'banana'}
```

When you print basket, you will get:

```
{'apple', 'banana', 'orange', 'pear'}
```

You may notice that a set is a collection of unique items. Duplicate items are removed from the set basket. Here, the set will contain only one item of 'orange' and 'apple'.

You can check whether an item is in a set or not using the **in keyword**.

'orange' in basket will give **True** and 'grapes' in basket will result to **False**

The built-in function len () gives the total number of elements in a set.

len(basket) will return 4

Set elements sorting:

As sets are inherently unordered collections of unique elements, Python set objects do not have a built-in .sort() method to sort the elements of a set. If you want to sort set elements, you can do it by using the global sorted() function, which accepts the set as an argument and returns a new, sorted list containing the set's elements.

sorted(basket) will produce ['apple', 'banana', 'orange', 'pear']

Some examples:

```
a = set('abrakadabra')
```

then print (a) will display: {'a', 'b', 'd', 'k', 'r'}

```
b = set('alexander')
```

theb print(b) will display: {'a', 'd', 'e', 'l', 'n', 'r', 'x'}

```
a-b          #letters present in set a, but not in set b
```

will result to: {'b', 'k'}

```
a | b          #Letters present in set a, set b, or both
```

will result to: {'a', 'b', 'd', 'e', 'k', 'l', 'n', 'r', 'x'}

```
a & b          #letters present in both set a and set b
```

will result to: {'a', 'd', 'r'} and

```
a ^ b          #letters present in set a or set b, but not both
```

will result to: {'b', 'e', 'k', 'l', 'n', 'x'}

2.7.4.1 Set Methods

You can get a list of all methods associated with a set by passing the set to dir().

Table -18 lists the various methods associated with the set.

Table -18: Various Set Methods

Set Methods	Syntax	Description
add()	set_name.add(item)	The add() method adds an item to the set set_name.
clear()	set_name.clear()	The clear() method removes all the items from the set set_name.
difference()	set_name. difference(*others)	The difference() method returns a new set with items in the set set_name that are not

		in the other sets.
discard()	set_name. discard(item)	The discard() method removes an item from the set set_name if it is present.
intersection()	set_name. intersection(*others)	The intersection() method returns a new set with items common to the set set_name and all other sets.
isdisjoint()	set_name. isdisjoint(other)	The isdisjoint() method returns True if the set set_name has no items in common with the other set. Sets are disjoint if and only if their intersection is the empty set.
issubset()	set_name. issubset(other)	The issubset() method returns True if every item in the set set_name is in the other set.
issuperset()	set_name. issuperset(other)	The issuperset() method returns True if every element in the other set is in the set set_name.
pop()	set_name.pop()	The method pop() removes and returns an arbitrary item from the set set_name. It raises KeyError if the set is empty.
remove()	set_name. remove(item)	The method remove() removes an item from the set set_name. It raises KeyError if the item is not contained in the set.
symmetric_difference()	set_name. symmetric_difference(other)	The method symmetric_difference() returns a new set with items from the set set_name and all other sets.
union()	set_name. union(*others)	The method union() returns a new set with items from the set set_name and all other sets.
update()	set_name. update(*others)	Update the set set_name by adding items from all other sets

Note: Replace the words “set_name”, “other” and “others” mentioned in the syntax with your actual set names in your code.

Python program to demonstrate methods of set:

Program:

```
f1 = {'sunflower', 'roses', 'lavender', 'tulips', 'goldcrest'}
f2 = {'roses', 'tulips', 'lilies', 'daisies'}
f2.add('orchids')
print("Set f1:", f1)
print("Set f2:", f2)
print("Difference f2 and f1:", f2.difference(f1))
print("Intersection f2 and f1:", f2.intersection(f1))
print("Is distinct f2, f1:", f2.isdisjoint(f1))
print("Intersection f2 and f1:", f2.issuperset(f1))
print("Is subset f2 and f1:", f2.issubset(f1))
print("Symmetric Difference f2 and f1:", f2.symmetric_difference(f1))
print("Update f2:", f2.update(f1))
print("Discard f2- rose :", f2.discard("roses"))
print("Pop f1:", f1.pop())
print("Clear f2:", f2.clear())
```

Output:

Set f1: {'goldcrest', 'sunflower', 'lavender', 'tulips', 'roses'}

Set f2: {'tulips', 'roses', 'lilies', 'daisies', 'orchids'}
 Difference f2 and f1: {'lilies', 'daisies', 'orchids'}
 Intersection f2 and f1: {'tulips', 'roses'}
 Is distinct f2, f1: False
 Intersection f2 and f1: False
 Is subset f2 and f1: False
 Symmetric Difference f2 and f1: {'lilies', 'goldcrest', 'sunflower', 'lavender',
 'daisies', 'orchids'}
 Update f2: None
 Discard f2- rose : None
 Pop f1: goldcrest
 Clear f2: None

The screenshot shows a PyCharm IDE with a file named `main.py` containing the following Python code:

```

20 f1 = {'sunflower', 'roses', 'lavender', 'tulips', 'goldcrest'}
21 f2 = {'roses', 'tulips', 'lilies', 'daisies'}
22 f2.add('orchids')
23 print("Set f1:", f1)
24 print("Set f2:", f2)
25 print("Difference f2 and f1:", f2.difference(f1))
26 print("Intersection f2 and f1:", f2.intersection(f1))
27 print("Is distinct f2, f1:", f2.isdisjoint(f1))
28 print("Intersection f2 and f1:", f2.issuperset(f1))
29 print("Is subset f2 and f1:", f2.issubset(f1))
30 print("Symmetric Difference f2 and f1:", f2.symmetric_difference(f1))
31 print("Update f2:", f2.update(f1))
32 print("Discard f2- rose :", f2.discard("roses"))
33 print("Pop f1:", f1.pop())
34 print("Clear f2:", f2.clear())
35

```

The Run console at the bottom displays the output of the code:

```

C:\Users\HP\PycharmProjects\PythonProject2\.venv\Scripts\python.exe C:\Users\HP\PycharmProjects\PythonProject2/main.py
Set f1: {'goldcrest', 'sunflower', 'lavender', 'tulips', 'roses'}
Set f2: {'tulips', 'roses', 'lilies', 'daisies', 'orchids'}
Difference f2 and f1: {'lilies', 'daisies', 'orchids'}
Intersection f2 and f1: {'tulips', 'roses'}
Is distinct f2, f1: False
Intersection f2 and f1: False
Is subset f2 and f1: False
Symmetric Difference f2 and f1: {'lilies', 'goldcrest', 'sunflower', 'lavender', 'daisies', 'orchids'}
Update f2: None
Discard f2- rose : None
Pop f1: goldcrest
Clear f2: None

```

2.7.4.2 Traversing of Sets

One can iterate through each item in a set using a for loop. Example code snippet for such operation is as follows:

for flower in f1 :

 print(flower, “ is a flower”)

will result into:

 lavender is a flower

 tulips is a flower

 goldcrest is a flower

 sunflower is a flower

2.7.4.3 Frozenset

A **frozenset** is basically the same as a set, except that it is *immutable*. Once a frozenset is created, its items cannot be changed. Because they are immutable, you can use them as members of other sets and as dictionary keys. Frozen sets have the same functions as sets, except those functions which change the contents (like update, remove, etc.).

Creating a frozen set:

```
fs = frozenset(["g","o","o","d"])    #creating/declaring frozenset
print( fs)                           #displaying the contents of frozenset
will display: frozenset({'d', 'g', 'o'})
```

here frozenset is used within a set:

```
pets = set([fs, "horse", "cow"]) #Frozenset type is used within a set
print( pets)
will display: {'cow', frozenset({'d', 'g', 'o'}), 'horse'}
```

Another example:

```
lang_used = {"english":59, "french":29, "spanish":21}
print( frozenset(lang_used) ) #keys in a dictionary are returned when
                              a dictionary is passed as an argument
                              to frozenset() function.
will display: frozenset({'english', 'french', 'spanish'})
```

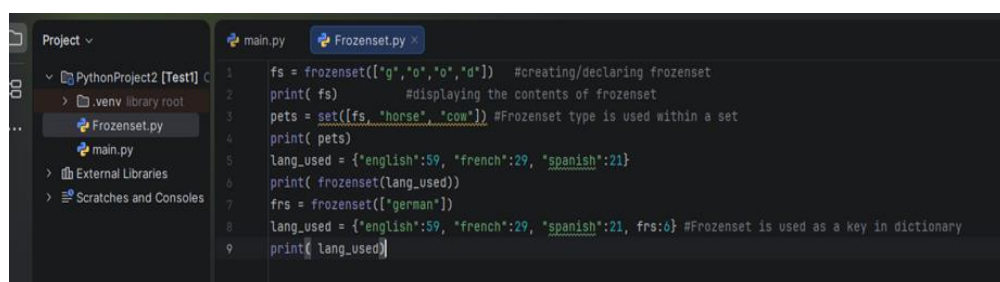
If we create frs as:

```
frs = frozenset(["german"])
#Frozenset is used as a key in dictionary
```

Then we may use frs as:

```
lang_used = {"english":59, "french":29, "spanish":21, frs:6}
print( lang_used)
will now display:
```

```
{'english': 59, 'french': 29, 'spanish': 21, frozenset({'german'}): 6}
```



```
1 fs = frozenset(["g","o","o","d"]) #creating/declaring frozenset
2 print( fs)                       #displaying the contents of frozenset
3 pets = set([fs, "horse", "cow"]) #Frozenset type is used within a set
4 print( pets)
5 lang_used = {"english":59, "french":29, "spanish":21}
6 print( frozenset(lang_used))
7 frs = frozenset(["german"])
8 lang_used = {"english":59, "french":29, "spanish":21, frs:6} #Frozenset is used as a key in dictionary
9 print( lang_used)
```

```

Run FrozenSet x
C:\Users\HP\PycharmProjects\PythonProject2\.venv\Scripts\python.exe C:\Users\HP\PycharmProjects\PythonProject2\FrozenSet.py
frozenset({'o', 'd', 'g'})
{'horse', 'cow', frozenset({'o', 'd', 'g'})}
frozenset({'spanish', 'english', 'french'})
{'english': 59, 'french': 29, 'spanish': 21, frozenset({'german'})}: 6

Process finished with exit code 0

```

Example: Use of Frozen Set

```

# Define sets of permissions as frozensets
admin_perms = frozenset(["read", "write", "delete", "manage_users"])
editor_perms = frozenset(["read", "write"])
viewer_perms = frozenset(["read"])
# Use frozensets as keys in a dictionary mapping to Role Names
role_lookup = { admin_perms: "Administrator", editor_perms: "Editor",
viewer_perms: "Viewer" }
# Simulate checking a user's permissions
current_user_perms = frozenset(["read", "write"])
# Lookup the role directly
role = role_lookup.get(current_user_perms, "Editor")
print(f"User Permissions: {set(current_user_perms)}")
print(f"Assigned Role: {role}")
# Immutability check by trying to modify a frozenset raises an error
try: editor_perms.add("delete")
except AttributeError as e: print(f"\nError Attempted: {e}")

```

```

Project v
PythonProject2 [Test1] C
  .venv library root
  FrozenSet.py
  main.py
  UseFrozenSet.py
External Libraries
Scratches and Consoles

main.py UseFrozenSet.py FrozenSet.py
1
2 # Define sets of permissions as frozensets
3 admin_perms = frozenset(["read", "write", "delete", "manage_users"])
4 editor_perms = frozenset(["read", "write"])
5 viewer_perms = frozenset(["read"])
6 # Use frozensets as keys in a dictionary mapping to Role Names
7 role_lookup = { admin_perms: "Administrator", editor_perms: "Editor", viewer_perms: "Viewer" }
8 # Simulate checking a user's permissions
9 current_user_perms = frozenset(["read", "write"])
10 # Lookup the role directly
11 role = role_lookup.get(current_user_perms, default: "Editor")
12 print(f"User Permissions: {set(current_user_perms)}")
13 print(f"Assigned Role: {role}")
14 # Immutability check by trying to modify a frozenset raises an error
15 try: editor_perms.add("delete")
16 except AttributeError as e: print(f"\nError Attempted: {e}")
17

Run UseFrozenSet x
C:\Users\HP\PycharmProjects\PythonProject2\.venv\Scripts\python.exe C:\Users\HP\PycharmProjects\PythonProject2\UseFrozenSet.py
User Permissions: {'read', 'write'}
Assigned Role: Editor

Error Attempted: 'frozenset' object has no attribute 'add'

Process finished with exit code 0

```

The program above demonstrates how to use a frozenset. This program also uses exception handling, which we will discuss in detail in a unit of the next block of this course.

Check your progress – 3

1. What will be the output of the following Python code?

```
d1={"Ravi":5,"Mohan":6,"Sohan":7}
```

```
print(d1[0])
```

- a) Ravi
- b) 5
- c) {"Ravi":5}
- d) Error

2. What is the output of the following union operation?

```
set1 = {10, 20, 30, 40}
```

```
set2 = {50, 20, "10", 60}
```

```
set3 = set1.union (set2)
```

```
print(set3)
```

- a) {40, 10, 50, 20, 60, 30}
- b) {40, '10', 50, 20, 60, 30}
- c) {40, 10, '10', 50, 20, 60, 30}
- d) SyntaxError: Different types cannot be used with sets

3. What is the output of the following set operation

```
set1 = {"Yellow", "Orange", "Black"}
```

```
set1.discard("Blue")
```

```
print(set1)
```

- a) {'Yellow', 'Orange', 'Black'}
- b) {'Yellow', 'Orange'}
- c) {'Yellow'}
- d) KeyError: 'Blue'

4. What is the difference between a set and a dictionary?

.....

.....

5. What is the difference between remove() and discard() in sets?

.....

.....

2.8 SUMMARY

This unit explained the basics of Python programming, including identifiers and variables. This unit also covered statements and expressions, operators, lists, tuples, sets, and dictionaries, which are the backbone of Python and make it unique and easy to program. These basic structures can be combined with control flow statements, which ultimately form a Python program. Mastering these core elements lays a solid foundation for writing modular, efficient, and readable code. This unit also sets the stage for developing reusable functions and leveraging Python's rich ecosystem of built-in and third-party libraries.

2.9 ANSWER TO CHECK YOUR PROGRESS

Answer to check your progress-1

1. d
2. d
3. b
4. b
5. d
6. b
7. a
8. c

9.

Identifiers must follow these rules:

- Must start with a letter (a–z or A–Z) or underscore (_)
- Cannot start with a digit
- Can contain letters, digits, and underscores
- Cannot be a Python keyword
- Case-sensitive

10 .

Yes, expressions are often part of statements.

`X = a+b`

Here, `a + b` is an expression inside an assignment statement.

Explanation:

`x = 10 + 5`

The Expression: `10 + 5`

- Python first evaluates this expression to produce the value 15.

The Statement: `x = 10 + 5`

- The entire line is an assignment statement. It takes the evaluated value (15) and assigns it to the variable `x`.

11.

No, Python does not require explicit variable declaration. However, it is a must to assign a value to a variable before you try to read or use it.

12.

`/` performs floating-point division and `//` performs floor division

In Python, the key difference between “`/`” and “`//`” is how they handle fractional parts (decimals) in division:

`/` (Floating-Point / True Division): Divides two numbers and always returns a float, keeping the exact decimal value.

Example:

`print(10 / 3)`

Output: 3.3333333333333335

`print(16 / 2)` # Output: 8.0 (Notice it is 8.0, not 5)

- `//` (Floor Division): Divides two numbers and rounds down to the nearest whole number (towards negative infinity), discarding the remainder.

Example:

`print(10 // 3)`

Output: 3

`print(10 // 2)`

Output: 5

Important to note that if either number is a float, in // operation, the result is returned as a float:

```
print(10.0 // 3), output will be : 3.0
```

Answer to check your progress-2

1. c
2. b
3. c
4. A list is said to be mutable because its elements can be changed, added, or removed after the list has been created, without creating a new list.
5. The “in” and “not in” operators in Python are membership operators used with lists to check whether an element is present or absent.
6. Tuples themselves are immutable, meaning their structure (size and element positions) cannot change, but the objects stored inside a tuple can be mutable. Python allows this because immutability applies to the tuple container, not to the contents it references.
7.

```
list_of_tuples = [(1, 2), (3, 4), (5, 6)]  
print(list_of_tuples)
```

Output:

```
[(1, 2), (3, 4), (5, 6)]
```

Answer to Check your progress-3

1. d
2. c
3. a
- 4.

Aspect	Set	Dictionary
Definition	A collection of unique elements	A collection of key–value pairs
Data stored	Only values	Keys mapped to values
Uniqueness	All elements must be unique	Keys must be unique; values may repeat
Syntax	{1, 2, 3}	{"a": 1, "b": 2}
Access	Elements are not accessed by index or key	Values are accessed using keys
Mutability	Mutable (elements can be added/removed)	Mutable (key–value pairs can be added/updated/removed)
Use case	Membership testing, removing duplicates	Storing and retrieving data with meaningful keys

5. In Python sets, both `remove()` and `discard()` are used to delete elements.

i. `remove()`

- Removes the specified element from the set.
- Raises a `KeyError` if the element is not found.

```
my_set = {1, 2, 3}  
my_set.remove(2) # Removes 2  
print(my_set)    # {1, 3}  
my_set.remove(5) # KeyError: 5
```

ii. discard()

- Removes the specified element from the set.
 - Does NOT raise an error if the element is not found; it does nothing.
- ```
my_set = {1, 2, 3}
my_set.discard(2) # Removes 2
print(my_set) # {1, 3}
my_set.discard(5) # No error, does nothing
print(my_set) # {1, 3}
```

---

## 2.10 REFERENCES/FURTHER READINGS

---

1. Yashavant Kanetkar and Aditya Kanetkar, “Let Us Python”, 4<sup>th</sup> Edition, BPB Publications, New Delhi , 2022
2. Paul Deitel and Harvey Deitel , “Python For Programmers” , 1<sup>st</sup> Edition, Pearson India Education , Chennai, 2023
3. Sheetal Taneja and Naveen Kumar, “Python Programming a Modular Approach” , 1<sup>st</sup> Edition, Pearson India Education , Chennai , 2022
4. S. Shridhar, J. Indumathi and V.M. Hariharan , “Python Programming” , 1<sup>st</sup> Edition, Pearson India Education , Chennai, 2024
5. S. Suma Christal Mary , R. Josphine Leela, N. Senthamilarasi and Y. Chrisbin Jeeva , “Problem Solving and Python Programming”, 1<sup>st</sup> Edition, Ane Books , New Delhi, 2018

---

## UNIT 3 PYTHON'S CONTROL FLOW STRUCTURES

---

### Structure

- 3.1 Introduction
- 3.2 Objectives
- 3.3 Control Flow Statements
  - 3.3.1 Sequential Control Flow Statements
  - 3.3.2 Decision Control Flow Statements
    - 3.3.2.1 The if Decision Control Flow Statement
    - 3.3.2.2 The if...else Decision Control Flow Statement
    - 3.3.2.3 The if...elif...else Decision Control Statement
    - 3.3.2.4 Nested if Statement
  - 3.3.3 Loop Control Flow Statements
    - 3.3.3.1 The while loop
    - 3.3.3.2 The for loop
    - 3.3.3.3 The continue and break Statements
- 3.4 Summary
- 3.5 Solution to Check your Progress
- 3.6 Reference/Further Readings

---

### 3.1 INTRODUCTION

---

In the previous unit, we have mainly seen programming examples in which programs execute line by line, top to bottom—following a strict, linear path. However, real-world problems are rarely that straightforward. Programs often need to make decisions based on changing data, repeat tasks efficiently, or skip/jump unnecessary steps. In this unit, you will learn about Control Flow Statements. These are the core mechanisms that direct the order in which code executes. You will explore how to transition from basic sequential execution to decision-making structures (if, if...else, if...elif...else, and nested conditions) that dynamically alter execution paths. In this unit, you will also learn to handle repetitive logic using loops (for and while) alongside control keywords like break and continue. Understanding these fundamental control structures will help you build flexible, efficient, and logical Python programs that can handle complex decision-making processes.

---

### 3.2 OBJECTIVES

---

After going through this unit, you will be able to:

- Understand usage of sequential control flow statements,
- Use decision structures in programming,
- Use nested decision making mechanisms in programming,
- Use looping constructs in programming, and

- Use continue and break statements.

---

## 3.3 CONTROL FLOW STATEMENTS

---

Python supports a set of control flow statements that you can integrate into your program. The statements inside your Python program are generally executed sequentially from top to bottom in the order that they appear. Apart from sequential control flow statements, you can use decision-making and looping control flow statements to break up the flow of execution, thus enabling your program to conditionally execute a particular block of code. Control flow describes the direction the program takes.

The control flow statements in the Python Programming Language are:

### 3.3.1 Sequential Control Flow Statements :

This refers to the line-by-line execution, in which the statements are executed sequentially, in the same order in which they appear in the program.

Example: Taking three values as input and displaying their sum

```
Prompt the user to enter three numbers
num1 = float(input("Enter the first number: "))
num2 = float(input("Enter the second number: "))
num3 = float(input("Enter the third number: "))
Calculate the sum
total_sum = num1 + num2 + num3
Display the result
print(f"The sum of {num1}, {num2}, and {num3} is: {total_sum}")
```

The above program is an example of sequential control flow. Also, one important point to note in the above program is that in line:

```
print(f"The sum of {num1}, {num2}, and {num3} is: {total_sum}")
```

The **f** stands for Formatted String Literal, commonly known as an **f-string**.

When you put an **f** right before the opening quote of a string, it tells Python: "Look inside this string for curly braces {} and replace them with the actual values of the variables inside them."

The screenshot shows the PyCharm IDE with a project named 'PythonProject2 [Test1]'. The file explorer on the left shows files: 'main.py', 'SequentialStatement.py', and 'UseFrozenSet.py'. The main editor displays the code in 'SequentialStatement.py':

```

1 # Prompt the user to enter three numbers
2 num1 = float(input("Enter the first number: "))
3 num2 = float(input("Enter the second number: "))
4 num3 = float(input("Enter the third number: "))
5 # Calculate the sum
6 total_sum = num1 + num2 + num3
7 # Display the result
8 print(f"The sum of {num1}, {num2}, and {num3} is: {total_sum}")

```

The Run console at the bottom shows the execution output:

```

C:\Users\HP\PycharmProjects\PythonProject2\.venv\Scripts\python.exe C:\Users\HP\PycharmProjects\PythonProject2\SequentialStatement.py
Enter the first number: 1
Enter the second number: 2
Enter the third number: 4
The sum of 1.0, 2.0, and 4.0 is: 7.0
Process finished with exit code 0

```

### 3.3.2 Decision Control Flow Statements :

Depending on whether a condition is True or False, the decision structure may skip the execution of an entire block of statements or even execute one block of statements instead of another. Python has *if*, *if...else* and *if...elif...else* statements for conditional control flow of execution.

#### 3.3.2.1 The if Decision Control Flow Statement :

In order to write useful programs, we almost need the ability to check conditions and change the behaviour of the program accordingly. The Decision Control Flow Statements give us this ability. The simplest form is the *if* statement.

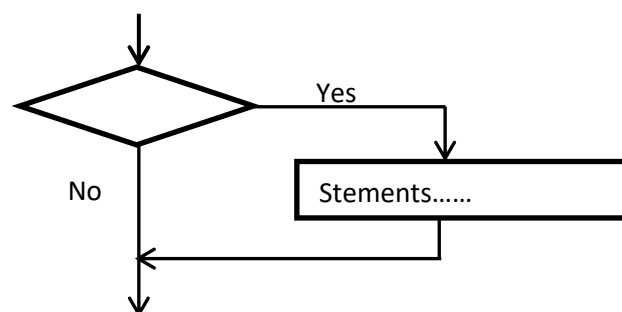
The syntax for *if* statement is:

```

if Boolean_Expression :
 statement (s)

```

The Boolean\_Expression after the *if* statement is called the condition. We end the *if* statement with a colon character (:) and the line(s) after the *if* statement are indented. The *if* statement decides whether to run statement (s) or not depending upon the value of the Boolean expression. If the Boolean expression evaluates to True, then the indented statements in the *if* block will be executed; if the result is False, then none of the statements is executed. e.g.,



**Fig. 3.1: If Logic**

```
if x > 0 :

 print ('x is positive')
```

Here, depending on the value of x, the print statement will be executed, i.e. only if  $x > 0$ .

There is no limit on the number of statements that can appear in the body, but there must be at least one. In Python, the if block statements are determined through indentation, and the first unindented statement marks the end. You don't need to use the == operator explicitly to check if the variable's value evaluates to True, as the variable name can itself be used as a condition.

**Program: Program reads your age and prints a message whether you are eligible to vote or not.**

```
age = int(input("Enter your age : "))
if age >= 18 :
 print ("Congratulations!!! you can vote")
 print ("!!! Thanks !!!")
 print ("Voting is your birth right. Use judicially")

age = int(input("Enter your age : "))
if age >= 18 :
 print ("Congratulations!!! you can vote")
 print ("!!! Thanks !!!")
 print ("Voting is your birth right. Use judicially")

Enter your age : 22
Congratulations!!! you can vote
!!! Thanks !!!
Voting is your birth right. Use judicially
```

Above you can see, the condition or Boolean expression is true, indented execution program would display:

```
Enter your age : 17
Voting is your birth right. Use judicially
```

---

Above, the condition is False, so the indented block would not be executed and only the unindented statement after the if block would be executed.

### 3.3.2.2 The if...else Decision Control Flow Statement

An if statement can also be followed by an else statement (which is optional). An else statement does not have any condition. Statements in the if block are executed if the Boolean\_Expression is True. Use the optional else block to execute statements if the Boolean\_Expression is False. The if...else statements allow for a two-way decision.

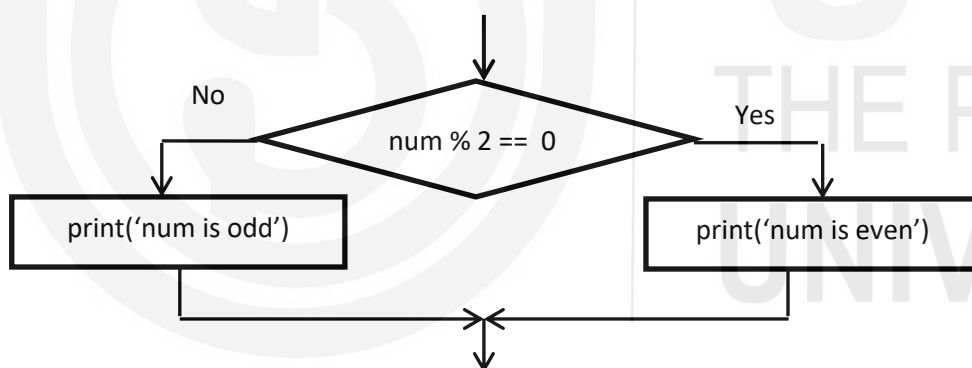
The syntax for *if...else* statement is:

```
if Boolean_Expression :
 statement_blk_1
else
 statement_blk_2
```

If the Boolean\_Expression evaluates to True, then statement\_blk\_1 (single or multiple statements) is executed, otherwise it is evaluated to False then statement\_blk\_2 (single or multiple statements) is executed. Indentation is used to separate the blocks. After the execution of either statement\_blk\_1 or statement\_blk\_2, the control is transferred to the next statement after the if statement. Also, if and else keywords should be aligned at the same column position.

**Program 10.6 : Program to find if a given number is odd or even**

```
num = int(input("Enter a number : "))
if num % 2 == 0 :
 print (num," is even ")
else:
 print (num," is odd ")
```



**Fig. 3.2: if-else Logic**

A number is read and stored in the variable named num. The num is checked using modulus operator to determine whether the num is perfectly divisible by 2 or not. The num entered is 15, which makes the evaluated expression False, so the else statement is executed, and num is odd.

```
num = int(input("Enter a number : "))
if num % 2 == 0:
 print (num," is even number")
else:
 print (num," is odd number")
```

```
Enter a number : 15
15 is odd number
```

### 3.3.2.3 The if...elif...else Decision Control Statement

The *if...elif...else* is also called a multi-way decision control statement. When you need to choose from several possible alternatives, an elif statement is used along with an if statement. The keyword 'elif' is short for 'else if' and is useful to avoid excessive indentation. The else statement must always come last, and will again act as the default action.

The syntax for if...elif...else statement is,

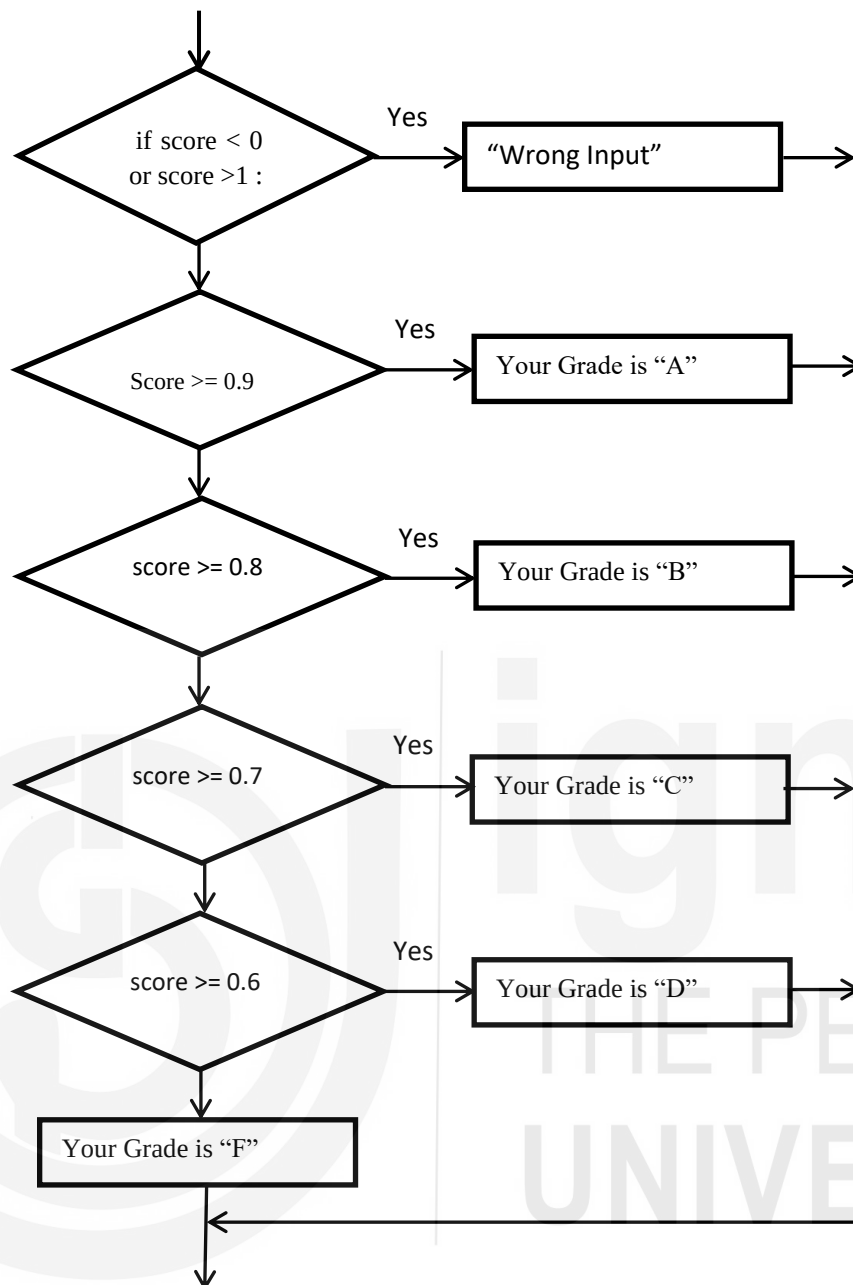
```
if Boolean_Expression_1 :
 statement_blk_1
elif Boolean_Expression_2 :
 statement_blk_2
elif Boolean_Expression_3 :
 statement_blk_3
:
:
:
else :
 statement_blk_last
```

This if...elif...else decision control statement is executed as follows:

- In the case of multiple Boolean expression, only the first logical Boolean expression which evaluates to True will be executed.
- If Boolean\_Expression\_1 is True, then statement\_blk\_1 is executed.
- If Boolean\_Expression\_1 is False and Boolean\_Expression\_2 is True, then statement\_blk\_2 is executed.
- If Boolean\_Expression\_1 and Boolean\_Expression\_2 are False and Boolean\_Expression\_3 is True, then statement\_blk\_3 is executed and so on...
- If none of the Boolean\_Expression is True, then statement\_blk\_last is executed.

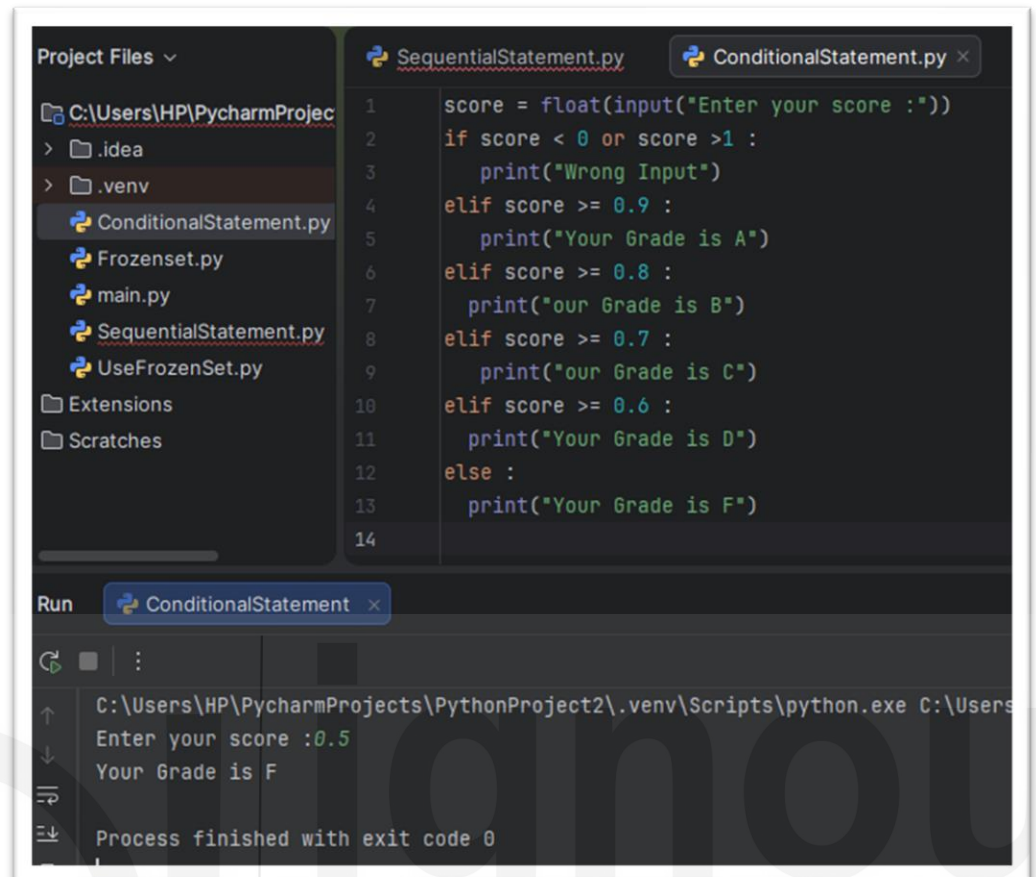
**Program:** Write a program to prompt for a score between 0.0 and 1.0. If the score is out of range, print an error. If the score is between 0.0 and 1.0, print a grade using the following table.

| Score  | Grade |
|--------|-------|
| >= 0.9 | A     |
| >= 0.8 | B     |
| >= 0.7 | C     |
| >= 0.6 | D     |
| < 0.6  | F     |



**Fig. 3.3 : Flowchart of the Program**

```
score = float(input("Enter your score :"))
if score < 0 or score > 1 :
 print("Wrong Input")
elif score >= 0.9 :
 print("Your Grade is A")
elif score >= 0.8 :
 print("our Grade is B")
elif score >= 0.7 :
 print("our Grade is C")
elif score >= 0.6 :
 print("Your Grade is D")
else :
 print("Your Grade is F")
```



```
Project Files
C:\Users\HP\PycharmProjec
> .idea
> .venv
ConditionalStatement.py
FrozenSet.py
main.py
SequentialStatement.py
UseFrozenSet.py
Extensions
Scratches

SequentialStatement.py
1 score = float(input("Enter your score :"))
2 if score < 0 or score >1 :
3 print("Wrong Input")
4 elif score >= 0.9 :
5 print("Your Grade is A")
6 elif score >= 0.8 :
7 print("our Grade is B")
8 elif score >= 0.7 :
9 print("our Grade is C")
10 elif score >= 0.6 :
11 print("Your Grade is D")
12 else :
13 print("Your Grade is F")
14

Run ConditionalStatement
C:\Users\HP\PycharmProjects\PythonProject2\.venv\Scripts\python.exe C:\Users
Enter your score :0.5
Your Grade is F
Process finished with exit code 0
```

#### 3.3.2.4 Nested if Statement

In some situations, you have to place an if statement inside another statement. An if statement that contains another if statement either in its if block or else block is called a **Nested if statement**.

he syntax of the nested if statement is:

```
if Boolean_Expression_1 :
 if Boolean_Expression_2 :
 statement_blk_1
 else :
 statement_blk_2
else :
 statement_blk_3
```

If the Boolean\_Expression\_1 is evaluated to True, then the control shifts to Boolean\_Expression\_2, and if this expression is evaluated to True, then statement\_blk\_1 is executed. If Boolean\_Expression\_2 is evaluated to False, then statement\_blk\_2 is executed. If the Boolean\_Expression\_1 is evaluated to False, then the statement\_blk\_3 is executed.

A three-branch example could be written as :

```
if x == y :
 print('x and y are equal')
```

```

else :
 if x < y :
 print('x is less than y')
 else :
 print('x is greater than y')

```

The outer conditional contains two branches. The first branch contains a simple statement. The second branch contains another if statement has two branches of its own. Those two branches are both simple statements, although they could have been conditional statements as well.

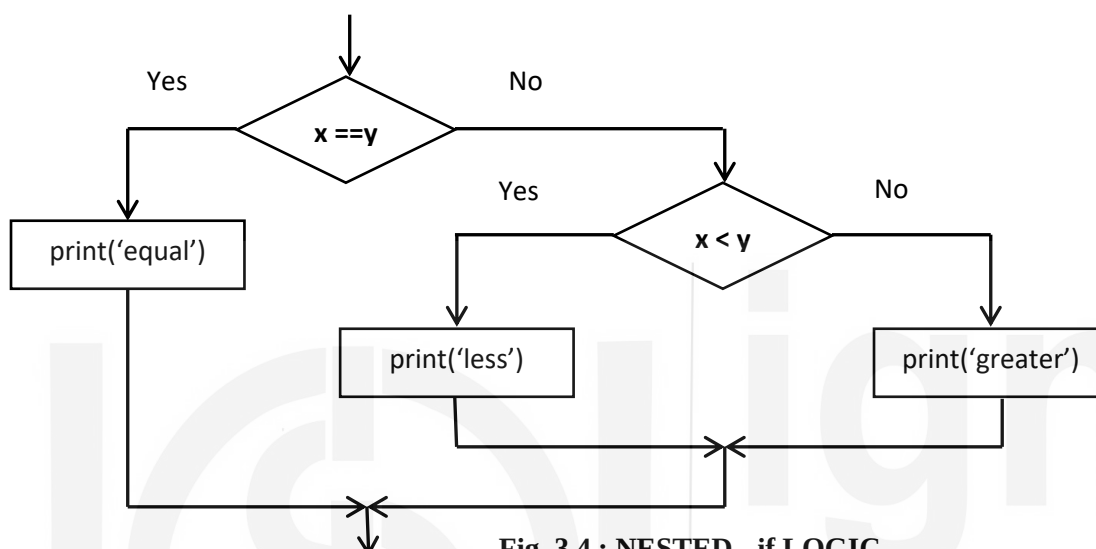


Fig. 3.4 : NESTED - if LOGIC

Program to check if a given year is a leap year

**Note:** All years which are perfectly divisible by 4 are leap years except for century years (years ending with 00), which are leap years only if they are perfectly divisible by 400. For example, years like 2012, 2004, 1968 are leap years, but 1971, 2006 are not leap years. Similarly, 1200, 1600, 2000, 2400 are leap years, but 1700, 1800, 1900 are not.

Screen shot of execution of Program :

```

year = int(input('Enter a year : '))
if year % 4 == 0:
 if year % 100 == 0 :
 if year % 400 == 0 :
 print(f'{year} is a Leap Year')
 else :
 print(f'{year} is not a Leap Year')
 else :
 print(f'{year} is a Leap Year')
else :
 print(f'{year} is not a Leap Year')

```

```

Enter a year : 2014
2014 is not a Leap Year

```

### Check your progress -1

- 1) Write a program to take a number as input and find whether the number is odd or even and positive or negative.

- 2) Write a program to calculate bonus for employees based on basic pay, DA, HRA and grade of the employee. Make necessary assumptions.

### 3.3.3 Loop Control Flow Statements

These control structures allow the execution of a block of statements multiple times until a loop termination condition is met. Loop Control Flow statements are also called Repetition statements or Iteration Statements.

#### 3.3.3.1 The *while* loop

Computers are often used to automate repetitive tasks. Repeating identical or similar tasks without making errors is something that computers do well, and people do poorly. Because iteration is so common, Python provides for loop and while loop constructs for writing iterative programming blocks.

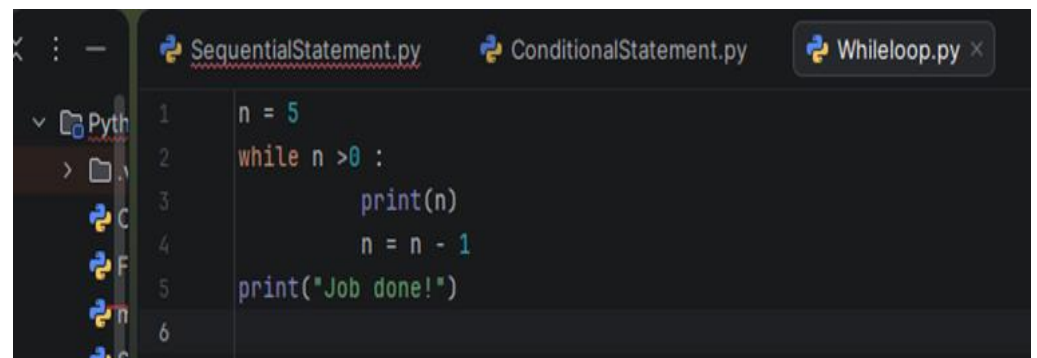
The syntax for while loop is :

```
while Boolean_Expression :
 statement(s)
```

Example: Program that counts down from five and then says "Job done!"

```
n = 5
while n > 0 :
 print(n)
 n = n - 1
print('Job done!')
```

Screenshot of execution of Program

A screenshot of a Python IDE with a dark theme. The top bar shows three open files: 'SequentialStatement.py', 'ConditionalStatement.py', and 'Whileloop.py'. The 'Whileloop.py' file is active, showing a Python script with five lines of code: 1. n = 5, 2. while n > 0 :, 3. print(n), 4. n = n - 1, 5. print("Job done!"). The left sidebar shows a file explorer with a 'Pyth' folder expanded, containing subfolders for 'C', 'F', 'n', and 'S'.

A screenshot of a Python IDE terminal window. The title bar says 'Run' and 'Whileloop'. The terminal shows the command prompt 'C:\Users\HP\PycharmProjects\PythonProject2\.venv\Scripts\python.exe C:\Users\HP\PycharmProjects\PythonProject2\'. The output of the program is: 5, 4, 3, 2, 1, Job done!. At the bottom, it says 'Process finished with exit code 0'.

You can almost read the while statement as if it were English. It means, “While n is greater than 0, display the value of n and then reduce the value of n by 1. When you get to 0, exit the while statement and display the “Job done”.

More formally, the flow of execution for a while statement is :

1. Evaluate the condition or the Boolean\_Expression, yielding True or False.
2. If the Boolean\_Expression is false, exit the while statement and then continue execution at the next statement.
3. If the Boolean\_Expression is true, execute the body and then go back.

This flow is called a loop because the third step loops back to the top. We call each time we execute the loop body an iteration. For the above loop, we would say, “It had five iterations”, which means that the body of the loop was executed five times.

The loop body should change the value of one or more variables so that the condition, or Boolean\_Expression, eventually becomes false and the loop terminates. We call the variable that changes each time the loop executes and controls when the loop finishes the iteration variable. If there is no iteration variable, the loop will **repeat forever**, resulting in an **infinite loop**.

Example: Program to display the Fibonacci sequence up to the nth term, where n is taken from the user as input.

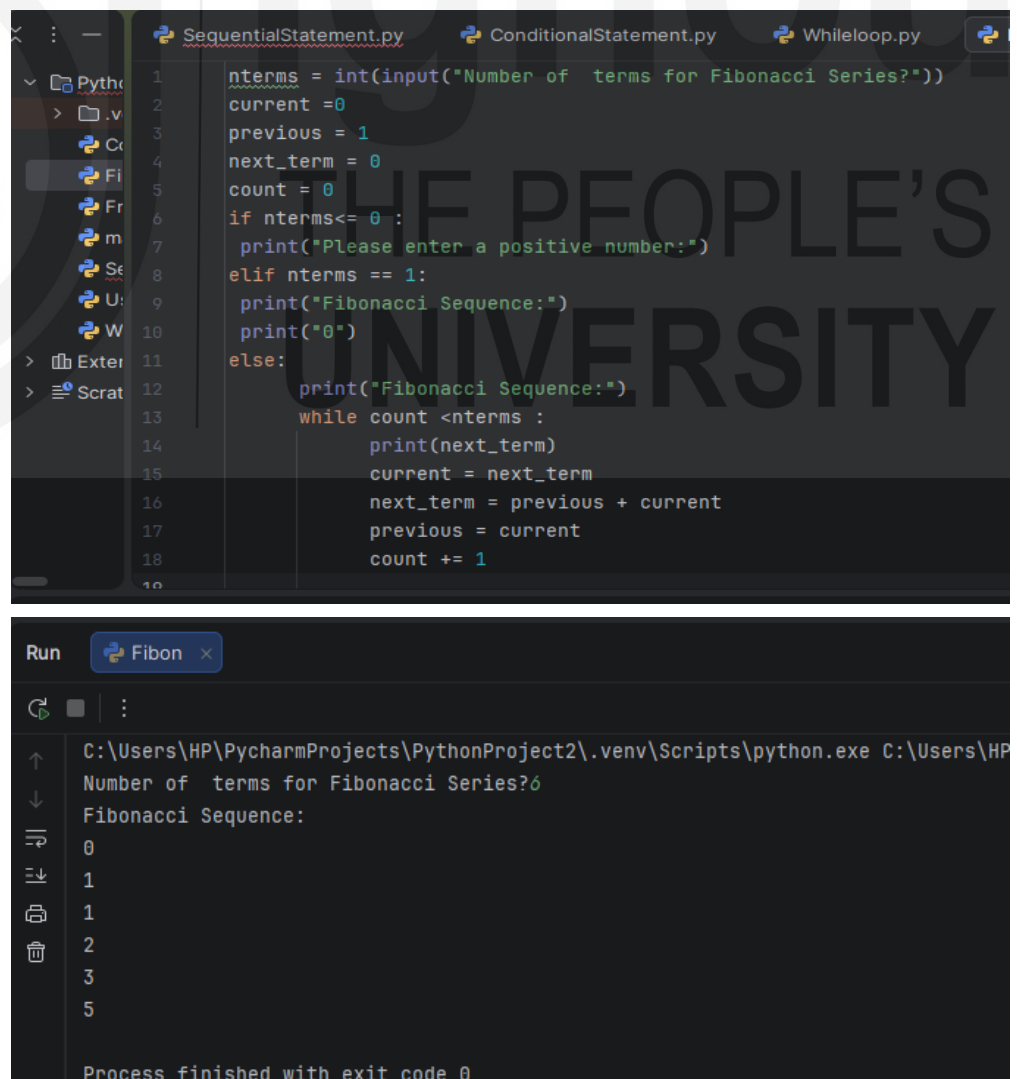
**Note:** In a Fibonacci sequence, the next number is obtained by adding the previous two numbers. The first two numbers of the Fibonacci sequence are 0 and 1. The next number is obtained by adding 1 and 1, which is 1. Again, the next number is obtained by adding 1 and 1, which is 2 and so on. Get a number from the user up to which you want to generate the Fibonacci sequence. Assign values to variables current, previous, next\_term and count. The variable count keeps track of the number of times the while block is executed. The user is required to enter a positive number to generate a single number in the sequence, then print zero. The next\_term is obtained by adding the previous and current variables, and the statements in the while block are repeated until the while block conditional expression becomes False.

#### Program:

```
nterms = int(input('How many terms?'))
current = 0
```

```
previous = 1
next_term = 0
count = 0
if nterms <= 0 :
 print('Please enter a positive number')
elif nterms == 1 :
 print('Fibonacci Sequence')
 print('0')
else :
 print("Fibonacci Sequence")
 while count < nterms :
 print(next_term)
 current = next_term
 next_term = previous + current
 previous = current
 count += 1
```

Screenshot of execution of the above program :



The screenshot displays a code editor with a Python script for generating a Fibonacci sequence. The script prompts the user for the number of terms, checks for valid input, and then prints the sequence using a while loop. Below the editor, the 'Run' console shows the program's execution, including the input '6' and the resulting sequence: 0, 1, 1, 2, 3, 5. The process finished with exit code 0.

```
SequentialStatement.py ConditionalStatement.py Whileloop.py
Python
> .venv
> C:\Users\HP\PycharmProjects\PythonProject2\Scripts\python.exe
1 nterms = int(input("Number of terms for Fibonacci Series?"))
2 current = 0
3 previous = 1
4 next_term = 0
5 count = 0
6 if nterms <= 0 :
7 print("Please enter a positive number:")
8 elif nterms == 1:
9 print("Fibonacci Sequence:")
10 print("0")
11 else:
12 print("Fibonacci Sequence:")
13 while count < nterms :
14 print(next_term)
15 current = next_term
16 next_term = previous + current
17 previous = current
18 count += 1
19
```

Run Fibon x

```
C:\Users\HP\PycharmProjects\PythonProject2\.venv\Scripts\python.exe C:\Users\HP
Number of terms for Fibonacci Series?6
Fibonacci Sequence:
0
1
1
2
3
5
Process finished with exit code 0
```

### 3.3.3.2 The for loop

We call the while statement an indefinite loop because it simply loops until some condition becomes false, whereas the for loop is looping through a known set of items so it runs through as many iterations as there are items in the set. Sometimes we want to loop through a set of things such as a list of words, the lines in a file, or a list of numbers. In other words, we use a *for loop* when we want to repeat a block of code a set number of times. When we have a list of things to loop through, we can construct a definite loop using a for statement.

The syntax for the for loop is :

```
for iteration_variable in sequence :
 statement(s)
```

The *for loop* starts with the *for keyword* and ends with a colon. The first item in the sequence gets assigned to the iteration variable *iteration\_variable*. Here, *iteration\_variable* can be any valid variable name. Then the statement block is executed. This process of assigning items from the sequence to the *iteration\_variable* and then executing the statement continues until all the items in the sequence are completed.

In Python, understanding the for loop is incomplete without knowledge of the *range()* function, a built-in function. It is very useful in demonstrating the *for loop*. The *range()* function generates a sequence of numbers that you can iterate through with a *for loop*. the syntax for *range()* function is,

```
range([start ,] stop [, step])
```

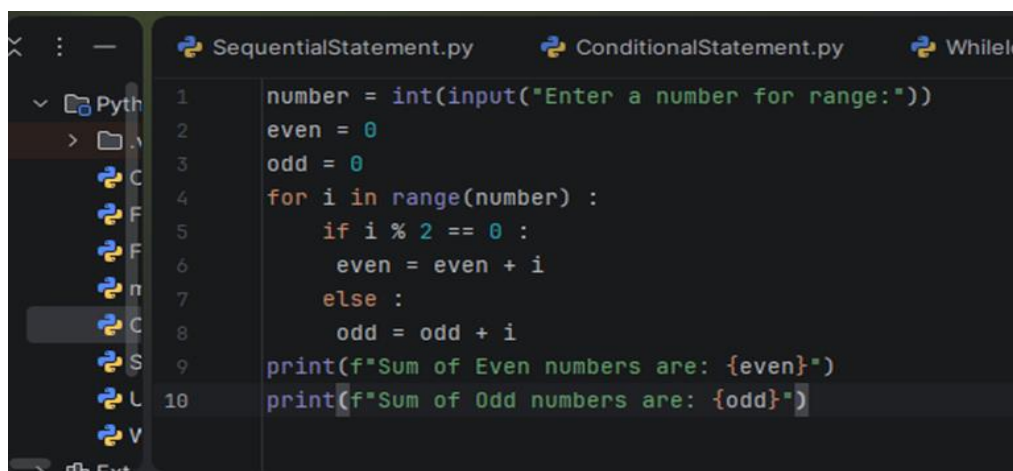
Both start and step arguments are optional and are represented with the help of square brackets, and the range argument value should always be an integer.

start: This value indicates the beginning of the sequence. If the start argument is not specified, the sequence starts at 0 by default.

stop: It generates numbers up to this value but not including the number itself.

step: It indicates the difference between every two consecutive numbers in the sequence. The step value can be negative or positive, but not zero.

Example: Program to find the sum of all odd and even numbers up to a number specified by the user.



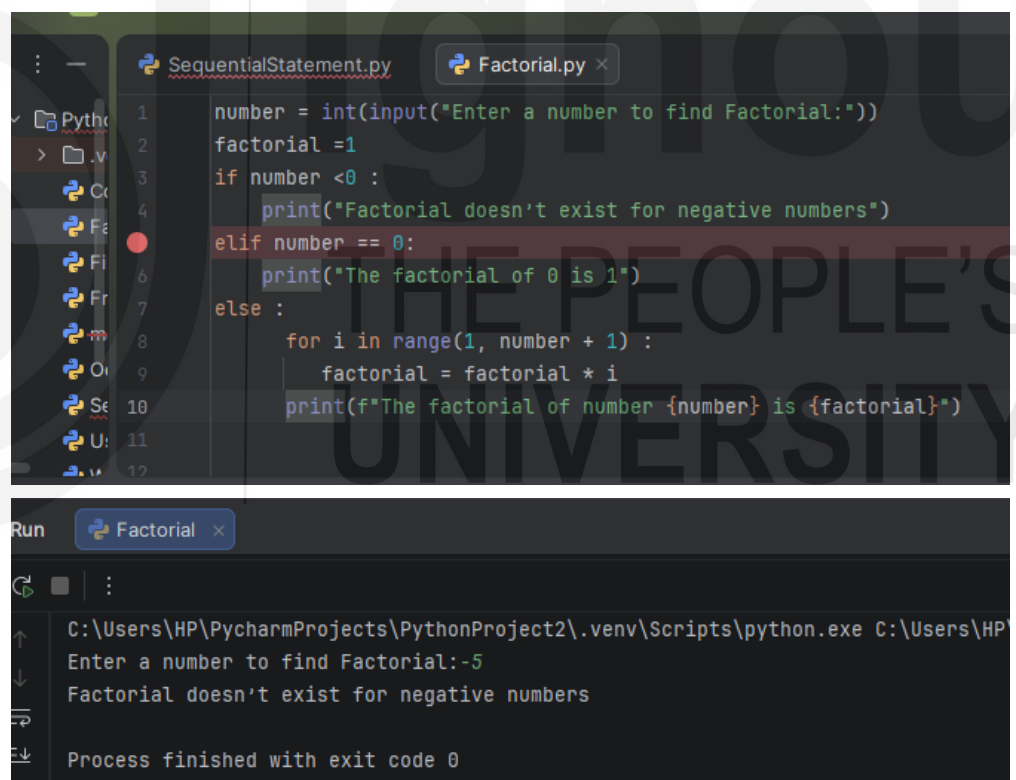
```
1 number = int(input("Enter a number for range:"))
2 even = 0
3 odd = 0
4 for i in range(number) :
5 if i % 2 == 0 :
6 even = even + i
7 else :
8 odd = odd + i
9 print(f"Sum of Even numbers are: {even}")
10 print(f"Sum of Odd numbers are: {odd}")
```



```
Run OddEvenSum x
C:\Users\HP\PycharmProjects\PythonProject2\.venv\Scripts\python.exe
Enter a number for range:25
Sum of Even numbers are: 156
Sum of Odd numbers are: 144
Process finished with exit code 0
```

In the above execution of the program, the value of number is given as 25. A range of numbers is generated using the range() function. Because the range() function only specifies the stop value, it generates numbers from 0 to 24. Then the generated numbers are separated into odd and even using the modulus operator in the for loop. All even numbers are added and assigned to a variable named even, and all odd numbers are added and assigned to a variable named odd; finally, the result is printed. The for loop will be executed/iterated the number of times entered by the user, which is 25 in this case.

Example: Program to find the factorial of a number.



```
SequentialStatement.py Factorial.py x
1 number = int(input("Enter a number to find Factorial:"))
2 factorial = 1
3 if number < 0 :
4 print("Factorial doesn't exist for negative numbers")
5 elif number == 0:
6 print("The factorial of 0 is 1")
7 else :
8 for i in range(1, number + 1) :
9 factorial = factorial * i
10 print(f"The factorial of number {number} is {factorial}")
11
12

Run Factorial x
C:\Users\HP\PycharmProjects\PythonProject2\.venv\Scripts\python.exe C:\Users\HP
Enter a number to find Factorial:-5
Factorial doesn't exist for negative numbers
Process finished with exit code 0
```

In the above screenshot, you can see that for -5 input, the program to find the factorial is executed, and it displays: Factorial doesn't exist for negative numbers

When you execute this program for, let us say, input 6, you will get the following output/result:

```
C:\Users\HP\PycharmProjects\PythonProject2\.venv\Scripts\python.exe
Enter a number to find Factorial:6
The factorial of number 6 is 720

Process finished with exit code 0
```

If you look at the code, the first line reads the number from the user. A value of 1 is assigned to the variable factorial. To find the factorial of a number, it has to be checked for a non-negative integer. If the user-entered number is zero, then the factorial is 1. To generate numbers from 1 to the user-entered number, the range() function is used. Every number is multiplied by the factorial variable and is assigned to the factorial variable inside the for loop. The for loop block repeats for each number from 1 up to the user-entered number. Finally, it prints the factorial value.

### Nested for Loops

A nested loop is a loop inside another loop. In the nested loop, the "inner loop" is executed one time for each iteration of the "outer loop":

Example: Program to print each adjective for every fruit :

```
adj=["red", "big", "tasty"]
fruits=["apple", "banana", "cherry"]
```

```
for x in adj:
 for y in fruits:
 print(x, y)
```

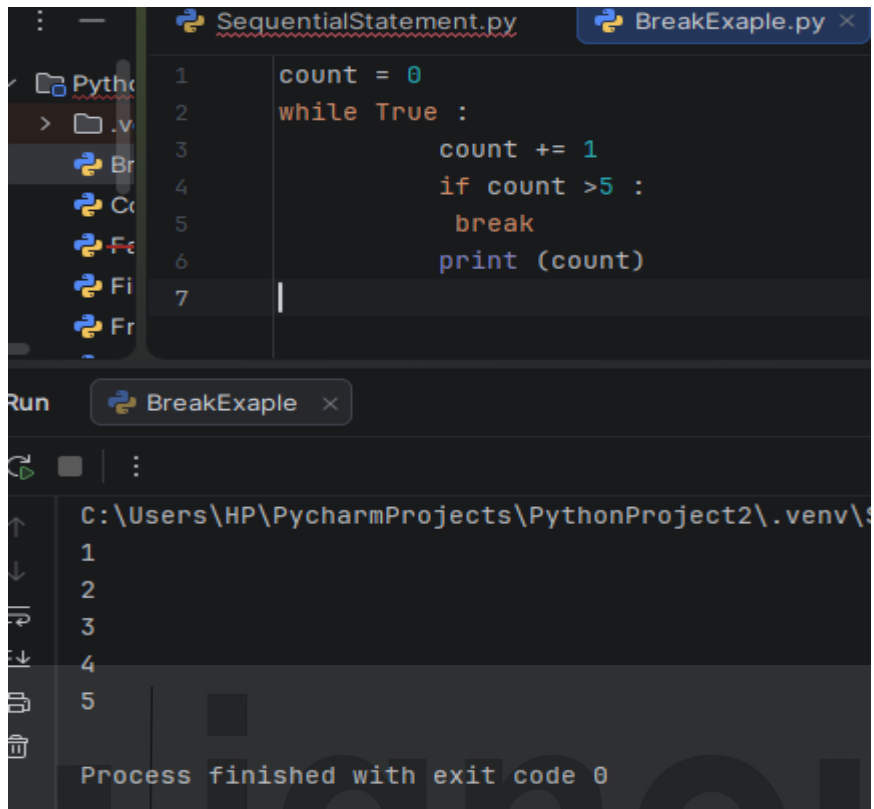
Output:

```
C:\Users\JAY PRAKASH\Desktop\Python\Example>python ex1.py
red apple
red banana
red cherry
big apple
big banana
big cherry
tasty apple
tasty banana
tasty cherry
```

#### 3.3.3.3 The continue and break Statements

The continue and break statements provide greater control over the execution of code in a loop. Whenever the break statement is encountered, the execution control immediately jumps to the first instruction following the loop. To pass control to the next iteration without exiting the loop, the *continue statement* is used. You can use both continue and break statements in while and for loops.

Example: Program that prints integers from zero to 5 using the break statement.



```
1 count = 0
2 while True :
3 count += 1
4 if count > 5 :
5 break
6 print (count)
7
```

Run BreakExaple x

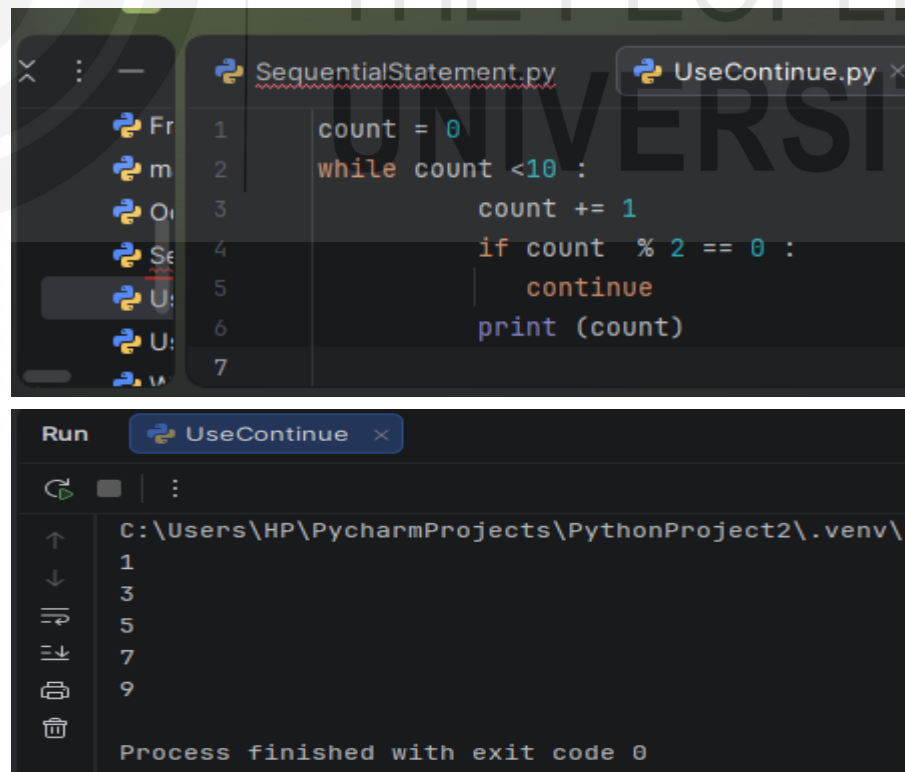
C:\Users\HP\PycharmProjects\PythonProject2\.venv\Scripts\python.exe

1  
2  
3  
4  
5

Process finished with exit code 0

In the above code, in the while loop, as soon as the count reaches 6, the break statement prevents the print statement from executing, and control exits the while loop.

Example: Program that processes only odd integers from 0 to 10 using continue statement.



```
1 count = 0
2 while count < 10 :
3 count += 1
4 if count % 2 == 0 :
5 continue
6 print (count)
7
```

Run UseContinue x

C:\Users\HP\PycharmProjects\PythonProject2\.venv\Scripts\python.exe

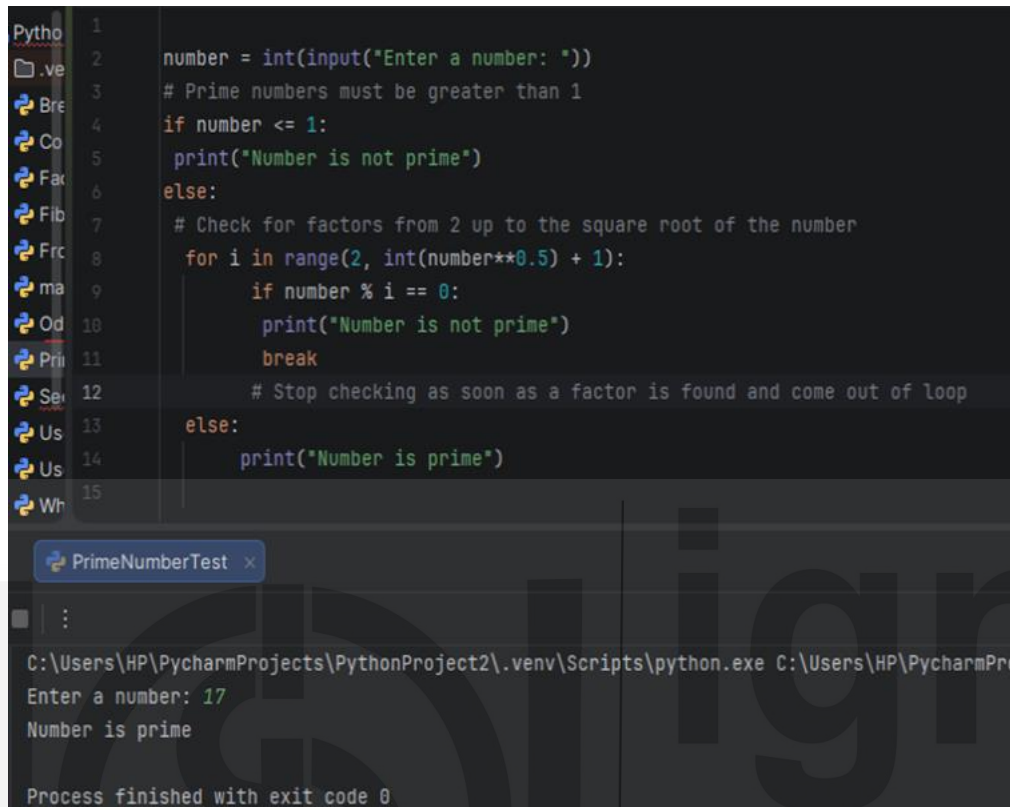
1  
3  
5  
7  
9

Process finished with exit code 0

The continue statement in the while loop will not let the print statement to execute if the count value is even ,and control will go for next iteration of the

loop. Here, the loop will go through all the iterations, and the *continue statement* will affect only the statements in the loop are to be executed or not occurring after the *continue statement*.

Example: Program to check whether a number is prime or not.



```

1 number = int(input("Enter a number: "))
2 # Prime numbers must be greater than 1
3 if number <= 1:
4 print("Number is not prime")
5 else:
6 # Check for factors from 2 up to the square root of the number
7 for i in range(2, int(number**0.5) + 1):
8 if number % i == 0:
9 print("Number is not prime")
10 break
11 # Stop checking as soon as a factor is found and come out of loop
12 else:
13 print("Number is prime")
14
15

```

PrimeNumberTest x

```

C:\Users\HP\PycharmProjects\PythonProject2\.venv\Scripts\python.exe C:\Users\HP\PycharmPro
Enter a number: 17
Number is prime
Process finished with exit code 0

```

## Check your progress-2

Q1). In a Python program, a control structure:

- Defines program-specific data structures
- Directs the order of execution of the statements in the program
- Dictates what happens before the program starts and after it terminates
- None of the above

2). Which of the following is False regarding loops in Python?

- Loops are used to perform certain tasks repeatedly
- A while loop is used when multiple statements are to be executed repeatedly until the given condition becomes False
- A while loop is used when multiple statements are to be executed repeatedly until the given condition becomes True.
- A for loop can be used to iterate through the elements of lists.

3). We can write if/else into one line in Python.

- True
- False

Q4). Given the nested if-else below, what will be the value of x when the code is executed successfully?

```

x = 0
a = 5
b = 5

```

```
if a > 0:
 if b < 0:
 x = x + 5
elif a > 5:
 x = x + 4
else:
 x = x + 2
print(x)
```

- a) 0
- b) 4
- c) 2
- d) 3

Q5). if -3 evaluates to:

- a) True
- b) False

Q6). What is the output of the following nested loop

```
numbers = [10, 20]
items = ["Chair", "Table"]
for x in numbers:
 for y in items:
 print(x, y)
```

- a) 10 Chair  
10 Table  
20 Chair  
20 Table
- b) 10 Chair  
20 Chair  
10 Table  
20 Table

Q7). What is the value of X?:

```
X = 0
```

```
while (X < 100):
```

```
 X+=2
```

```
print(x)
```

- a) 101
- b) 99
- c) None of the above, this is an infinite loop
- d) 100

Q8). What is the value of the var after the for loop completes its execution?

```
var = 10
```

```
for i in range(10):
```

```
 for j in range(2, 10, 1):
```

```
 if var % 2 == 0:
```

```
 continue
```

```
 var += 1
```

```
 var+=1
```

```
else:
```

```
 var+=1
```

print(var)

- a) 20
- b) 21
- c) 10
- d) 30

Q9). Find the output of the following program:

```
a = {i :i * i for i in range(6)}
```

```
print (a)
```

- a) Dictionary comprehension doesn't exist
- b) {0:0, 1:1, 2:4, 3:9, 4:16, 5:25, 6:36}
- c) {0:0, 1:1, 4:4, 9:9, 16:16, 25:25}
- d) {0:0, 1:1, 2:4, 3:9, 4:16, 5:25}

10) Write the benefits of using iterations or loops in Python.

11) What are range functions? Why do we need range functions in a for loop?

12) Write Python code using a for loop that outputs all the multiples of 5 between 108 and 205.

## 3.4 SUMMARY

This unit explains how sequential and control flow statements are used in Python. All the basic structures of Python can be combined with control flow statements, which ultimately form a Python program. This unit explained how to use if, if..else, if..elif, and looping statements like for and while with several programming examples. However, this is not all; there is still a long way to go in the Python programming journey. Python has many more unique features which make it a programmer's language.

## 3.5 SOLUTION/ANSWER TO CHECK YOUR PROGRESS

### Answer to Check your progress-1

1.

```
1 # Program to check whether the given number is positive/negative and even/odd
2 num = (int)(input("Enter the number: "))
3 # Check whether number is positive, negative, or zero
4 if num > 0:
5 sign = "Positive"
6 if num%2 == 0:
7 print ("The number entered is Even and", sign)
8 else:
9 print("The number entered is Odd and", sign)
10 elif num < 0:
11 sign = "Negative"
12 if num%2 == 0:
13 print ("The number entered is Even and", sign)
14 else:
15 print("The number entered is Odd and", sign)
16 else:
17 sign = "Zero (neither positive nor negative)"
18 print(sign)
```

C:\Users\HP\PycharmProjects\PythonProject2\.venv\Scripts\python.exe C:\Users\HP\PycharmProjects\PythonProject2\Scripts\python.exe C:\Users\HP\PycharmProjects\PythonProject2\Scripts\python.exe C:\Users\HP\PycharmProjects\PythonProject2\Scripts\python.exe

Enter the number: 45  
The number entered is Odd and Positive

Enter the number: -67  
The number entered is Odd and Negative

Process finished with exit code 0

2.

*Program for bonus calculation. Assuming different rates of DA, HRA and bonus for different grades*

# Taking input from the user

```
emp_id = input("Enter Employee ID: ")
```

```
name = input("Enter Employee Name: ")
```

```
basic_pay = float(input("Enter Basic Pay: "))
```

```
grade = input("Enter Grade (A, B, or C): ").upper()
```

# Calculating DA and HRA using if-elif-else statements

```
if grade == 'A':
```

```
 da = 0.50 * basic_pay
```

```
 hra = 0.30 * basic_pay
```

```

 bonus_percentage = 0.20
elif grade == 'B':
 hra = 0.20 * basic_pay
 da = 0.40 * basic_pay
 bonus_percentage = 0.15
elif grade == 'C':
 da = 0.30 * basic_pay
 hra = 0.10 * basic_pay
 bonus_percentage = 0.10
else:
 print("Invalid Grade entered!By default Grade is C.")
 grade = 'C'
 da = 0.30 * basic_pay
 hra = 0.10 * basic_pay
 bonus_percentage = 0.10
Calculating Gross Salary and Bonus
gross_salary = basic_pay + da + hra
bonus = gross_salary * bonus_percentage
Displaying the results
print("="*40)
print(f"PAYSLIP FOR EMPLOYEE: {name} (ID: {emp_id})")
print("=" * 40)
print(f"Grade : {grade}")
print(f"Basic Pay : ₹{basic_pay:,.1f}")
print(f"DA : ₹{da:,.1f}")
print(f"HRA : ₹{hra:,.1f}")
print(f"Gross Salary : ₹{gross_salary:,.1f}")
print(f"Bonus : ₹{bonus:,.1f}")
print("=" * 40)

```

Output of the Program:

```

C:\Users\HP\PycharmProjects\PythonProject2\.venv\Scripts\python.exe C:\Users\HP\PycharmProjects
Enter Employee ID: 125
Enter Employee Name: Mohan
Enter Basic Pay: 50000
Enter Grade (A, B, or C): A
=====
PAYSLIP FOR EMPLOYEE: Mohan (ID: 125)
=====
Grade : A
Basic Pay : ₹50,000.0
DA : ₹25,000.0
HRA : ₹15,000.0
Gross Salary : ₹90,000.0
Bonus : ₹13,500.0
C:\Users\HP\PycharmProjects\PythonProject2\.venv\Scripts\python.exe C:\Users\HP\PycharmProjects
=====
Enter Employee ID: 125
Enter Employee Name: Mohan
Enter Basic Pay: 50000
Enter Grade (A, B, or C): A
Process fin:

```

```
C:\Users\DELL\PycharmProjects\PythonProject\.venv\Scripts\python.exe
Enter Employee ID: 1
Enter Employee Name: mp
Enter Basic Pay: 50000
Enter Grade (A, B, or C): A

PAYSALIP FOR EMPLOYEE: mp (ID: 1)

Grade : A
Basic Pay : ₹50,000.00
DA : ₹25,000.00
HRA : ₹15,000.00
Gross Salary : ₹90,000.00
Bonus : ₹18,000.00

Process finished with exit code 0
```

#### Answer to Check your progress-2

1. b
2. b
3. c
4. a
5. a
6. a
7. d
8. b
9. d

10.

Loops make Python programs simpler, more efficient, and more powerful. Iteration and loops (for and while) are fundamental structures in Python that allow you to execute a block of code repeatedly. In Python, iteration allows you to traverse and manipulate data structures of variable lengths (like lists, dictionaries, tuples, and sets) without needing to know their exact size in advance. They help handle variable-sized datasets dynamically. They also help create clear code structure.

11.

The range() function is essential in Python for creating number sequences and controlling loops efficiently, making programs cleaner, faster, and easier to understand.

12.

```
for num in range(108,205,1):
 if(num%5==0):
 print(num)
```

---

## 3.6 REFERENCES/FURTHER READINGS

---

1. Yashavant Kanetkar and Aditya Kanetkar, “Let Us Python”, 4<sup>th</sup> Edition, BPB Publications, New Delhi , 2022
2. Paul Deitel and Harvey Deitel , “Python For Programmers” , 1<sup>st</sup> Edition, Pearson India Education , Chennai, 2023
3. Sheetal Taneja and Naveen Kumar, “Python Programming a Modular Approach” , 1<sup>st</sup> Edition, Pearson India Education , Chennai , 2022
4. S. Shridhar, J. Indumathi and V.M. Hariharan , “Python Programming” , 1<sup>st</sup> Edition, Pearson India Education , Chennai, 2024
5. S. Suma Christal Mary , R. Josphine Leela, N. Senthamilarasi and Y. Chrisbin Jeeva , “Problem Solving and Python Programming”, 1<sup>st</sup> Edition, Ane Books , New Delhi, 2018



---

## UNIT 4 FUNCTIONS AND FILE HANDLING IN PYTHON

---

### Structure

- 4.0 Introduction
- 4.1 Objectives
- 4.2 Function definition and calling
- 4.3 Scope of Variable
- 4.4 Function arguments
- 4.5 Returning from a function
- 4.6 Function objects
- 4.7 Lambda / Anonymous Functions
- 4.8 File Operations
- 4.9 Summary
- 4.10 Solution to Check your Progress
- 4.11 References/Further Readings

---

## 4.0 INTRODUCTION

---

Python lets you organise tasks into more manageable units called functions. Functions make the code modular. Modularity is the process of decomposing a problem into a set of sub-problems to reduce its complexity. Functions also make the code reusable. File handling is another important aspect discussed in this unit. File handling includes creating, deleting, and manipulating files. You can handle files using Python. In this unit, you will learn how to write your own functions. This unit also covers Python file handling, including opening, reading, writing, and closing files, which enables you to manage and manipulate data stored in files.

---

## 4.1 OBJECTIVES

---

After completing this unit, you will be able to:

- Define and call functions in your code,
- Explain the scope of functions,
- Define function objects,
- Create lambda functions, and
- Use file handling operations.

---

## 4.2 FUNCTION DEFINITION AND CALLING

---

A function is a block of code that performs a specific task. As a program grows, its complexity increases. Hence, it becomes important to organise the program into more manageable units. Further, it makes the code reusable.

There are three types of functions in Python-

1. Built-in functions – these are the functions which are already defined in the language. Example `print()`, `max()`, `input()`, etc.
2. User-defined functions- these are the functions that can be created or defined by the users according to their needs.
3. Anonymous functions

#### 4.2.1 Creating user-defined functions

You can define a function using the *def statement* and give it a suitable name by following identifier rules. The process of defining a function is called *function definition*.

Syntax of function definition

```
def function_name (list_of_parameters) :
 Statement (s)
```

Where,

`function_name` is the name of the function. A set of parentheses must follow a function name. Any name can be given to a function following the rules of identifiers.

The `list_of_parameters` is an optional field used to pass values or inputs to a function. It can be none or a comma-separated list of variables.

`Statement(s)` is a set of statements or commands within the function. Each time the function is called, all statements execute. Together, these statements form a function's body.

Even if a function is defined, it can never be executed until it is called. Therefore, to use a function, you must call it using a *function call statement*. A function can be called by its name with a set of parentheses and an optional list of arguments.

Syntax of function calling:

```
function_name(list_of_arguments)
```

Example: A function to display HELLO WORLD

```
def first_function(): # function definition
 print("HELLO WORLD...!!!")

first_function() # function calling

===== RESTART: C:/Users/test/Desktop/python_programs/abc.py =====
HELLO WORLD...!!!
>>>
```

A function needs to be defined only once, but you can call it any number of times using calling statements. They can be called not only in the same program, but also in different programs. This will be further discussed in the next unit.

Another example shown below is a program having a function to calculate the factorial of a number.

Example: A function to display the factorial of a number.

```
def factorial(): # function definition
 fact=1
 n=int(input("Enter a number :"))
 for i in range(1,n+1):
 fact=fact*i
 print("Factorial of ",n," is ",fact)

factorial() # function calling
```

```
===== RESTART: C:/Users/test/Desktop/python_programs/factorial.py =====
Enter a number :4
Factorial of 4 is 24
>>>
```

## 4.3 SCOPE OF VARIABLE

The part of a code in which a variable can be accessed is called *the scope of a variable*. A variable's scope can be global or local, depending on its availability region.

Global variables are variables created outside any function. Their scope is global, hence can be used anywhere. You can also create global variables within a function using the keyword *global*.

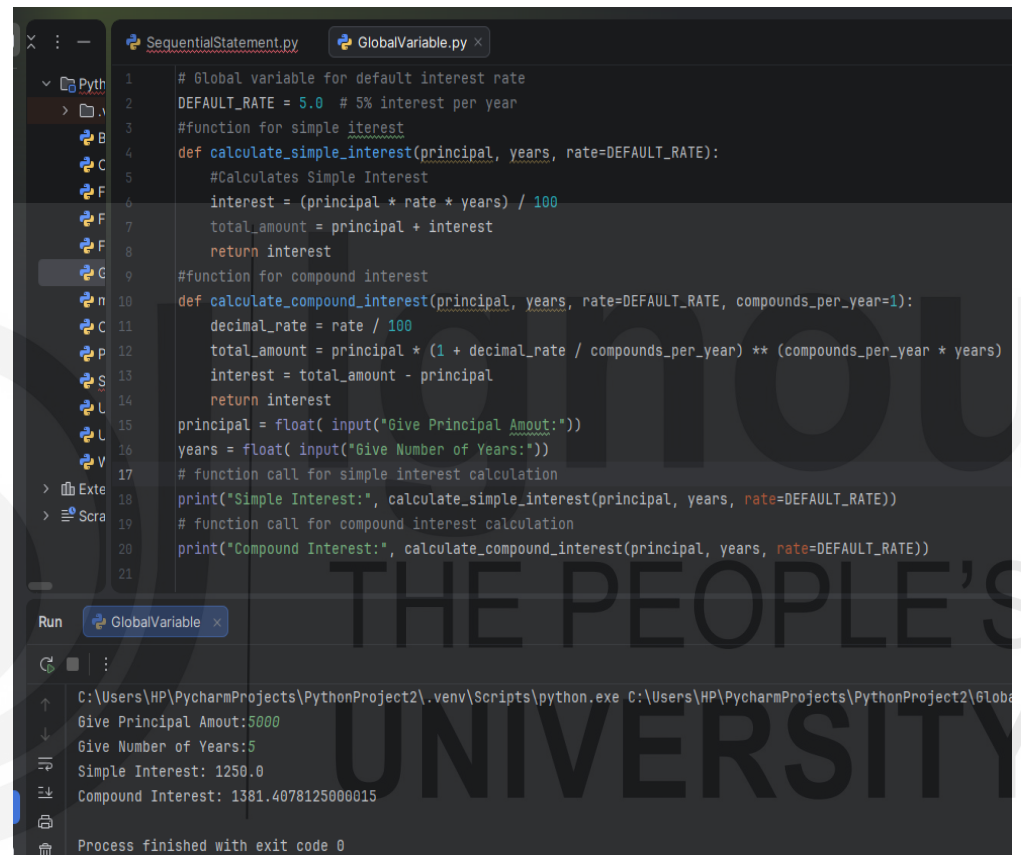
**Local variables** are variables created within a function's body or function's scope. You cannot access them outside the function. Their scope is limited to the function in which they are created.

Example: Use of local and global variables in interest calculation

```
Global variable for default interest rate
DEFAULT_RATE = 5.0 # 5% interest per year
#function for simple interest
def calculate_simple_interest(principal, years,
rate=DEFAULT_RATE):
 #Calculates Simple Interest
 interest = (principal * rate * years) / 100
 total_amount = principal + interest
 return interest
#function for compound interest
def calculate_compound_interest(principal, years,
rate=DEFAULT_RATE, compounds_per_year=1):
 decimal_rate = rate / 100
 total_amount = principal * (1 + decimal_rate / compounds_per_year)
 ** (compounds_per_year * years)
 interest = total_amount - principal
 return interest
```

```
principal = float(input("Give Principal Amount:"))
years = float(input("Give Number of Years:"))
function call for simple interest calculation
print("Simple Interest:", calculate_simple_interest(principal, years,
rate=DEFAULT_RATE))
function call for compound interest calculation
print("Compound Interest:", calculate_compound_interest(principal,
years, rate=DEFAULT_RATE))
```

Execution of Screenshot of above Program:



The screenshot shows a Python IDE with two files: `SequentialStatement.py` and `GlobalVariable.py`. The code in `GlobalVariable.py` defines a global variable `DEFAULT_RATE = 5.0` and two functions: `calculate_simple_interest` and `calculate_compound_interest`. It then calls these functions with user input. The Run window shows the output: `Give Principal Amount:5000`, `Give Number of Years:5`, `Simple Interest: 1250.0`, and `Compound Interest: 1381.4078125000015`. The process finished with exit code 0.

In the above example, `DEFAULT_RATE` is global variable; hence, it can be used inside and outside any function. The variable `interest` is created inside the functions `calculate_simple_interest` and `calculate_compound_interest()`; it is local to them, and its scope is limited to these functions, so it cannot be used outside.

You can define global variables inside functions using *the global keyword*, as shown in the next example.

Example: Program to demonstrate creation of a global variable inside a function.

```
G_Var is Global Variable
G_Var =45
def Fun_1():
```

```

L_Var =5
global G_Var
print("Global Variable:",G_Var)
G_Var = G_Var+10
print("Local Variable inside Function 1:", L_Var)
def Fun_2():
 # Here G_Var is a local variable and it will override the Global
 Variable
 G_Var= 15
 print("Local Vasriable inside Function 2 :", G_Var)
 G_Var +10
Fun_1()
Fun_2()
print("Global Variable:", G_Var)

```



The screenshot shows a Python IDE with a file named 'Global Local Test'. The code in the editor is as follows:

```

1 # G_Var is Global Variable
2 G_Var =45
3 def Fun_1():
4 L_Var =5
5 global G_Var
6 print("Global Variable:",G_Var)
7 G_Var = G_Var+10
8 print("Local Variable inside Function 1:", L_Var)
9 def Fun_2():
10 # Here G_Var is a local variable and it will override the Global Variable
11 G_Var= 15
12 print("Local Vasriable inside Function 2 :", G_Var)
13 G_Var +10
14 Fun_1()
15 Fun_2()
16 print("Global Variable:", G_Var)

```

The output window shows the following results:

```

Global Variable: 45
Local Variable inside Function 1: 5
Local Vasriable inside Function 2 : 15
Global Variable: 55
Process finished with exit code 0

```

If we create a variable inside a function having the same name as that of a global variable, then a separate variable of the same name gets created. In this case, the function accesses the local variable and hence cannot refer to the global variable, as shown in the example. In Fun\_1(), the global keyword is used. In Python, we use the global keyword to modify a variable outside the current scope (usually inside a function) without creating a new local variable. In the above example *global G\_Var* statement is for this purpose only.

Another example is given below where the global variable x is being modified inside the test() function.

```
x=1 # global variable
def test():
 global x # to access global variable for modification
 print("inside function before modification x=",x)
 x=x+2
 print("inside function after modification x=",x)

test()
print("outside function x=",x)
```

===== RESTART: C:/Users/test/Desktop/python\_programs/test.py =====

```
inside function before modification x= 1
inside function after modification x= 3
outside function x= 3
```

Example: Program showing use of Local and global variables of same name.

```
x=1 # global variable
def test():
 x=2 # local variable
 print("inside function x=",x)

test()
print("outside function x=",x)
```

===== RESTART: C:/Users/test/Desktop/python\_programs/test.py =====

```
inside function x= 2
outside function x= 1
>>>
```

In Python, a function can access a global variable directly, but it cannot modify it unless you indicate that the variable is global. Then how can we modify global variables inside a function? The answer is the *global* keyword. It can be used not only to define/create global variables inside functions, but also to modify them. A very important point to note is that:

- i. You can use the global keyword to create a brand-new global variable from inside a function, even if that variable did not exist at the module level before.
- ii. By default, when you assign a value to a variable inside a function, Python creates a **local variable**. Therefore, if a function is not informed about a global variable properly using the global keyword, it will override the global variable with its local one.

The following program shows that you cannot modify a global variable inside a function. When you run this program, you will get the following error:

```

1 # x is Global Variable
2 x = 5
3 def Fun_1():
4 x = x + 2
5 print("Global Variable G_Var1:", x)
6 Fun_1()
7 print("Value of Global Variable G_Var1 at this stage:", x)
8
Run Global Access in Functionm x

C:\Users\HP\PycharmProjects\PythonProject2\.venv\Scripts\python.exe "C:\Users\HP\PycharmProjects\Python
Traceback (most recent call last):
 File "C:\Users\HP\PycharmProjects\PythonProject2\Global Access in Functionm.py", line 6, in <module>
 Fun_1()
 File "C:\Users\HP\PycharmProjects\PythonProject2\Global Access in Functionm.py", line 4, in Fun_1
 x = x + 2
 ^
UnboundLocalError: cannot access local variable 'x' where it is not associated with a value

Process finished with exit code 1

```

Let us see another example where a global variable is created inside a function named Fun\_1().

```

1 # G_Var1 is Global Variable
2 G_Var1 = 45
3 Sum = 0
4 def Fun_1():
5 L_Var = 5
6 global G_Var1
7 print("Global Variable G_Var1:", G_Var1)
8 G_Var1 = G_Var1 + 10
9 global G_Var2 # G_Var2 is a global variable
10 G_Var2 = 10
11 G_Var2 = G_Var2 + 10
12 print("Local Variable inside Function 1:", L_Var)
13 def Fun_2():
14 # Here G_Var is a local variable and it will override the Global Variable
15 G_Var1 = 15
16 print("Local Variable inside Function 2 named G_var1:", G_Var1)
17 Fun_1()
18 Fun_2()
19 print("Value of Global Variable G_Var1 at this stage:", G_Var1)
20 Sum = G_Var1 + G_Var2
21 print("Sum of Global Variables G_Var1 and G_Var2:", Sum)
22
Run Global Local Test x

C:\Users\HP\PycharmProjects\PythonProject2\.venv\Scripts\python.exe "C:\Users\HP\PycharmProjects
Global Variable G_Var1: 45
Local Variable inside Function 1: 5
Local Variable inside Function 2 named G_var1: 15
Value of Global Variable G_Var1 at this stage: 55
Sum of Global Variables G_Var1 and G_Var2: 75

Process finished with exit code 0

```

## Check your Progress – 1

1. What are the benefits of using functions? Also differentiate between function definition and function calling.

2. What is Scope of a variable? Explain local and global variables with example.

3. Write a function named *patter* to display the pattern given below:

```
1
1 2
1 2 3
1 2 3 4
1 2 3 4 5
```

---

## 4.4 FUNCTION ARGUMENTS

---

Arguments are a way of passing values or input to a function. Arguments are passed to a function during function calls. Their values replace the function's parameters in the function definition.

Python provides various mechanisms for passing arguments to a function.

1. Required arguments
2. Positional arguments
3. Default arguments
4. Keyword arguments
5. Arbitrary arguments

### 1. Required Arguments

A required argument is any parameter without a default value. If you do not provide it when calling the function, Python raises a `TypeError`.

Example Program:

```
def Child(name, sex, age=18): # 'name' and 'sex' is required, 'age' is
 optional (has default 18)
 print(f"Hello {name}, you are a:, {sex}, you are {age}, years old.")
 Child("Rani", "Female") # Works! 'name' was supplied.
 Child() # TypeError: missing 1 required positional argument: 'name'
```

If you run the above program, you will get an error that :

TypeError: Child() missing 2 required positional arguments: 'name' and 'sex'  
Because these two parameters are required parameters in the Child ( ) function.

```

1 def Child(name, sex, age=18): # 'name' and 'sex' is required, 'age' is optional (has default 18)
2 print(f"Hello {name}, you are a: {sex}, you are {age}, years old.")
3 Child(name="Rani", sex="Female") # Works! 'name' was supplied.
4 Child() # TypeError: missing 1 required positional argument: 'name'
5
Run FunctionArguments x
C:\Users\HP\PycharmProjects\PythonProject2\.venv\Scripts\python.exe C:\Users\HP\PycharmProjects\PythonProject2\
Hello Rani, you are a: Female, you are 18, years old.
Traceback (most recent call last):
 File "C:\Users\HP\PycharmProjects\PythonProject2\FunctionArguments.py", line 4, in <module>
 Child() # TypeError: missing 1 required positional argument: 'name'
 ^^^^^^^
TypeError: Child() missing 2 required positional arguments: 'name' and 'sex'

Process finished with exit code 1

```

Now let us run the above program by modifying it as:

```

def Child(name, sex, age=18): # 'name' and 'sex' is required; 'age' is
optional (has default 18)
 print(f"Hello {name}, you are, {sex}, you are {age}, years old.")
Child("Rani", "Female") # Works! 'name' was supplied.
Child("Ravi", "Male", 23) # TypeError: missing 1 required positional
argument: 'name'

```

Then you will get output:

```

1 def Child(name, sex, age=18): # 'name' and 'sex' is required, 'age' is optional (has default 18)
2 print(f"Hello {name}, you are, {sex}, you are {age}, years old.")
3 Child(name="Rani", sex="Female") # Works! 'name' was supplied.
4 Child(name="Ravi", sex="Male", age=23) # TypeError: missing 1 required positional argument: 'name'
5
Run FunctionArguments x
C:\Users\HP\PycharmProjects\PythonProject2\.venv\Scripts\python.exe C:\Users\HP\PycharmProjects\PythonProject2\Fun
Hello Rani, you are, Female, you are 18, years old.
Hello Ravi, you are, Male, you are 23, years old.
Process finished with exit code 0

```

In required arguments, the number of arguments should match the number of function parameters and should be given in the same order. If the number of arguments is not the same, an error will occur; also, if the correct order is not followed, the output can be incorrect.

## 2. Positional Arguments

A positional argument is assigned based on its **order/position** in the function call.

Example Program: Passing arguments to a function.

```
def calculator(a,b):
 print("Addition :",a+b)
 print("Subtraction :",a-b)
 print("Multiplication:",a*b)
 print("Division :",a/b)

calculator(5,2)
```

```
===== RESTART: C:/Users/test/Desktop/python_programs/arguments.py =====
Addition : 7
Subtraction : 3
Multiplication: 10
Division : 2.5
>>>
```

## 3. Default arguments

Default values of variables can be given in the parameters itself. So, if an argument is not given (passed) at the time of calling, it is replaced by the default value. Default values can be given for any number of arguments. Ensure that default arguments follow non-default arguments.

Example program to demonstrate use of default arguments:

```
def simple_interest(p,r=2,t=5):
 interest=(p*r*t)/100
 print(" Simple Interest is:",interest)

simple_interest(1000) # p=1000, r=2 (default) ,t=5(default)
simple_interest(1000,5) # p=1000, r=5 ,t=5(default)
simple_interest(1000,5,10) # p=1000, r=2 ,t=5
```

```
===== RESTART: C:\Users\test\Desktop\python_programs\arguments.py =====
Simple Interest is: 100.0
Simple Interest is: 250.0
Simple Interest is: 500.0
>>>
```

## 4. Keyword Arguments

As shown above, arguments must be given in the order in which parameters are defined; otherwise, the result will be affected. Keyword arguments are one way by which we can give arguments in any order. You specify the parameter name with each argument during the function call. In this case, position does not matter, but name does matter; hence, they are also called named arguments.

Example to demonstrate use of keyword arguments

```
def compound_interest(p,r,t):
 amount = p*pow(1+ (r/100),t)
 interest = amount-p
 print("Compound Interest is:",interest)

compound_interest(r=13,p=100000,t=5) # passing keyword arguments
compound_interest(100000,13,5) # passing positional arguments

===== RESTART: C:/Users/test/Desktop/python_programs/compound_interest.py =====
Compound Interest is: 84243.51792999991
Compound Interest is: 84243.51792999991
>>>
```

## 5. Arbitrary Arguments

Sometimes you need to pass a different number of arguments to the same function, or you don't know at function definition time how many arguments will be used when calling the function. Python provides the feature of arbitrary arguments to address this issue. Arbitrary arguments can be declared using the \* symbol.

Example to demonstrate use of arbitrary arguments

```
def sum(*a): # declaration of arbitrary argument
 sum=0
 for i in a:
 sum=sum+i
 print("Sum of Series",a," is:",sum)

sum(1,2,5,7,8,10,2) # passing arbitrary arguments
sum(10,20,25)

===== RESTART: C:/Users/test/Desktop/python_programs/Arbitrary.py =====
Sum of Series (1, 2, 5, 7, 8, 10, 2) is: 35
Sum of Series (10, 20, 25) is: 55
>>>
```

## 4.5 RETURNING FROM A FUNCTION

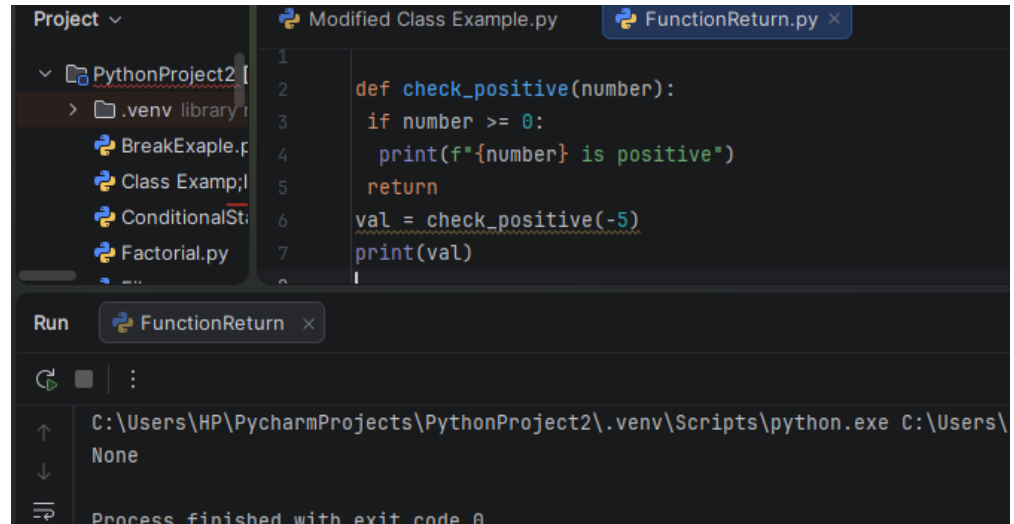
As we can pass values as input to function parameters, we can also return values from a function. You can do this using a **return statement**. By default, when we do not have any return statement in a function, the value **None** is returned from the function.

Syntax of returning from a function

```
def function_name (arg1,arg2....) :

 return value
```

Function in below example program return None.



```
1
2 def check_positive(number):
3 if number >= 0:
4 print(f"{number} is positive")
5 return
6 val = check_positive(-5)
7 print(val)
```

Run FunctionReturn x

C:\Users\HP\PycharmProjects\PythonProject2\.venv\Scripts\python.exe C:\Users\None

Process finished with exit code 0

In the example given below, the function example ( ) returns the average value.

```
def example(a,b,c): # function with return statement
 avg=(a+b+c)/3
 return avg

result=example(2,3,4) # returned value passed to result variable
print("AVG OF NUMBERS:",result) # display return value by print statement

===== RESTART: C:/Users/test/Desktop/python_programs/return_example.py =====
AVG OF NUMBERS: 3.0
>>>
```

Although a function can have only one return statement, it does not mean that we can return only one value. It means the return statement can return only one object, and an object in Python can contain a single variable or multiple values (like a list, tuple, etc.). Hence, we can return multiple values with a single return statement. See the example below, where a function returns more than one value (one is sum and the other is avg) using a single return statement.

```
def example(a,b,c):
 sum=a+b+c
 avg=sum/3
 return sum,avg # function returning two values with single statement

result1,result2=example(2,3,4)
print("SUM OF NUMBERS:",result1)
print("AVG OF NUMBERS:",result2)

===== RESTART: C:/Users/test/Desktop/python_programs/return_example.py =====
SUM OF NUMBERS: 9
AVG OF NUMBERS: 3.0
>>>
```

Benefits of using a return statement :

1. Values returned by the function can be used again in the rest of the program.

2. They can also be passed as arguments to other functions, providing better flow of control.
3. They can also be used to break out of a function in the middle.

When the return statement is executed( occurs ) in the middle of statements in a function, then the statements following the return statement will never be executed. In the following example program, inside a function, there is a loop ranging from 1 to 9, but it will only be executed 4 times, since the function will return when the value of i reaches 5; hence, the remaining iterations will be skipped.

Program to demonstrate that a function returning in between the loop:

```
def example():
 for i in range(1,10):
 if(i==5):
 return # function returns when i=5, no further execution
 else:
 print(i)

example()

===== RESTART: C:/Users/test/Desktop/python_programs/return_example.py =====
1
2
3
4
>>>
```

## 4.6 FUNCTION OBJECTS

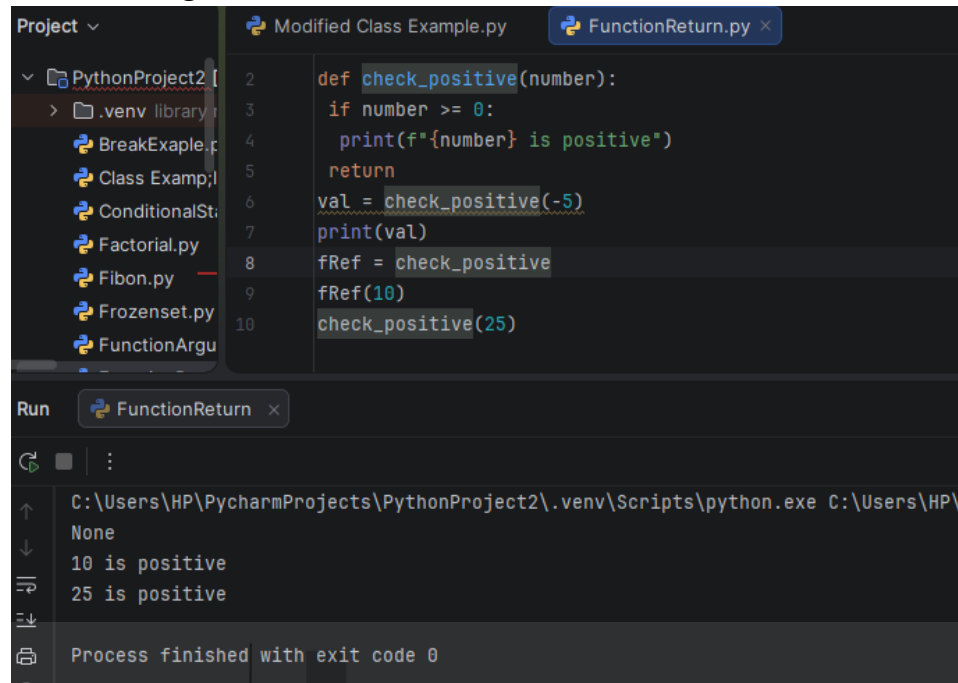
In Python, data is represented as objects. In fact, every piece of data—from numbers and strings to functions, classes, and modules – is an object in memory. Like Lists, Strings Sets etc, functions are also treated as objects. Functions in Python are first-class objects. First-class objects mean they are treated just like any other piece of data (like an integer, string, or list). Because functions are objects, they offer various features additional to the existing ones. These are:

1. Functions can be assigned to a variable
2. Functions can be used as elements of data structures
3. Functions can be sent as arguments to another function
4. Functions can be nested

### 1. Functions assigned to a variable

As functions are objects, they can be assigned to a variable (object). So you can call the function using the variable and parentheses. Assigning a function to a variable creates a reference to the same function or a function with two names. If you delete one function or variable, you can still reference the function by another name. The following example shows a function assigned to a variable.

Function assigned to variable:



The screenshot shows the PyCharm IDE with a project named 'PythonProject2'. The file explorer on the left lists several files, including 'FunctionReturn.py'. The main editor window displays the code in 'FunctionReturn.py':

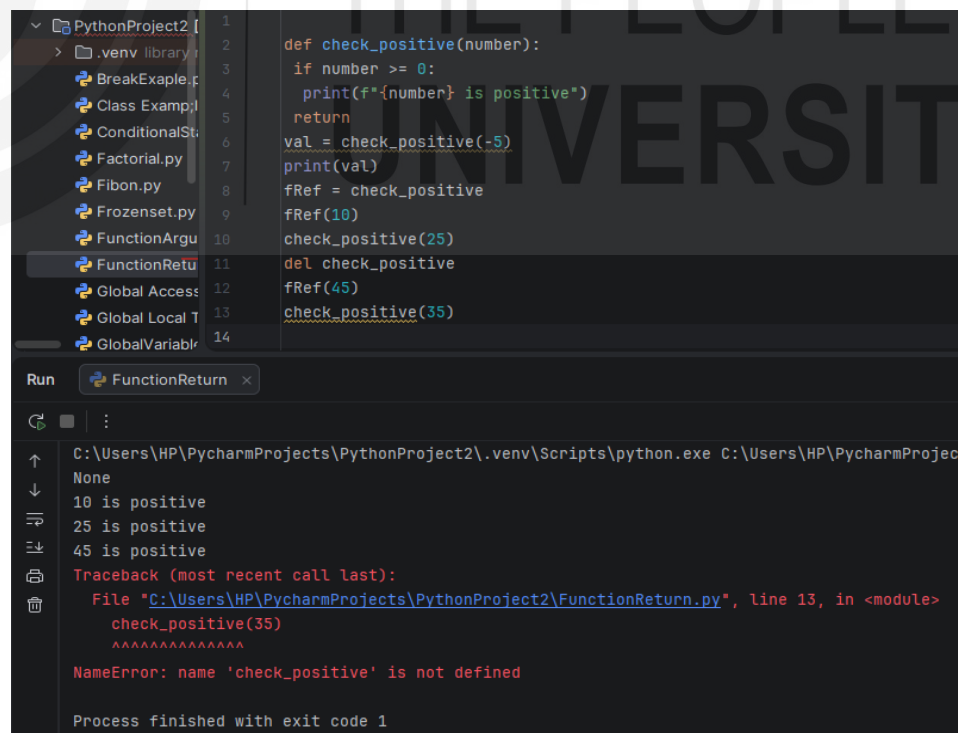
```
1 def check_positive(number):
2 if number >= 0:
3 print(f"{number} is positive")
4 return
5
6 val = check_positive(-5)
7 print(val)
8 fRef = check_positive
9 fRef(10)
10 check_positive(25)
```

The Run window at the bottom shows the execution output:

```
C:\Users\HP\PycharmProjects\PythonProject2\.venv\Scripts\python.exe C:\Users\HP\PycharmProjects\PythonProject2\FunctionReturn.py
None
10 is positive
25 is positive
Process finished with exit code 0
```

In the following code, you can clearly see that after deleting a function, you cannot access it by the same name. However, you can still call it using another name, as shown in the example program.

Deleting a reference to a function: The function call `check_positive(35)` can execute and generate an error because the reference to this function is deleted before it is called.



The screenshot shows the PyCharm IDE with the same project. The file explorer now includes 'FunctionReturn.py'. The main editor window displays the code in 'FunctionReturn.py':

```
1 def check_positive(number):
2 if number >= 0:
3 print(f"{number} is positive")
4 return
5
6 val = check_positive(-5)
7 print(val)
8 fRef = check_positive
9 fRef(10)
10 check_positive(25)
11 del check_positive
12 fRef(45)
13 check_positive(35)
14
```

The Run window at the bottom shows the execution output:

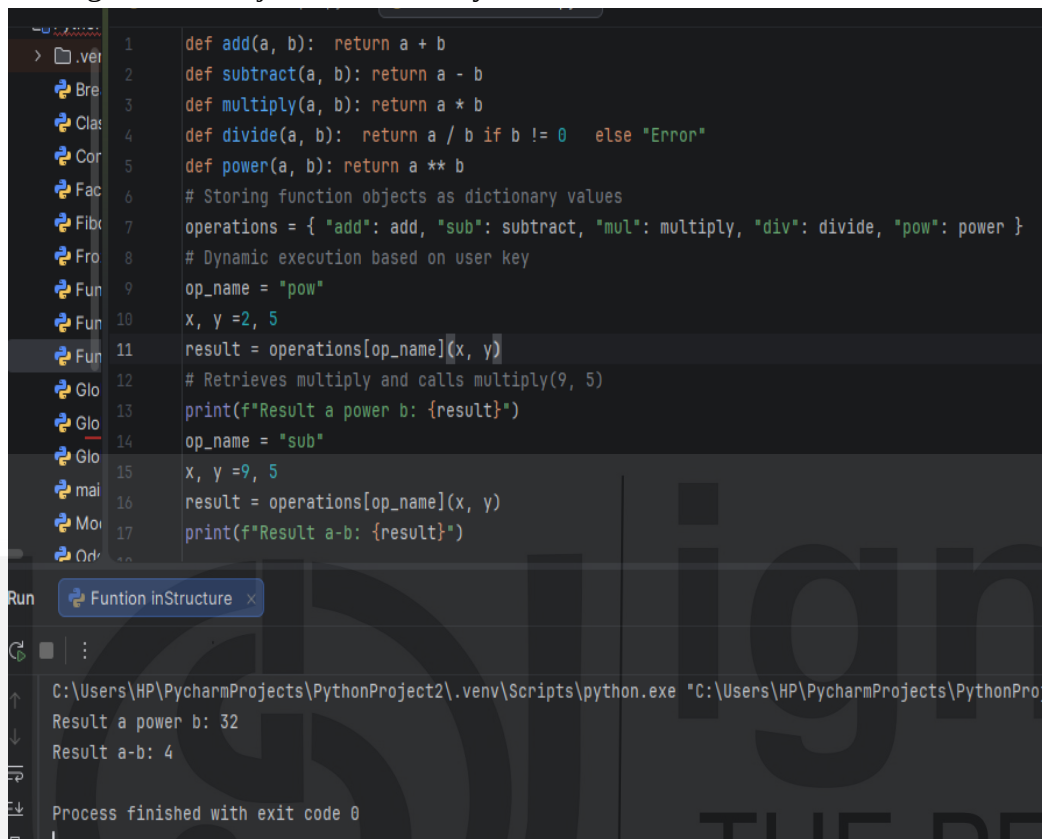
```
C:\Users\HP\PycharmProjects\PythonProject2\.venv\Scripts\python.exe C:\Users\HP\PycharmProjects\PythonProject2\FunctionReturn.py
None
10 is positive
25 is positive
45 is positive
Traceback (most recent call last):
 File "C:\Users\HP\PycharmProjects\PythonProject2\FunctionReturn.py", line 13, in <module>
 check_positive(35)
 ^^^^^^^^^^^^^^^^^
NameError: name 'check_positive' is not defined
Process finished with exit code 1
```

## 2. Functions as elements of a data structure

Functions can be passed as elements in any data structure. This is useful when we want to apply different functions to the same input. Storing functions as

elements inside data structures (lists, dictionaries, tuples, sets, etc.) is a powerful technique in Python. Below is an example. We use various built-in functions to demonstrate this.

Storing function objects as dictionary values:



```
1 def add(a, b): return a + b
2 def subtract(a, b): return a - b
3 def multiply(a, b): return a * b
4 def divide(a, b): return a / b if b != 0 else "Error"
5 def power(a, b): return a ** b
6 # Storing function objects as dictionary values
7 operations = { "add": add, "sub": subtract, "mul": multiply, "div": divide, "pow": power }
8 # Dynamic execution based on user key
9 op_name = "pow"
10 x, y = 2, 5
11 result = operations[op_name](x, y)
12 # Retrieves multiply and calls multiply(9, 5)
13 print(f"Result a power b: {result}")
14 op_name = "sub"
15 x, y = 9, 5
16 result = operations[op_name](x, y)
17 print(f"Result a-b: {result}")
```

Run Function in Structure

C:\Users\HP\PycharmProjects\PythonProject2\.venv\Scripts\python.exe "C:\Users\HP\PycharmProjects\PythonProject2\PythonProject2.py"

Result a power b: 32  
Result a-b: 4

Process finished with exit code 0

### 3. Function as argument to another function

You can pass objects as arguments to a function. Since functions are objects in Python, they can also be passed as arguments to functions. This technique allows you to apply multiple operations to a particular set of inputs. Given below is an example of a function calling another function as an argument, where the function named `add` is passed as an argument to the function `display`.

Example to show a function calling another function as an argument :

```
def add(n):
 sum=0
 for i in n:
 sum=sum+i
 return sum

def display(l,func):
 print("Addition of elements:",func(l))

display([1,2,3,4,5,6,7],add) # function display() calling function add()
```

===== RESTART: C:/Users/test/Desktop/python\_programs/return\_example.py =====  
Addition of elements: 28  
>>>

## 4. Nested function

Functions created within another function are called nested functions. As in Python, functions are objects. The inner function, which is created within some outer function, has access to all the variables of the enclosing scope. Inner functions can never be directly called outside the outer function. Hence, it provides a security feature called Encapsulation. Inner functions are hidden from the global scope ( provide encapsulation). They can only be called from within the enclosing function.

Example: Nested function

```
def outer(text): # outer function
 print(str.upper(text))

 def inner(): # nested or inner function
 print(str.lower(text))

 inner() # nested function can only be called inside outer function

outer("Nested Function") # call to outer function makes indirect call to inner
inner("Nested Function") # shows error: nested function cannot be called outside
```

==== RESTART: C:/Users/test/Desktop/python\_programs/return\_example.py =====

```
NESTED FUNCTION
nested function
Traceback (most recent call last):
 File "C:/Users/test/Desktop/python_programs/return_example.py", line 12, in <module>
 inner("Nested Function") # shows error: nested function cannot be called outside
NameError: name 'inner' is not defined
>>>
```

In the above program, as the inner function's name is bound only within the local namespace of the outer function, calling it directly from the outside results in a NameError:

You should use an inner function when a function requires a small helper routine that is only useful inside that specific function; defining it as an inner function keeps it hidden from the rest of your program. Also, if a multi-step function repeats a 3-to-4 line operation multiple times internally, you can define an inner function that prevents duplicating code without exposing the operation globally.

---

## 4.7 LAMBDA / ANONYMOUS FUNCTIONS

---

Python allows you to create unnamed functions called anonymous *functions*. They are not declared with the *def* keyword; the *lambda* keyword is used to create these functions. Hence, these functions are also called lambda functions. Like other functions, these functions can also contain arguments, but there can be *only one statement* in the body of these functions. The statement is evaluated, and the result is returned. When you need a function for a short operation (similar to macros in the C language) or only for a short time, lambda functions are best suited.

```
Variable = lambda arg : statement
```

Here, arg is one or more arguments passed to a function, and statement consists of the operation performed by a function.

Below is an example in which two lambda functions are created. The first function calculates the cube of a given argument, and the second function adds two given arguments. The lambda function is then assigned to a variable (a function assigned to a variable, discussed in the previous section), which can then be used to call the function with arguments and parentheses.

Example to show use of Lambda functions:



The screenshot shows a Python IDE with three tabs: 'Modified Class Example.py', 'Funtion inStructure.py', and 'LambdaFn.py'. The 'LambdaFn.py' tab is active, displaying the following code:

```
1 divide = lambda a, b : a / b if b != 0 else "Error"
2 power = lambda a, b : a ** b
3 print("a/b:", divide(a: 20, b: 5))
4 print("a power b:", power(a: 3, b: 4))
5
```

Below the code editor, the 'Run' button is clicked, and the output is displayed in the console:

```
C:\Users\HP\PycharmProjects\PythonProject2\.venv\Scripts\python.exe C:\Users\HP\Py
a/b: 4.0
a power b: 81
Process finished with exit code 0
```

Lambda functions are mostly used as arguments to other higher-order functions like `map()` and `filter()`.

**map() function:** `map()` is a built-in function which takes two arguments; the first argument given is a function (can be a lambda function), and the second argument is given as a list. The `map()` function returns a list of elements obtained by applying the given function to each element of a list.

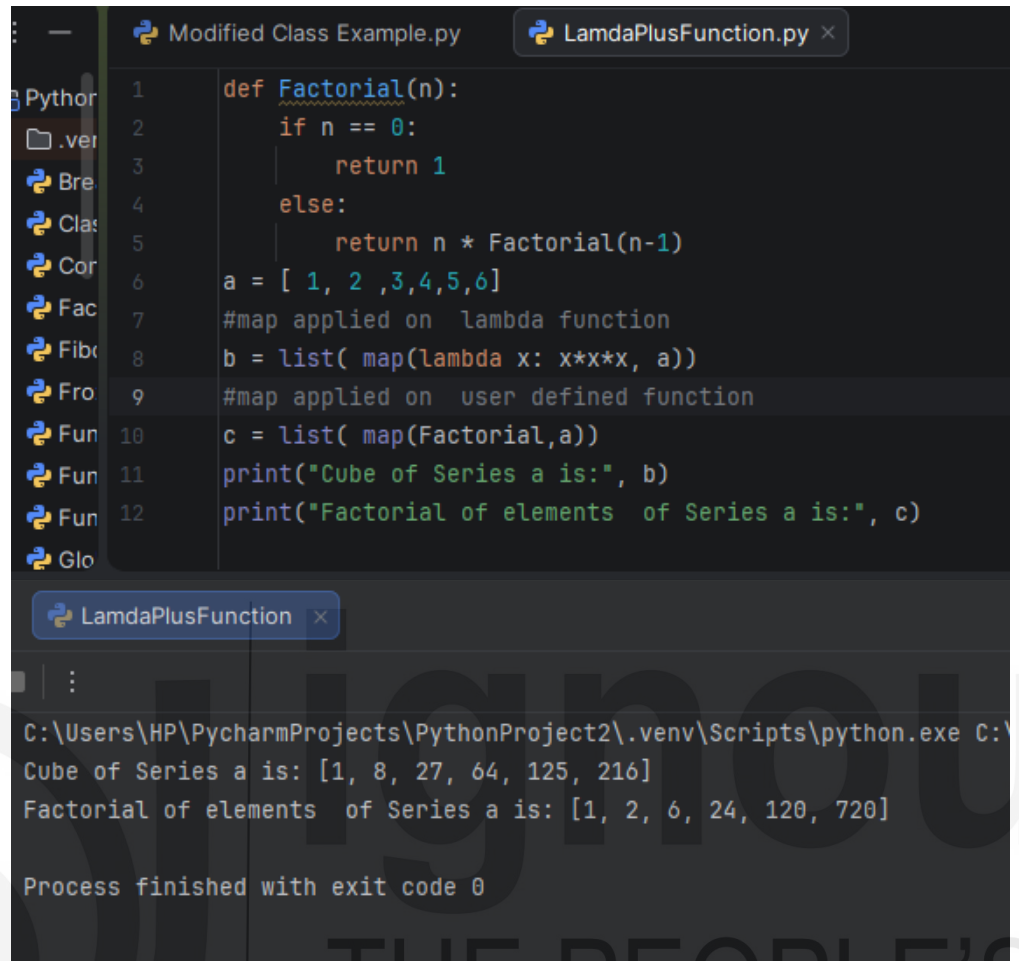
**filter() function:** `filter()` is another built-in function which takes two arguments. The first argument is a function, and the second is a list of elements, just as the `map()` function. Here, the given function is applied to each of the elements of the list and returns a list of items for which the function evaluates to True.

Syntax of map and filter functions

```
map (function , [list])
```

```
filter (function , [list])
```

Example of application of map() function with lambda function and user-defined function:

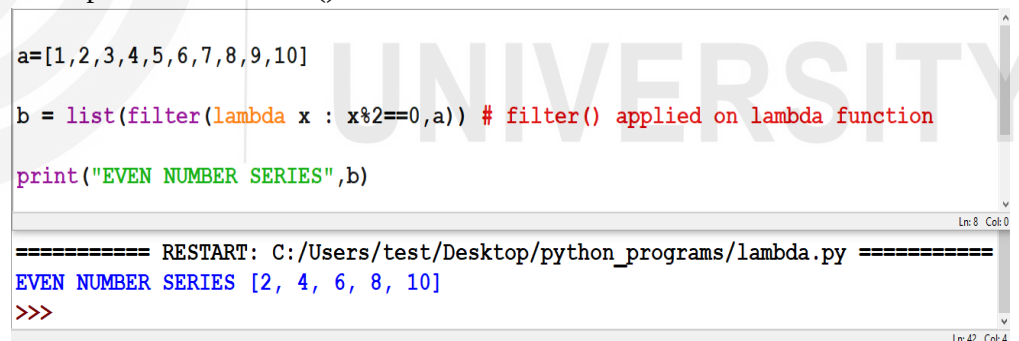


```
1 def Factorial(n):
2 if n == 0:
3 return 1
4 else:
5 return n * Factorial(n-1)
6 a = [1, 2 ,3,4,5,6]
7 #map applied on lambda function
8 b = list(map(lambda x: x*x*x, a))
9 #map applied on user defined function
10 c = list(map(Factorial,a))
11 print("Cube of Series a is:", b)
12 print("Factorial of elements of Series a is:", c)
```

Output:

```
C:\Users\HP\PycharmProjects\PythonProject2\.venv\Scripts\python.exe C:\Users\HP\PycharmProjects\PythonProject2\lambda_plus_function.py
Cube of Series a is: [1, 8, 27, 64, 125, 216]
Factorial of elements of Series a is: [1, 2, 6, 24, 120, 720]
Process finished with exit code 0
```

Example of use of filter() function with lambda function



```
a=[1,2,3,4,5,6,7,8,9,10]
b = list(filter(lambda x : x%2==0,a)) # filter() applied on lambda function
print("EVEN NUMBER SERIES",b)
```

Output:

```
===== RESTART: C:/Users/test/Desktop/python_programs/lambda.py =====
EVEN NUMBER SERIES [2, 4, 6, 8, 10]
>>>
```

## Check your Progress – 2

1. Write a function multiply\_all(\*numbers) which multiplies all the arguments and returns its value.

.....

.....

.....

.....

2. What are the various ways of passing arguments to a function?

.....

.....

.....

.....

3. Write a function which takes a list of numbers as an argument and returns a list of unique elements from it.

.....

.....

.....

.....

4. Write a program to find the cube of numbers in a list using a lambda function.

.....

.....

.....

.....

5. Write a program to add two given lists using the map function.

.....

.....

.....

.....

6. Write a program to display vowels from a given list of characters using the filter function.

.....

.....

.....

.....

---

## 4.8 FILE OPERATIONS

---

Main memory is volatile; hence, nothing can be stored permanently in it. To store the information permanently, files are used. You can permanently store data in non-volatile memory (e.g. hard disk, pen drive) using a file.

A file is a named location on disk to store related information. File handling is an important feature provided by all languages to address this need. You can take input from files instead of from users; you can also save output permanently in files for later access. You can append and update a file. All these functionalities come under file handling. Like other languages, Python also provides various built-in functions for file handling operations.

In Python, when you are programming, a “file” refers to a dedicated Python file object. This object is used to interact with data stored on your hard drive (like .txt, .csv, or other files). File handling in Python involves opening, reading, writing, and closing files, enabling the management and manipulation of data stored in files.

Python supports two types of files: binary and text.

**Binary files** – These files can be represented as 0's and 1's. As you know, everything in a computer is saved as 0s and 1s. Applications can process these files if they know the file's structure. Image files are an example of binary files. More examples of binary files include: .png, .pdf, .mp3, .exe, .dat. For these files, Python leaves the raw bytes as-is (bytes object). It does not try to parse them into letters or lines of text, because doing so would corrupt non-text data like images, compressed archives, or executables.

**Text files** – These files are organised as a sequence of characters. Here, each line is separated by a special end-of-line character. Some examples of text files are: .txt, .py, .csv, .log . Python interprets the underlying bytes as human-readable text using a character encoding such as: UTF-8 or ASCII. When you read them, Python converts the raw bytes into a string (str).

Any file handling operation can be performed in three steps-

1. Opening a file
2. Operating on the file – read, write, append, etc
3. Closing a file

#### Opening a file

A file can be opened in Python using the built-in function `open()`. By default, files are opened in read text ('r') file mode.

Syntax of `open()` function

```
file = open ("file_name.ext ", "mode_of_operation")
```

Where,

`File_name.ext` - is the name of the file to be opened, and `ext` is the extension of the file.

`Mode_of_operation` - is the mode in which the file needs to be opened.

As a Python file is an object returned by the function. You can use this object for further operations on the file. The possible modes of operation in Python are:

| Mode of opening file | Functionality                                                                                                 |
|----------------------|---------------------------------------------------------------------------------------------------------------|
| 'r'                  | Opens a file for reading. It is the default mode                                                              |
| 'w'                  | Opens a file for writing. Overwrites a file if it already exists.<br>Creates a new file if it does not exist. |
| 'a'                  | Opens a file for appending.                                                                                   |
| 'r+'                 | Opens a file for both reading and writing.                                                                    |

|      |                                                                                      |
|------|--------------------------------------------------------------------------------------|
| 'w+' | Same as 'r+' but creates a new file if it does not exist and overwrites if it exists |
| 'a+' | Opens a file for appending and reading                                               |
| 'x'  | Creates a new file. Fails if already exists. Added in Python 3.                      |
| 'rb' | Opens a file for reading in binary mode.                                             |
| 'wb' | Same as 'w' except data is in binary                                                 |
| 'ab' | Same as 'a' except data is in binary                                                 |

Example of opening a file in different mode:



```

1 # 1. Relying on default behavior: file is opened in read text ('r') file mode |
2 with open("Test.txt") as file:
3 filecontent = file.read()
4 print(filecontent)
5 # 2. Explicitly setting read mode
6 with open("Test.txt", "r") as file:
7 filecontent = file.read()
8 print(filecontent)
9 # 3. Explicitly setting read + text mode
10 with open("Test.txt", "rt") as file:
11 filecontent = file.read()
12 print(filecontent)
13 file.close()
14 print("File is closed")

```

FileHandling x FileHandling x

C:\Users\DELL\PycharmProjects\PythonProject\.venv\Scripts\python.exe C:\Users\DELL\PycharmF

Let us test the file operation  
Let us test the file operation  
Let us test the file operation  
File is closed

Process finished with exit code 0

You can see in the above program that the file named “Test.txt” is opened, and here, because of its safety and readability, the with statement is used. Use of *with* is considered very good practice for file handling in modern Python. Various file-handling operations are performed.

### Reading from a file

There are many ways to read from a file. The functions below are available for reading from a file. To read data from a file, it must exist. In the above program, the `filenameread()` function is used to read a file named “Test.txt” in the directory files within the present directory.

In the table below, it is assumed that file is the object returned from the open() function.

| Function    | Syntax           | Description                                                     |
|-------------|------------------|-----------------------------------------------------------------|
| read()      | file.read ()     | Returns entire file contents as a string                        |
|             | file.read (n)    | Returns n characters from beginning of file as string           |
| readline()  | file.readline()  | Returns single line of file at a time. First line in this case. |
| readlines() | file.readlines() | Returns a list of all lines                                     |

### Writing to a file

Similar to read operations, Python provides functions to write data to a file.

| Function | Syntax             | Description                     |
|----------|--------------------|---------------------------------|
| write()  | file.write(“text”) | Writes text or string to a file |

### Some more Operations on files

Python allows many other file operations. Listed below are some functions used for different file-handling operations.

| Function          | Syntax                     | Description                                                                                                        |
|-------------------|----------------------------|--------------------------------------------------------------------------------------------------------------------|
| tell()            | file.tell()                | Returns the current cursor position in file                                                                        |
| seek()            | file.seek(pos)             | Places the file cursor to the position pos                                                                         |
| os.stat()         | os.stat(filename)          | This method is used to get the display status of the file given as an argument.                                    |
| os.path.exists()  | os.path.exists(‘arg’)      | Returns true if the file or directory of name ‘arg’ exists                                                         |
| os.path.isfile()  | os.path.isfile(‘arg’)      | Returns true if the file of name ‘arg’ exists                                                                      |
| os.path.isdir()   | os.path.isdir(‘arg’)       | Returns true if the directory of name ‘arg’ exists                                                                 |
| shutil.copy()     | shutil.copy(src,dst)       | This function is part of Python’s built-in shutil (Shell Utilities) module. It copies a file from src to des file. |
| os.path.getsize() | os.path.getsize (filename) | Returns the size of the file                                                                                       |

### Closing a file

After completing all operations on a file, you must close it properly. This step frees up the resources and allows graceful termination of file operations.

```
file.close()
```

In Python, `import` is a keyword used to bring built-in or external code libraries (called modules) into your script. You will learn more about `import` in the coming units of this course. Let us look at an example program on Python File handling. In this program, we will *import* `os` and `time` (two essential Python Standard Library modules). These modules come built into Python, so you do not need to install them separately. The following Python program creates a file and writes a few lines in it. Then it reads the file's contents and displays them. It checks the file size, absolute path, and creation date and time. Then it appends more content to the file and rechecks the file size.

```
import os
import time
Creating the file name = "My.txt"
print("---- 1. Creating FILE and WRITING few lines ----")
Open file in write mode ('w')
with open("My.txt", "w", encoding="utf-8") as file:
 file.write("Line 1: Welcome to Python Programming.\n")
 file.write("Line 2: This course is dedicated to Python.\n")
 file.write("Line 3: The course code of this course is MCS-082.\n")
print(f"Successfully created and written to '{'My.txt'}'.\n")
print("---- 2. Reading FILE content ----")
Now open file in read mode ('r')
with open("My.txt", "r", encoding="utf-8") as file:
 content = file.read()
 print("File Contents:")
 print(content)
print("---- 3. Checking FILE Size & modification details ----")
1. File existence check
if os.path.exists("My.txt"):
 print(f"File exists: Yes")
2. File size in bytes
file_size = os.path.getsize("My.txt")
print(f"File Size: {file_size} bytes")
3. Absolute path
abs_path = os.path.abspath("My.txt")
print(f"Absolute Path: {abs_path}")
4. Last modification time
mod_time = os.path.getmtime("My.txt")
readable_time = time.ctime(mod_time)
print(f"Last Modified: {readable_time}")
print("\n--- 4. Append more content to FILE ---")
Open file in append mode ('a') to add lines without overwriting
with open("My.txt", "a", encoding="utf-8") as file:
 file.write("Line 4: This line we have appended later.\n")
Re-checking size after append
new_size = os.path.getsize("My.txt")
print(f"Updated File Size: {new_size} bytes")
file.close()
```

After executing this file, you will get:

```
C:\Users\HP\PycharmProjects\PythonProject2\.venv\Scripts\python.exe C:\Users\HP\
--- 1. Creating FILE and WRITING few lines ---
Successfully created and written to 'My.txt'.

--- 2. Reading FILE content ---
File Contents:
Line 1: Welcome to Python Programming.
Line 2: This course is dedicated to Python.
Line 3: The course code of this course is MCS-082.

--- 3. Checking FILE Size & modification details ---
File exists: Yes
File Size: 137 bytes
Absolute Path: C:\Users\HP\PycharmProjects\PythonProject2\My.txt
Last Modified: Tue Aug 11 14:32:45 2026

--- 4. Append more content to FILE ---
Updated File Size: 180 bytes

Process finished with exit code 0
```

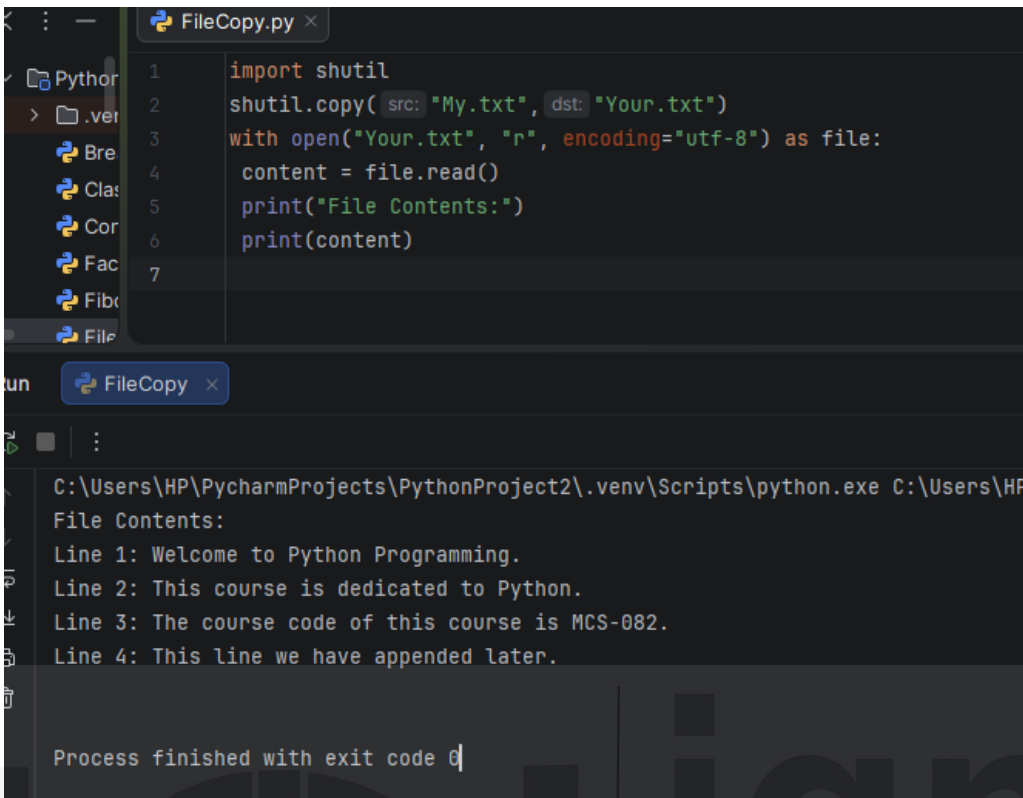
You can read the final content of the file and display the result as a practice exercise in the program above.

Note that the file we want to read or write may be in another directory or folder. In that case, we must provide the file's absolute or relative path along with its name. The path separated by \ must be preceded by one more \ (backslash). If the file to be read or written is in the same folder in which the Python program is, then we need not give its path.

Some file-handling methods may not be safe. An exception may occur during file operations, and the code may exit without closing the file. To make this easier, Python uses a statement called *with*, which ensures the file closes when the code within the *with* block exits. You don't need to call the `close()` function explicitly. You can see the use of *with* in the program given on the previous page.

### Copying a file

To copy a file from one place to another, Python offers several methods. One such way of copying a file is by importing a module named 'shutil', which contains a built-in function to create a copy of a file. You can see in the following program where the file "My.txt", which we created in the previous example, is copied to a file named "Your.txt". You may also notice that the content of "Your.txt" matches the content we kept in "My.txt" after appending.



```

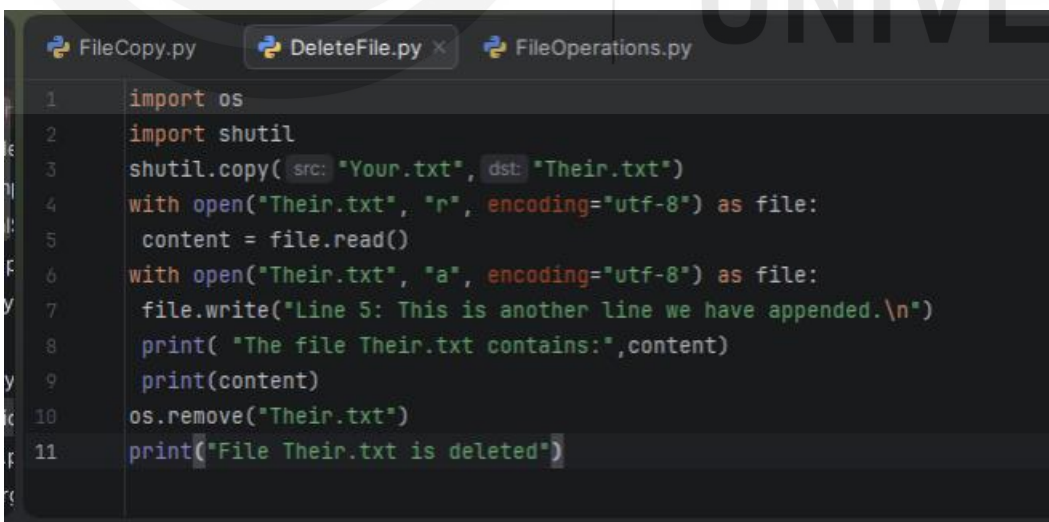
FileCopy.py
1 import shutil
2 shutil.copy(src="My.txt", dst="Your.txt")
3 with open("Your.txt", "r", encoding="utf-8") as file:
4 content = file.read()
5 print("File Contents:")
6 print(content)
7
run FileCopy
C:\Users\HP\PycharmProjects\PythonProject2\.venv\Scripts\python.exe C:\Users\HP
File Contents:
Line 1: Welcome to Python Programming.
Line 2: This course is dedicated to Python.
Line 3: The course code of this course is MCS-082.
Line 4: This line we have appended later.

Process finished with exit code 0

```

## Deleting a file or folder

A file can be deleted using the `remove()` function from the `os` module. Before using this function, you should move to the directory in which the file to be removed exists. Similarly, to delete a folder or directory, you can use the `os.rmdir()` function. The directory to be removed must be empty; otherwise an error will be shown. In the example given below, we want to delete a file named “Their.txt”. You can use the `os.listdir()` function to display the list of files present in a directory.



```

FileCopy.py DeleteFile.py FileOperations.py
1 import os
2 import shutil
3 shutil.copy(src="Your.txt", dst="Their.txt")
4 with open("Their.txt", "r", encoding="utf-8") as file:
5 content = file.read()
6 with open("Their.txt", "a", encoding="utf-8") as file:
7 file.write("Line 5: This is another line we have appended.\n")
8 print("The file Their.txt contains:", content)
9 print(content)
10 os.remove("Their.txt")
11 print("File Their.txt is deleted")

```

When you execute the above program, you will get:

```
C:\Users\HP\PycharmProjects\PythonProject2\.venv\Scripts\python.exe C:\Users\HP
The file Their.txt contains: Line 1: Welcome to Python Programming.
Line 2: This course is dedicated to Python.
Line 3: The course code of this course is MCS-082.
Line 4: This line we have appended later.

Line 1: Welcome to Python Programming.
Line 2: This course is dedicated to Python.
Line 3: The course code of this course is MCS-082.
Line 4: This line we have appended later.

File Their.txt is deleted

Process finished with exit code 0
```

---

### Check your Progress – 3

1. Write a program to display the frequency of each word in a given file.

.....

.....

.....

.....

2. Write a program to display the first n lines from a given file, where n is given by the user.

.....

.....

.....

.....

3. Write a program to display the size of a file in bytes.

.....

.....

.....

.....

---

## 4.9 SUMMARY

This unit explores fundamental Python programming concepts, with greater emphasis on modular code design through functions. It covers how to define and call functions. Also, explained variable scope and the distinction between local and global visibility. Additional key topics include passing arguments to functions and writing concise Lambda expressions. Finally, the unit explains file handling in Python by demonstrating how to create, open, read, write, and safely manage files to ensure persistent program data.

---

## 4.10 SOLUTIONS TO CHECK YOUR PROGRESS

---

### Answer to Check Your Progress-1

1. The various benefits of using functions are –

- i) It decomposes a larger problem into more manageable units and helps in eliminating repetitive code blocks
- ii) It allows reusability of code
- iii) Break complex problems into smaller, isolated, manageable chunks and reduce duplication
- iv) Hide complex underlying algorithms( implementation) behind a clear, descriptive function name and make the code self-documenting.
- v) It makes code easy to understand, use and debug

The process of defining a function is called *function definition*. It describes how a function works. It includes – function name, list of arguments and function body.

```
Syntax def function_name():
 # body of function

```

Even if a function is defined, it cannot be executed until it is called. Hence, for using a function, it must be called using *function call statement*. Calling of a function includes the function name and a list of arguments (no function body).

Syntax:

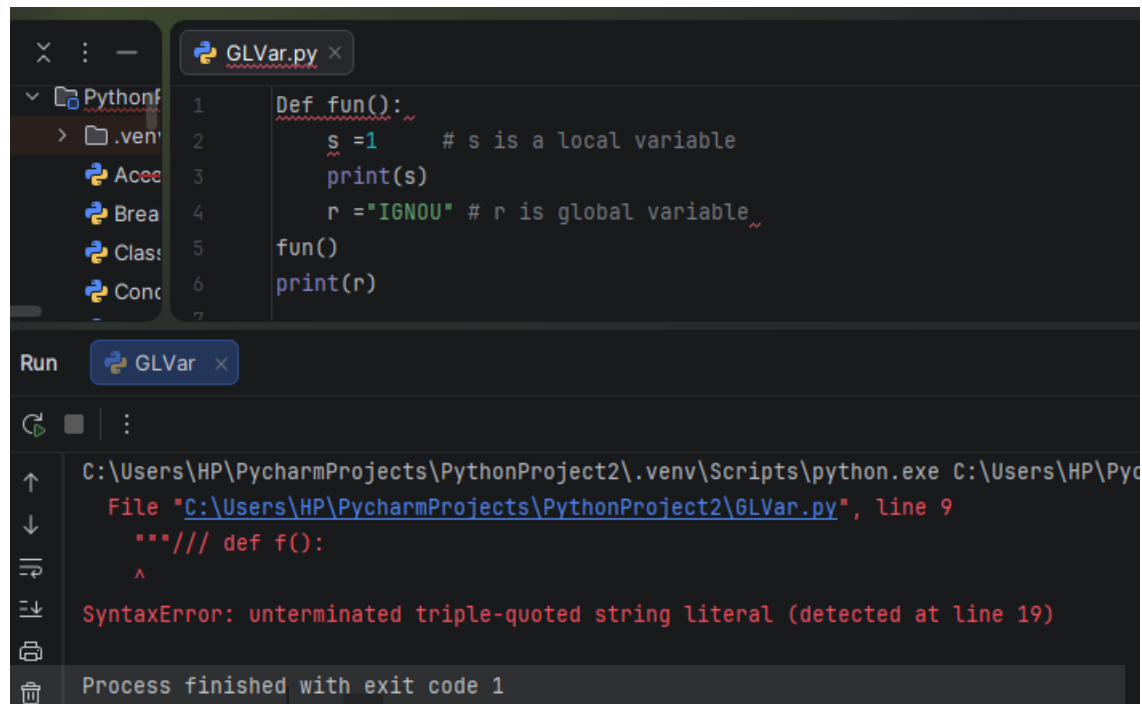
```
function_name(list of arguments)
```

2. The scope of a variable refers to the region of a program where that variable is recognized and accessible. Part of a code in which a variable can be accessed is called *the scope of a variable*.

**Local variables:** These are variables created within a function's body or function's scope. You cannot access them outside the function. Their scope is only limited to the function in which they are created.

**Global variables:** These are variables created outside any function. Their scope is global, hence can be used anywhere. Also, you can create global variables within a function using the global keyword.

Example:



```
1 def fun():
2 s = 1 # s is a local variable
3 print(s)
4 r = "IGNOU" # r is global variable
5 fun()
6 print(r)
```

Run GLVar

C:\Users\HP\PycharmProjects\PythonProject2\.venv\Scripts\python.exe C:\Users\HP\PycharmProjects\PythonProject2\GLVar.py, line 9

""" def f():

^

SyntaxError: unterminated triple-quoted string literal (detected at line 19)

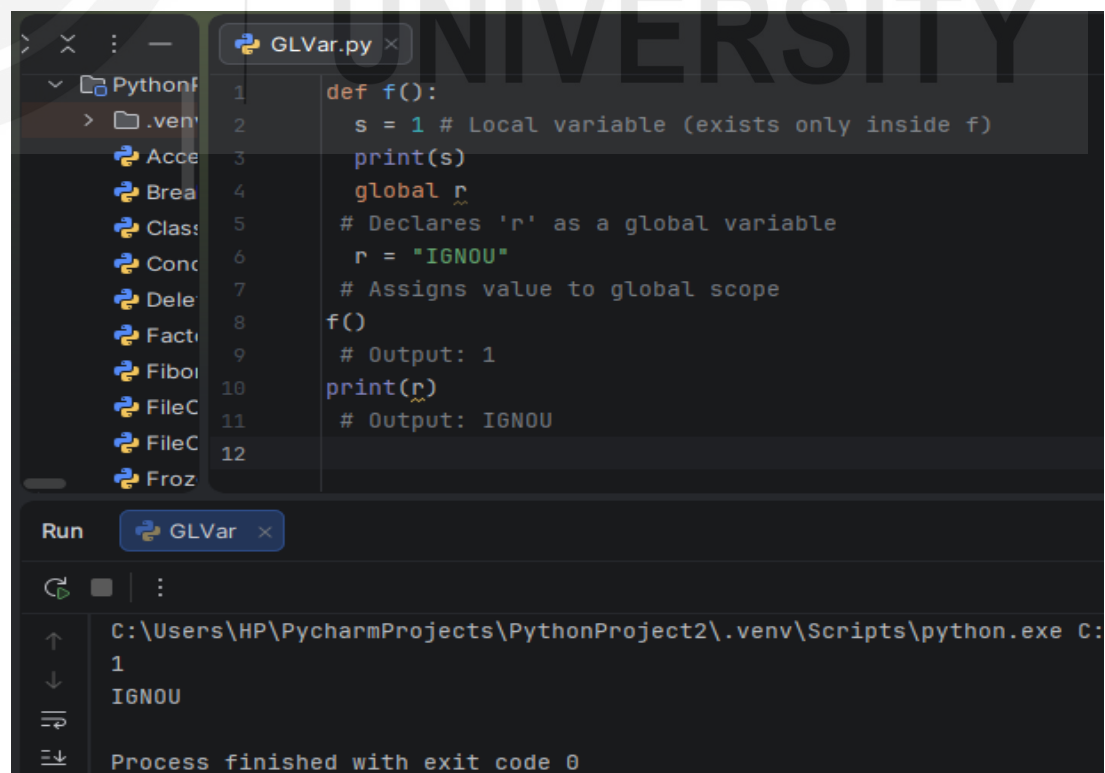
Process finished with exit code 1

#### Code Correction and Explanation:

The global keyword is used inside a function to declare that a variable belongs to the global scope. It lets you create or modify a global variable from within the function.

In the code snippet above, `r = "IGNOU"` is a local variable because the global keyword was not used before assignment. Calling `print(r)` outside the function raised a `NameError: name 'r' is not defined`.

Below is the corrected code with proper Python syntax and declarations:



```
1 def f():
2 s = 1 # Local variable (exists only inside f)
3 print(s)
4 global r
5 # Declares 'r' as a global variable
6 r = "IGNOU"
7 # Assigns value to global scope
8 f()
9 # Output: 1
10 print(r)
11 # Output: IGNOU
```

Run GLVar

C:\Users\HP\PycharmProjects\PythonProject2\.venv\Scripts\python.exe C:\Users\HP\PycharmProjects\PythonProject2\GLVar.py

1

IGNOU

Process finished with exit code 0

- Without global r, r remains restricted to f() and is destroyed when f() finishes executing.
- Adding global r binds r to the global namespace, making it accessible outside the function call.

3.



```

1 def pattern():
2 for i in range(1, 6):
3 for j in range(1, i + 1):
4 print(j, end= " ")
5 print()
6 pattern()

```

Output:

```

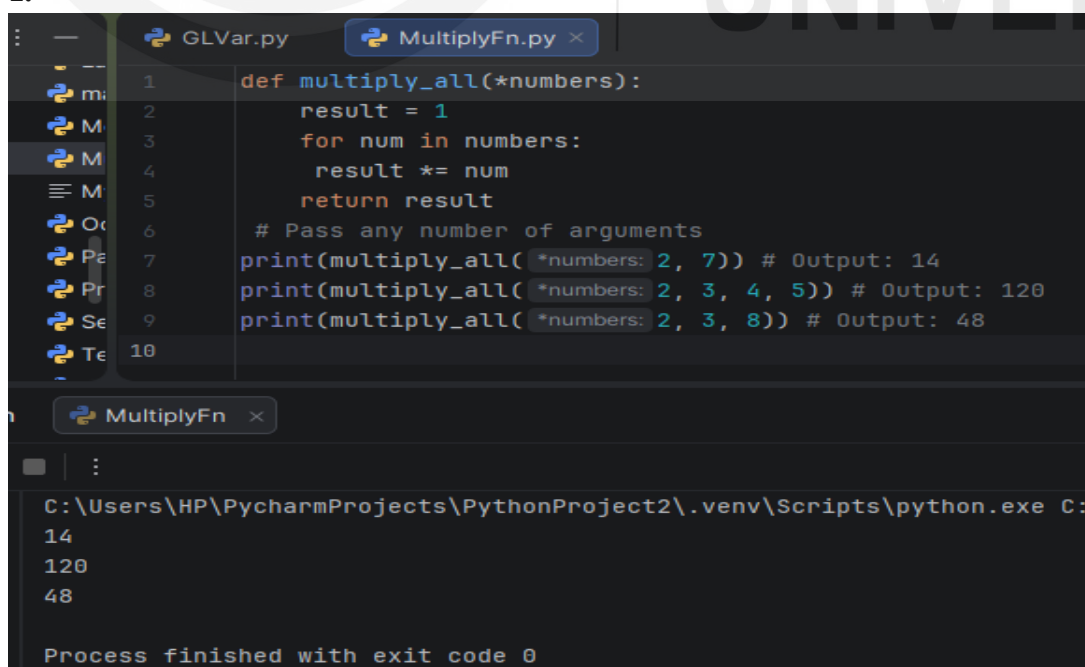
1
1 2
1 2 3
1 2 3 4
1 2 3 4 5

```

Process finished with exit code 0

## Answer to Check your Progress 2

1.



```

1 def multiply_all(*numbers):
2 result = 1
3 for num in numbers:
4 result *= num
5 return result
6 # Pass any number of arguments
7 print(multiply_all(*numbers: 2, 7)) # Output: 14
8 print(multiply_all(*numbers: 2, 3, 4, 5)) # Output: 120
9 print(multiply_all(*numbers: 2, 3, 8)) # Output: 48
10

```

Output:

```

14
120
48

```

Process finished with exit code 0

2. Python provides various mechanisms for passing arguments to a function.

1. Required arguments
2. Default arguments
3. Keyword arguments
4. Arbitrary arguments

3.

```
def unique(numbers):
 out = []
 for i in numbers:
 if i not in out:
 out.append(i)
 return out

ans=unique([1,2,1,1,2,2,3,4,6,3])
print(ans)
```

4.

```
nums = [1, 2, 3, 4, 5]
print("\nCub of numbers:")
cube = list(map(lambda x: x ** 3, nums))
print(cube)
```

5.

```
a = [1, 2, 3]
b = [10, 20, 6]

result = map(lambda x, y: x + y, a, b)
print("\nAddition of two list:")
print(list(result))
```

6.

```
def vowel(a):
 v=['a','e','i','o','u']
 if a in v:
 return a

l=['a','b','f','o','x','z','y']
print("LIST OF VOWELS:")
print(list(filter(vowel,l)))
```

### Answer to Check your Progress 3

1.

```
from collections import Counter
def wordcount(fname):
 with open(fname) as f:
 return Counter(f.read().split())

print("Frequency of words:",wordcount("abc.txt"))
```

2.

```
n=int(input("enter number of lines:"))
c=1
file=open("test.txt","r")
for i in file:
 if c<=n:
 print(i)
 c+=1
file.close()
```

3.

```
def fsize(fname):
 import os
 status = os.stat(fname)
 return status.st_size

print("File size in bytes: ",fsize("test.txt"))
```

---

## 4.11 REFERENCES/FURTHER READINGS

---

1. Yashavant Kanetkar and Aditya Kanetkar, “Let Us Python”, 4<sup>th</sup> Edition, BPB Publications, New Delhi , 2022
2. Paul Deitel and Harvey Deitel , “Python For Programmers” , 1<sup>st</sup> Edition, Pearson India Education , Chennai, 2023
3. Sheetal Taneja and Naveen Kumar, “Python Programming a Modular Approach” , 1<sup>st</sup> Edition, Pearson India Education , Chennai , 2022
4. S. Shridhar, J. Indumathi and V.M. Hariharan , “Python Programming” , 1<sup>st</sup> Edition, Pearson India Education , Chennai, 2024
5. S. Suma Christal Mary , R. Josphine Leela, N. Senthamilarasi and Y. Chrisbin Jeeva , “Problem Solving and Python Programming”, 1<sup>st</sup> Edition, Ane Books , New Delhi, 2018